

VOORWOORD

Deze cursusnota's horen bij het opleidingsonderdeel *Numerieke Lineaire Algebra* uit de derde bachelor Informatica. De structuur van de cursus is erg gelijklopend aan de eerste vier delen van het boek

Numerical Linear Algebra
L.N. Trefethen en D. Bau
SIAM 1997, ISBN-13: 9780898714876

en heel wat begrippen worden dan ook op analoge wijze in deze nota's geïntroduceerd.

Benny Van Houdt
September 2021

Contents

VECTOR- EN MATRIXNORMEN	6
1.1 Vectornormen	6
1.2 Vector geïnduceerde matrixnormen	7
1.3 Algemene matrixnormen	10
1.4 Oefeningen	12
QR FACTORISATIE	14
2.1 Orthogonale vectoren en unitaire matrices	14
2.2 Definitie	17
2.2.1 Gereduceerde QR factorisatie	17
2.2.2 Volledige QR factorisatie	18
2.3 Existentie	19
2.4 Algoritmes	21
2.4.1 Modified Gram-Schmidt	21
2.4.2 Householder reflecties	23
2.4.3 Givens rotaties	29
2.5 Toepassingen	31
2.5.1 Oplossen van lineaire stelsels	31
2.5.2 Kleinste kwadraten	32
2.5.3 Hessenberg decompositie	34
2.6 Oefeningen	35
SINGULIERE WAARDEN ONTBINDING	38
3.1 Definitie en geometrische interpretatie	38
3.1.1 Gereduceerde SVD	39
3.1.2 Volledige SVD	40
3.2 Existentie	41
3.3 Eigenschappen	44
3.4 Toepassingen	46
3.4.1 Lage-rang approximaties	46

3.4.2	Kleinste kwadraten	48
3.4.3	Herkennen van handgeschreven cijfers	50
3.5	Berekenen van de SVD	53
3.5.1	Golub-Kahan bidiagonalisatie	54
3.6	Oefeningen	56
CONDITIONERING EN STABILITEIT		60
4.1	Conditiegetallen	61
4.1.1	Definitie en voorbeelden	61
4.1.2	Conditiegetal van een matrix	64
4.2	Stabiliteit	66
4.2.1	Floating point rekenen en de machineprecisie	67
4.2.2	Definitie	68
4.2.3	Achterwaartse stabiliteit	69
4.2.4	Eenvoudige voorbeelden	70
4.2.5	Achterwaartse substitutie	73
4.2.6	Matrixvermenigvuldiging	77
4.2.7	Householder triangularisatie	79
GAUSS ELIMINATIE		83
5.1	LU factorisatie	83
5.2	Oplossen van lineaire stelsels	87
5.3	Pivotering	89
5.3.1	Permutatiematrices	90
5.3.2	Gauss eliminatie met gedeeltelijke pivotering	91
5.3.3	Gauss eliminatie met complete pivotering	95
5.3.4	Stabiliteit	95

NOTATIES

We starten met enkele afspraken te maken met betrekking tot de notaties gehanteerd in deze cursusnota's. Vele gelijkaardige notaties kwamen ook reeds voor in de cursus Lineaire Algebra, met dat verschil dat we ons hier niet zullen beperken tot de verzameling $\mathbb{R}$ van de reële getallen, maar ook complexe getallen beschouwen.

Een complex getal heeft de vorm $z = a + bi$ met $a, b \in \mathbb{R}$ en $i^2 = -1$, waarbij we a het reële deel en b het imaginaire deel van z noemen. De verzameling van de complexe getallen noteren we als $\mathbb{C}$ en het complex toegevoegde z^* van een complex getal $z = a + bi$ is gelijk aan $a - bi$. De *absolute waarde* of *modulus* van een complex getal $z = a + bi$, aangegeven door $|z|$, wordt gedefinieerd als:

$$|z| = \sqrt{a^2 + b^2} = \sqrt{(a - bi)(a + bi)} = \sqrt{z^* z}.$$

Deze waarde kan worden gevisualiseerd als de lengte van de vector z in het complexe vlak (deze wordt berekend met behulp van de stelling van Pythagoras). Merk op, indien z een reëel getal is, dan is $|z|$ de vertrouwde absolute waarde van z . We kunnen z ook steeds schrijven als $z = |z|(\cos \theta + i \sin \theta)$, met $\theta \in [0, 2\pi)$. In dit geval noemen we θ het argument van z en we noteren dit als $\arg(z)$. Laat z^{-1} de inverse zijn van een complex getal z , wat wil zeggen dat $zz^{-1} = 1$, dan zien we dat $z^{-1} = |z|^{-1}(\cos \theta - i \sin \theta)$.

De verzameling van complexe (reële) matrices met m rijen en n kolommen noteren we als $\mathbb{C}^{m,n}$ ($\mathbb{R}^{m,n}$). Een kolomvector (rijvector) is een matrix die slechts één kolom (rij) bevat. De verzameling van kolomvectoren met lengte m noteren we als $\mathbb{C}^m$. Wanneer $A \in \mathbb{C}^{m,n}$ dan noteren we het element van A dat voorkomt op rij i en kolom j als a_{ij} en noteren we de j -de kolom van A als a_j en de i -de rij van A als a_i^* (merk op, het j -de element van a_i^* is a_{ij} en niet zijn complex toegevoegde).

Wanneer $A \in \mathbb{C}^{m,n}$ dan zal het (i, j) -de element van de complex geconjugeerde matrix van A , die we noteren als A^* , gelijk zijn aan a_{ji}^* . Men kan eenvoudig aantonen dat $(A^*)^* = A$ en $(AB)^* = B^*A^*$. We noemen een matrix A Hermitisch indien $A = A^*$. Merk op, indien $A \in \mathbb{R}^{m,n}$, dan

is A^* gewoon de getransponeerde matrix. Reële Hermitische matrices zijn bijgevolg symmetrische matrices, d.i., matrices waarvoor $a_{ij} = a_{ji}$. Merk op, indien b een kolomvector is, d.i., $b \in \mathbb{C}^m$ dan zal b^* een rijvector zijn, vandaar ook de notatie a_i^* voor de i -de rij van A (met dat verschil dat het j -de element van b^* wel gelijk is aan b_j^*).

Het scalaire product x^*y van twee complexe vectoren $x, y \in \mathbb{C}^m$ definiëren we als

$$x^*y = \sum_{i=1}^m x_i^* y_i.$$

Wanneer x en y reële vectoren zijn, dan is $x^*y = \sum_{i=1}^m x_i y_i = y^*x$. Verder zien we ook dat $x^*x = \sum_{i=1}^m x_i^* x_i = \sum_{i=1}^m |x_i|^2$. Tot slot zullen we de eenheidsmatrix met m rijen en kolommen noteren als

$$I = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

en de i -de kolom van I noteren als e_i (waardoor e_i^* de i -de rij voorstelt).

VECTOR- EN MATRIXNORMEN

1.1 Vectornormen

DEFINITIE 1.1.1. Een afbeelding $\|\cdot\|$ van $\mathbb{C}^m$ naar $\mathbb{R}$ is een **vectornorm** indien volgende drie eigenschappen gelden, waarbij x en y vectoren in $\mathbb{C}^m$ zijn en $z \in \mathbb{C}$:

1. $\|x\| \geq 0$ en $\|x\| = 0$ als en slechts als $x = 0$,
2. $\|zx\| = |z| \|x\|$,
3. $\|x + y\| \leq \|x\| + \|y\|$.

De drie voornaamste vectornormen zijn de 1-, 2- en ∞ -norm en deze worden als volgt gedefinieerd:

1. $\|x\|_1 = \sum_{i=1}^m |x_i|$,
2. $\|x\|_2 = \sqrt{\sum_{i=1}^m |x_i|^2} = \sqrt{x^* x}$,
3. $\|x\|_\infty = \max_{i=1}^m |x_i|$,

waarbij de 2-norm overeenstemt met de klassieke Euclidisch norm zoals die ook werd gebruikt in de cursus Lineaire Algebra. Het is eenvoudig om na te gaan dat elk van deze drie normen inderdaad voldoet aan de definitie van een vectornorm (zie oefeningen).

Alle vectornormen zijn eveneens **equivalent** aan elkaar, hiermee bedoelen we dat we voor elke twee willekeurige normen $\|\cdot\|_\alpha$ en $\|\cdot\|_\beta$ steeds twee constanten $0 \leq k \leq K$ kunnen vinden zodat voor alle x

$$k \|x\|_\alpha \leq \|x\|_\beta \leq K \|x\|_\alpha.$$

Merk op, k en K zijn niet afhankelijk van x (zie oefeningen). Dit impliceert dat als een rij vectoren $(x_n)_{n \geq 0}$ convergeert naar de vector x^* in een bepaalde vectornorm, d.i.,

$$\lim_{n \rightarrow \infty} \|x_n - x^*\| = 0,$$

dan is dit eveneens het geval voor elke andere vectornorm.

1.2 Vector geïnduceerde matrixnormen

Met elke vectornorm $\|\cdot\|_\alpha$ kunnen we eveneens een matrixnorm associëren als volgt:

DEFINITIE 1.2.1. *Laat $\|\cdot\|_\alpha$ een vectornorm zijn en A een $m \times n$ matrix, dan definiëren we de **matrixnorm** $\|A\|_\alpha$ van A (geïnduceerd door $\|\cdot\|_\alpha$) als de kleinste reële waarde C zodat voor alle $y \in \mathbb{C}^n$ geldt*

$$\|Ay\|_\alpha \leq C \|y\|_\alpha,$$

wat overeenstemt met

$$\|A\|_\alpha = \sup_{y \in \mathbb{C}^n, y \neq 0} \frac{\|Ay\|_\alpha}{\|y\|_\alpha} = \sup_{x \in \mathbb{C}^n, \|x\|_\alpha = 1} \|Ax\|_\alpha.$$

Merk op, de tweede gelijkheid in deze definitie volgt uit de tweede eigenschap van een vectornorm daar

$$\frac{\|Ay\|}{\|y\|} = \left\| A \left(\frac{y}{\|y\|} \right) \right\| \quad \text{en} \quad \left\| \left(\frac{y}{\|y\|} \right) \right\| = \frac{\|y\|}{\|y\|} = 1.$$

Verder is A een continue afbeelding van $\mathbb{C}^n$ naar $\mathbb{C}^m$, waardoor het supremum in de definitie vervangen mag worden door een maximum (want het beeld van een gesloten verzameling is gesloten onder een continue afbeelding en de set van alle vectoren in $\mathbb{C}^n$ met norm één, d.i., de eenheidssfeer, is gesloten).

De matrixnorm geeft dus eigenlijk aan hoezeer een vector y uitgerokken kan worden door A . Merk op, indien de norm van A kleiner is dan één is de maximale uitrekking eigenlijk een minimale inkrimping. Voor een reële 2×2 matrix kunnen we de geïnduceerde matrixnorm ook grafisch weergeven. Bijvoorbeeld, laat

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix},$$

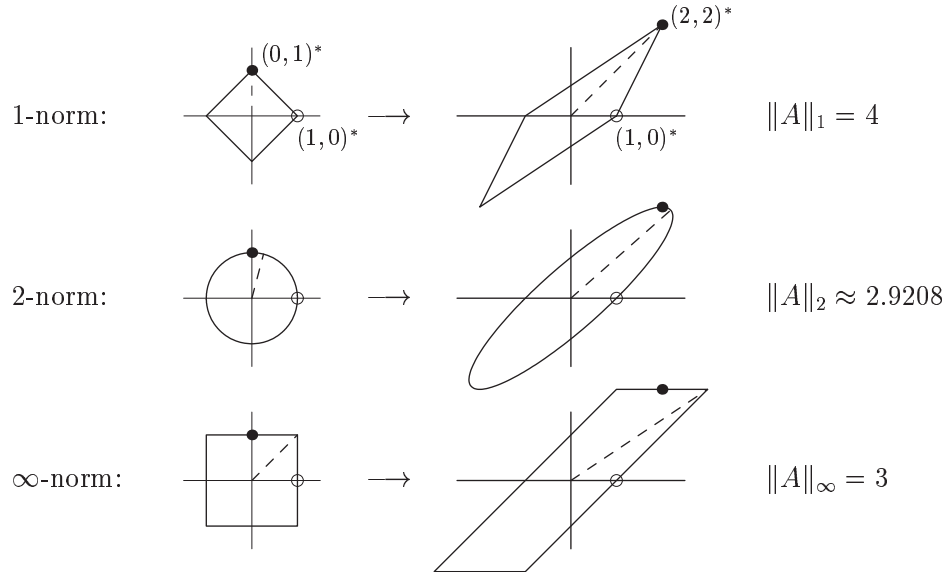


Figure 1.1: Links de eenheidssfeer in $\mathbb{R}^2$ onder de normen $\|\cdot\|_1$, $\|\cdot\|_2$ en $\|\cdot\|_\infty$, rechts hun beeld onder A .

dan zien we in Figuur 1.1 een grafische voorstelling van de matrixnorm van A voor de 1-, 2- en ∞ -norm. In dit voorbeeld heeft het beeld van de vector $e_2 = (0, 1)^*$ de grootste 1-norm, namelijk 4 wat eveneens de grootste kolomsom is van A . Het beeld van $(1, 1)^*$ heeft dan weer de grootste ∞ -norm, namelijk 3 wat gelijk is aan de grootste rijsum. Dat dit niet toevallig is blijkt uit volgende eigenschap:

EIGENSCHAP 1.2.1. *De 1-norm van een $m \times n$ matrix A is gelijk aan de grootste kolomsom, d.i.*

$$\|A\|_1 = \max_{j=1}^n \|a_j\|_1 = \max_{j=1}^n \sum_{i=1}^m |a_{ij}|,$$

waarbij a_j de j -de kolom van A weergeeft.

Bewijs. Laat $x \in \mathbb{C}^n$ en $\|x\|_1 = \sum_{j=1}^n |x_j| \leq 1$ dan

$$\begin{aligned} \|Ax\|_1 &= \left\| \sum_{j=1}^n x_j a_j \right\|_1 \leq \sum_{j=1}^n \|x_j a_j\|_1 \leq \sum_{j=1}^n |x_j| \|a_j\|_1 \\ &\leq \sum_{j=1}^n |x_j| (\max_{j=1}^n \|a_j\|_1) \leq \max_{j=1}^n \|a_j\|_1 \left(\sum_{j=1}^n |x_j| \right) \leq \max_{j=1}^n \|a_j\|_1, \end{aligned}$$

waarbij we voor de eerste twee ongelijkheden gebruik hebben gemaakt van de derde en de tweede eigenschap van een vectornorm. Hieruit volgt dat $\|A\|_1 \leq \max_{j=1}^n \|a_j\|_1$. Indien we nu $x = e_j$, waar j zo gekozen is dat $\|a_j\|_1$ maximaal is, dan is $Ax = a_j$ en bijgevolg $\|Ax\|_1 = \|a_j\|_1 = \max_{j=1}^n \|a_j\|_1$. Dit bewijst de eigenschap. $\square$

EIGENSCHAP 1.2.2. *De ∞ -norm van een $m \times n$ matrix A is gelijk aan de grootste rijsum, d.i.*

$$\|A\|_\infty = \max_{i=1}^m \|a_i^*\|_1 = \max_{i=1}^m \sum_{j=1}^n |a_{ij}|,$$

waarbij a_i^* de i -de rij van A weergeeft.

Deze eigenschap kan eveneens zonder veel moeite aangetoond worden (zie oefeningen). De 2-norm van een matrix A is doorgaans moeilijker te berekenen:

EIGENSCHAP 1.2.3. *De 2-norm van een $m \times n$ matrix A is gelijk aan*

$$\|A\|_2 = \sqrt{\max_{i=1}^n \lambda_i(A^*A)},$$

waarbij $\lambda_i(A^*A)$ de i -de eigenwaarde van de matrix A^*A weergeeft.

Kortom, de 2-norm is de vierkantswortel van de grootste eigenwaarde van de Hermitische matrix A^*A (deze matrix is Hermitisch daar $(A^*A)^* = A^*(A^*)^* = A^*A$). We merken hierbij op dat alle eigenwaarden van een Hermitische matrix reëel zijn (zie oefeningen). Later in de cursus, in het hoofdstuk over de singuliere waarden ontbinding, zullen we eveneens aantonen dat de 2-norm van een matrix A gelijk is aan zijn grootste singuliere waarde σ_1 (zie Eigenschap 3.3.3), waaruit de bovenstaande eigenschap volgt via Eigenschap 3.3.5.

We eindigen deze sectie met volgende nuttige eigenschap voor matrixnormen geïnduceerd door een vectornorm:

EIGENSCHAP 1.2.4. *Laat $\|\cdot\|_\alpha$ een matrixnorm zijn geïnduceerd door de vectornorm $\|\cdot\|_\alpha$, dan geldt*

1. $\|Ax\|_\alpha \leq \|A\|_\alpha \|x\|_\alpha$,
2. $\|AB\|_\alpha \leq \|A\|_\alpha \|B\|_\alpha$.

Bewijs. De eerste ongelijkheid volgt onmiddellijk uit de definitie. De tweede kunnen we bekomen door de eerste ongelijkheid twee maal toe te passen:

$$\|ABx\|_\alpha = \|A(Bx)\|_\alpha \leq \|A\|_\alpha \|Bx\|_\alpha \leq (\|A\|_\alpha \|B\|_\alpha) \|x\|_\alpha.$$

Aangezien $\|AB\|_\alpha$ de kleinste constante C is waarvoor $\|ABx\|_\alpha \leq C \|x\|_\alpha$ is het bewijs compleet. $\square$

1.3 Algemene matrixnormen

Naast de matrixnormen geïnduceerd door een vectornorm, kunnen matrixnormen ook in een meer algemene vorm worden gedefinieerd als volgt:

DEFINITIE 1.3.1. *Een afbeelding $\|\cdot\|$ van $\mathbb{C}^{m,n}$ naar $\mathbb{R}$ is een **matrixnorm** indien volgende drie eigenschappen gelden, waarbij A en B $m \times n$ matrices zijn en $z \in \mathbb{C}$:*

1. $\|A\| \geq 0$ and $\|A\| = 0$ indien $A = 0$,
2. $\|zA\| = |z| \|A\|$,
3. $\|A + B\| \leq \|A\| + \|B\|$.

De matrixnormen uit de voorgaande sectie voldoen aan deze definitie. Er zijn echter ook matrixnormen die niet worden geïnduceerd door een vectornorm. De belangrijkste van deze normen is de Frobenius norm die wordt gedefinieerd als

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}.$$

Met andere woorden de Frobenius norm is gelijk aan de 2-norm wanneer we alle mn elementen van A als één lange vector beschouwen. We kunnen de Frobenius norm eveneens schrijven als

$$\|A\|_F = \sqrt{\sum_{j=1}^n \|a_j\|_2^2} = \sqrt{\sum_{i=1}^m \|a_i^*\|_2^2}, \quad (1.1)$$

waarbij a_j wederom de j -de kolom van A voorstelt en a_i^* de i -de rij.

EIGENSCHAP 1.3.1. *De Frobenius norm kan uitgedrukt worden als*

$$\|A\|_F = \sqrt{\text{tr}(A^*A)},$$

waarbij $\text{tr}(\cdot)$ staat voor de trace of som van de diagonaal elementen.

Bewijs. Dit volgt meteen uit:

$$\begin{aligned} \|A\|_F^2 &= \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 = \sum_{i=1}^m \sum_{j=1}^n a_{ij}^* a_{ij} = \sum_{i=1}^m \sum_{j=1}^n (A^*)_{ji} a_{ij} \\ &= \sum_{j=1}^n (A^*A)_{jj} = \text{tr}(A^*A), \end{aligned}$$

aangezien $|z|^2 = z^*z$ voor $z \in \mathbb{C}$. □

Later in de cursus (zie Eigenschap 3.3.3), zullen we aantonen dat de Frobenius norm van een matrix A ook gelijk is aan de vierkantswortel van de som van de kwadraten van de singuliere waarden van A .

EIGENSCHAP 1.3.2. *Laat A een $m \times n$ en B een $n \times p$ matrix zijn dan geldt*

$$\|AB\|_F \leq \|A\|_F \|B\|_F,$$

Bewijs. We gaan hierbij gebruik maken van de ongelijkheid van Cauchy-Schwartz (of kortweg Schwartz) die aangeeft dat

$$|x^*y| \leq \|x\|_2 \|y\|_2,$$

waarbij $x, y \in \mathbb{C}^n$ (zie cursus Lineaire Algebra). Wanneer $C = AB$ and zal $c_{ij} = a_i^* b_j$ (waarbij a_i^* de i -de rij van A voorstelt en b_j de j -de kolom van B). Bijgevolg zien we

$$\|AB\|_F^2 = \sum_{i=1}^m \sum_{j=1}^p |c_{ij}|^2 \leq \sum_{i=1}^m \sum_{j=1}^p \|a_i^*\|_2^2 \|b_j\|_2^2 = \|A\|_F^2 \|B\|_F^2,$$

waarbij de laatste gelijkheid volgt uit (1.1). □

Deze eigenschap geldt echter niet voor alle algemene matrixnormen. De Frobenius norm werd bekomen door de matrix A te schrijven als een mn vector en vervolgens de 2-norm hiervan te nemen. Men kan echter ook de 1- of ∞ -norm van deze vector nemen om een matrixnorm te bekomen. We noteren deze respectievelijk als

$$\|A\|_{F_1} = \sum_{i=1}^m \sum_{j=1}^n |a_{ij}| \qquad \|A\|_{F_\infty} = \max_{i=1}^m \max_{j=1}^n |a_{ij}|.$$

Al deze matrixnormen zijn eveneens equivalent, wat aangeeft dat er voor elke twee willekeurige matrixnormen $\|\cdot\|_\alpha$ en $\|\cdot\|_\beta$ steeds twee constantes $0 \leq k \leq K$ bestaan zodat

$$k \|A\|_\alpha \leq \|A\|_\beta \leq K \|A\|_\alpha,$$

waarbij k en K niet afhankelijk zijn van A .

1.4 Oefeningen

OEFENING 1.4.1. Matrix- en vectornormen

1. Toon aan dat de afbeeldingen $\|\cdot\|_1$ en $\|\cdot\|_2$ vectornormen zijn (voor de 2-norm werd dit reeds bewezen in de cursus Lineaire Algebra).
2. Laat $\|\cdot\|$ een geïnduceerde matrixnorm zijn op $\mathbb{C}^{m,m}$. Toon aan dat $\rho(A) \leq \|A\|$, met $\rho(A)$ de *spectral radius* van A , d.i., $\rho(A) = \max_{i=1}^m |\lambda_i|$ met $\{\lambda_1, \dots, \lambda_m\}$ de set van eigenwaarden van A .
3. Laat $A = uv^*$ met $u \in \mathbb{C}^m$ en $v \in \mathbb{C}^n$, toon dan aan dat $\|A\|_2 = \|u\|_2 \|v\|_2$ (gebruik de ongelijkheid van Cauchy-Schwartz).
4. Toon volgende ongelijkheden aan, waarbij $x \in \mathbb{C}^m$ en $A \in \mathbb{C}^{m,n}$:
 - a. $\|x\|_\infty \leq \|x\|_2$,
 - b. $\|x\|_2 \leq \sqrt{m} \|x\|_\infty$,
 - c. $\|A\|_\infty \leq \sqrt{n} \|A\|_2$,
 - d. $\|A\|_2 \leq \sqrt{m} \|A\|_\infty$,

Geef ook telkens een voorbeeld van een matrix A of vector x waarvoor de gelijkheid opgaat.

5. Bewijs Eigenschap 1.2.2 die stelt dat de ∞ -norm van een matrix A gelijk is aan de maximale rijsum.
6. Toon aan dat alle eigenwaarden van een Hermitische matrix A , d.i., $A^* = A$, reëel zijn. Kijk hiervoor naar de uitdrukking van de vorm x^*Ax .

QR FACTORISATIE

2.1 Orthogonale vectoren en unitaire matrices

In de cursus Lineaire Algebra werd reeds aangegeven dat twee vectoren x en y orthogonaal zijn indien hun scalair product x^*y gelijk is aan 0. Indien x en y reëel zijn, dan staan deze vectoren loodrecht op elkaar. Een verzameling S van vectoren (allen verschillend van nul) noemen we orthogonaal indien voor elke twee vectoren $x, y \in S$ geldt dat ze orthogonaal zijn. Wanneer de 2-norm $\|x\|_2$ van alle vectoren $x \in S$ eveneens gelijk is aan één, dan spreken we van een orthonormale set van vectoren.

EIGENSCHAP 2.1.1. *Een set $S = \{v_1, \dots, v_m\}$ van orthogonale vectoren is lineair onafhankelijk.*

Bewijs. Stel dat $v_k \in S$ geschreven kan worden als lineaire combinatie van de overige elementen in S

$$v_k = \sum_{i=1, i \neq k}^m c_i v_i.$$

Als we beide kanten vermenigvuldigen met v_k^* verkrijgen we dat

$$\|v_k\|_2^2 = \sum_{i=1, i \neq k}^m c_i (v_k^* v_i).$$

Daar S orthogonaal is, zien we dat $(v_k^* v_i)$ gelijk is aan nul voor alle $i \neq k$, waardoor $\|v_k\|_2^2 = 0$. Dit laatste is een tegenspraak op het feit dat alle vectoren in S , per definitie, verschillen van nul. $\square$

Merk op, indien S een orthogonale set is van m vectoren in $\mathbb{C}^m$, dan vormt S een basis voor $\mathbb{C}^m$.

DEFINITIE 2.1.1. Een matrix $Q \in \mathbb{C}^{m,m}$ met kolommen $q_1, \dots, q_m$ noemen we **unitair** indien $S = \{q_1, \dots, q_m\}$ een orthonormale set van vectoren vormt. Indien Q reëel is, dan spreekt men eveneens van een orthogonale matrix.

EIGENSCHAP 2.1.2. Unitaire matrices zijn inverteerbaar en $Q^{-1} = Q^*$, dat is, $Q^*Q = I$.

Bewijs. Indien q_j de j -de kolom van Q weergeeft, dan is q_i^* gelijk aan de i -de rij van Q^* . Indien we $C = Q^*Q$, dan zal $c_{ij} = q_i^*q_j$. Dit laatste is gelijk aan nul indien $i \neq j$ en $q_i^*q_i = \|q_i\|_2^2 = 1$, wat aantoonst dat $C = I$. $\square$

Uit het bovenstaande bewijs zien we dat het omgekeerde duidelijk ook waar is: indien $Q^* = Q^{-1}$, dan vormen de kolommen van Q een orthonormale set. Kortom, een unitaire matrix kunnen we eveneens definiëren als een matrix waarvoor geldt $Q^* = Q^{-1}$. Hiermee kunnen we ook direct volgende eigenschap inzien.

EIGENSCHAP 2.1.3. De matrix Q^* is unitair indien Q unitair is. Het product Q_1Q_2 van twee unitaire matrices Q_1 en Q_2 is eveneens unitair.

Bewijs. Indien Q unitair is, is $Q^*Q = I$, waardoor Q de inverse is van Q^* en $Q = (Q^*)^*$. Indien Q_1 en Q_2 unitair zijn, dan is $(Q_1Q_2)^*Q_1Q_2 = Q_2^*(Q_1^*Q_1)Q_2 = Q_2^*Q_2 = I$. $\square$

Wanneer we x bekomen door een vector b te vermenigvuldigen met een unitaire matrix Q^* , d.i., $x = Q^*b$, dan zien we dat dit equivalent is aan

$$Qx = \sum_{j=1}^m x_j q_j = b,$$

wat aangeeft dat $x = Q^*b$ de coëfficiënten bevat van b ten opzichte van de basis $\{q_1, \dots, q_m\}$. Vermenigvuldigen met Q^* impliceert dus eigenlijk dat we van basis veranderen. Volgende eigenschap geeft aan dat de (Euclidische) structuur van onze ruimte niet wijzigt indien we vermenigvuldigen met een unitaire matrix.

EIGENSCHAP 2.1.4. De vermenigvuldiging met een unitaire matrix Q heeft geen impact op het scalair product tussen twee vectoren of op de 2-norm van een vector x , d.i., $(Qx)^*(Qy) = x^*y$ en $\|Qx\|_2 = \|x\|_2$. Verder is $\|Q\|_2 = 1$.

Bewijs. Daar $(Qx)^* = x^*Q^*$, zien we dat $(Qx)^*(Qy) = x^*(Q^*Q)y = x^*y$ en dat

$$\|Qx\|_2 = \sqrt{(Qx)^*(Qx)} = \sqrt{x^*(Q^*Q)x} = \sqrt{x^*x} = \|x\|_2.$$

Hieruit volgt onmiddellijk

$$\|Q\|_2 = \sup_{x \in \mathbb{C}^n, \|x\|_2=1} \|Qx\|_2 = \sup_{x \in \mathbb{C}^n, \|x\|_2=1} \|x\|_2 = 1.$$

□

EIGENSCHAP 2.1.5. *Laat $Q_1 \in \mathbb{C}^{m,m}$ en $Q_2 \in \mathbb{C}^{n,n}$ unitaire matrices zijn en $A \in \mathbb{C}^{m,n}$, dan is $\|Q_1 A\|_2 = \|A\|_2$, $\|Q_1 A\|_F = \|A\|_F$, $\|AQ_2\|_2 = \|A\|_2$ en $\|AQ_2\|_F = \|A\|_F$.*

Bewijs. Uit de voorgaande eigenschap weten we dat $\|Q_1\|_2 = 1$. Gebruik makend van Eigenschap 1.2.4 zien we dat $\|Q_1 A\|_2 \leq \|Q_1\|_2 \|A\|_2 = \|A\|_2$. Omwille van dezelfde eigenschap zien we eveneens $\|A\|_2 = \|Q_1^* Q_1 A\|_2 \leq \|Q_1^*\|_2 \|Q_1 A\|_2 = \|Q_1 A\|_2$. Het bewijs voor de Frobenius norm maakt gebruik van Eigenschap 1.3.1:

$$\|Q_1 A\|_F^2 = \text{tr}((Q_1 A)^* Q_1 A) = \text{tr}(A^* Q_1^* Q_1 A) = \text{tr}(A^* A) = \|A\|_F^2.$$

□

We eindigen deze sectie met volgende eigenschap die aangeeft dat de absolute waarde van de determinant van een unitaire matrix gelijk is aan één.

EIGENSCHAP 2.1.6. *Laat $Q \in \mathbb{C}^{m,m}$ unitair zijn, dan is $|\det Q| = 1$.*

Bewijs. Aangezien $\det(AB) = \det(A)\det(B)$ (zie cursus Lineaire Algebra), weten we dat $1 = \det I = \det(Q^*Q) = \det(Q^*)\det(Q)$. Verder geldt dat $\det(A^*) = \det(A)^*$ voor $A \in \mathbb{C}^{m,m}$ (dit is een veralgemening van $\det(A^t) = \det(A)$ voor $A \in \mathbb{R}^{m,m}$), wat impliceert dat $\det(Q)^*\det(Q) = |\det(Q)|^2 = 1$. □

Bijgevolg is de determinant $\det Q$ van een reële unitaire matrix Q steeds gelijk is aan 1 of -1 . Wanneer $\det Q = 1$ hebben we te maken met een rotatiebeweging (zoals in de cursus Computer Graphics in $\mathbb{R}^3$), anders stemt Q overeen met een reflectie matrix (zie Sectie 2.4.2).

2.2 Definitie

De QR factorisatie vormt één van de belangrijkste algoritmen uit de numerieke lineaire algebra. Ze bestaat in twee vormen: de gereduceerde QR factorisatie en de volledige QR factorisatie. We starten met de eerste.

2.2.1 Gereduceerde QR factorisatie

Laat A een $m \times n$ matrix zijn met $m \geq n$. Dan kunnen we gaan kijken naar de opeenvolgende ruimtes voorgebracht door de kolommen $a_1, \dots, a_n$ van A :

$$\langle a_1 \rangle \subseteq \langle a_1, a_2 \rangle \subseteq \langle a_1, a_2, a_3 \rangle \subseteq \dots \subseteq \langle a_1, \dots, a_n \rangle.$$

Het idee van de (gereduceerde) QR factorisatie bestaat erin een set van orthonormale vectoren $q_1, \dots, q_n \in \mathbb{C}^m$ te vinden die deze n ruimtes opspant. In het bijzonder willen we, voor elke $j = 1, \dots, n$, dat $a_1, \dots, a_j$ geschreven kan worden als lineaire combinatie van $q_1, \dots, q_j$, of nog,

$$a_1, \dots, a_j \in \langle q_1, \dots, q_j \rangle,$$

voor $j = 1, \dots, n$. We zoeken dus een set van vectoren $q_1, \dots, q_n$ en coëfficiënten r_{ij} , met $i \leq j$ zodat

$$\begin{aligned} a_1 &= r_{11}q_1 \\ a_2 &= r_{12}q_1 + r_{22}q_2 \\ a_3 &= r_{13}q_1 + r_{23}q_2 + r_{33}q_3 \\ &\vdots \\ a_n &= r_{1n}q_1 + r_{2n}q_2 + \dots + r_{nn}q_n \end{aligned} \tag{2.2}$$

We kunnen dit herschrijven in matrixvorm als

$$\left[\begin{array}{c|c|c|c} a_1 & a_2 & \dots & a_n \end{array} \right] = \left[\begin{array}{c|c|c|c} q_1 & q_2 & \dots & q_n \end{array} \right] \left[\begin{array}{cccc} r_{11} & r_{12} & \dots & r_{1n} \\ & r_{22} & & \vdots \\ & & \ddots & \\ & & & r_{nn} \end{array} \right].$$

of kortweg als

$$A = \hat{Q}\hat{R},$$

met $\hat{R}$ een $n \times n$ bovendriehoeksmatrix en $\hat{Q}$ een $m \times n$ matrix met orthonormale kolommen.

Indien de rang van A gelijk is aan n , resulteert dit in

$$\langle a_1, \dots, a_j \rangle = \langle q_1, \dots, q_j \rangle,$$

voor $j = 1, \dots, n$. In dit geval kan q_j eveneens geschreven worden als lineaire combinatie van $\{a_1, \dots, a_j\}$, voor $j = 1, \dots, n$.

2.2.2 Volledige QR factorisatie

Bij een volledige QR factorisatie gaan we de gereduceerde factorisatie van $A = \hat{Q}\hat{R}$ verder uitbreiden, door aan de set $\{q_1, \dots, q_n\}$ nog $m - n$ orthonormale vectoren $\{q_{n+1}, \dots, q_m\}$ toe te voegen zodat $\{q_1, \dots, q_m\}$ een basis vormt voor $\mathbb{C}^m$. Wanneer we de matrices Q en R dan definiëren als

$$Q = \left[\begin{array}{c|c|c|c|c} & & & & \\ & & & & \\ \hat{Q} & & & & \\ & & & & \\ & q_{n+1} & & & \\ & \vdots & & & \\ & q_m & & & \end{array} \right] \quad \text{en} \quad R = \left[\begin{array}{cccc} r_{11} & r_{12} & \dots & r_{1n} \\ & r_{22} & & \vdots \\ & & \ddots & \\ & & & r_{nn} \end{array} \right],$$

dan zien we dat Q een unitaire $m \times m$ matrix is, R een $m \times n$ bovendriehoeksmatrix en

$$A = QR.$$

Merk op, dit impliceert dat $Q^*A = R$, wat aangeeft dat Q^*a_j gelijk is aan de j -de kolom van R . De volledige QR factorisatie komt dus neer op het vinden van een nieuwe basis $\{q_1, \dots, q_m\}$ van $\mathbb{C}^m$ zodat enkel de eerste j coëfficiënten van de j -de kolom van A kunnen verschillen van nul in de basis $\{q_1, \dots, q_m\}$.

2.3 Existentie

In deze sectie tonen we aan dat er een QR factorisatie bestaat voor elke $m \times n$ matrix A met $m \geq n$. We doen dit door een procedure op te geven die de vectoren $q_1, \dots, q_m$ berekent, alsook de coëfficiënten r_{ij} . Deze procedure kwam reeds aan bod in de cursus Lineaire Algebra, het gaat namelijk om de Gram-Schmidt orthogonalisatie (1883). We zullen deze voor de volledigheid nog even terug opfrissen.

We starten met het bepalen van de gereduceerde QR factorisatie en nemen voorlopig aan dat de rang van A gelijk is aan n . De Gram-Schmidt methode bestaat in dit geval uit n stappen. Tijdens de j -de stap zoeken we een vector v_j die behoort tot $\langle a_1, \dots, a_j \rangle$ en die orthogonaal is aan $q_1, \dots, q_{j-1}$. Deze vector normaliseren we vervolgens om q_j te bekomen, d.i., $q_j = v_j / \|v_j\|_2$. De vector v_j kunnen we als volgt definiëren:

$$v_j = a_j - (q_1^* a_j)q_1 - (q_2^* a_j)q_2 - \dots - (q_{j-1}^* a_j)q_{j-1}. \quad (2.3)$$

We kunnen als volgt inzien dat deze vector orthogonaal is aan $q_1, \dots, q_{j-1}$. De bovenstaande gelijkheid vermenigvuldigen met q_k^* , voor $k = 1, \dots, j-1$ impliceert dat

$$q_k^* v_j = q_k^* a_j - (q_1^* a_j)(q_k^* q_1) - (q_2^* a_j)(q_k^* q_2) - \dots - (q_{j-1}^* a_j)(q_k^* q_{j-1}).$$

Aangezien $\{q_1, \dots, q_{j-1}\}$ een orthonormale set is, geeft dit aan dat

$$q_k^* v_j = q_k^* a_j - (q_k^* a_j) \|q_k\|_2^2 = 0,$$

wat de vereiste orthogonaliteit bewijst. Verder behoort v_j tot $\langle a_1, \dots, a_j \rangle$, want het is een lineaire combinatie van $q_1, \dots, q_{j-1}, a_j$ en $q_k \in \langle a_1, \dots, a_k \rangle$ voor $k = 1, \dots, j-1$.

De set van vergelijkingen voor de gereduceerde QR factorisatie in (2.2) kunnen we herschikken als

$$\begin{aligned} q_1 &= \frac{a_1}{r_{11}} \\ q_2 &= \frac{a_2 - r_{12}q_1}{r_{22}} \\ q_3 &= \frac{a_3 - r_{13}q_1 - r_{23}q_2}{r_{33}} \\ &\vdots \\ q_n &= \frac{a_n - \sum_{i=1}^{n-1} r_{in}q_i}{r_{nn}} \end{aligned}$$

Klassieke Gram-Schmidt Algoritme

```
for  $j = 1$  to  $n$ 
     $v_j = a_j$ 
    for  $i = 1$  to  $j - 1$ 
         $r_{ij} = q_i^* a_j$ 
         $v_j = v_j - r_{ij} q_i$ 
     $r_{jj} = \|v_j\|_2$ 
     $q_j = v_j / r_{jj}$ 
```

Figure 2.2: Klassieke Gram-Schmidt algoritme voor rang n matrix A (onstabiel).

Wanneer we deze combineren met (2.3), zien we dat we de coëfficiënten r_{ij} gelijk kunnen stellen aan

$$r_{ij} = q_i^* a_j,$$

voor $i < j$. Verder moet $\|q_j\|_2 = 1$ wat aangeeft dat

$$|r_{jj}| = \left\| a_j - \sum_{i=1}^{j-1} r_{ij} q_i \right\|_2.$$

Merk op dat het teken van r_{jj} nog onbepaald is. We kunnen $r_{jj} > 0$ kiezen, waardoor de diagonaal elementen van $\hat{R}$ positief zullen zijn. Dit resulteert in het Algoritme in Figuur 2.2.

Het klassieke Gram-Schmidt algoritme volstaat om te bewijzen dat de QR factorisatie steeds bestaat, zoals hieronder aangegeven, maar is numeriek niet stabiel (zie Sectie 4.2 voor een definitie van stabiliteit). Vandaar dat we in de volgende sectie eveneens het aangepaste (modified) Gram-Schmidt algoritme zullen bekijken.

EIGENSCHAP 2.3.1. *Elke matrix $A \in \mathbb{C}^{m,n}$ met $m \geq n$ heeft een volledige QR factorisatie en bijgevolg ook een gereduceerde.*

Bewijs. Indien de rang van A gelijk is aan n dan volgt het bestaan van de gereduceerde QR factorisatie uit het klassieke Gram-Schmidt algoritme. De enige manier waarop dit algoritme kan falen, treedt op indien op een bepaald

ogenblik $v_j = 0$, waardoor $r_{jj} = 0$ en we moeten delen door nul. Echter indien $v_j = 0$ dan kan a_j geschreven worden als een lineaire combinatie van $q_1, \dots, q_{j-1}$ (omwille van (2.3)) en deze behoren tot $\langle a_1, \dots, a_{j-1} \rangle$, wat in tegenspraak is met het feit dat de rang van A gelijk is aan n .

Wanneer A een rang heeft kleiner dan n , dan zullen er tijdens de uitvoering van het klassieke Gram-Schmidt algoritme één of meerdere v_j vectoren zijn die gelijk zijn aan nul. In dat geval mag q_j gekozen worden als een willekeurige vector in $\mathbb{C}^m$ die orthogonaal is aan $q_1, \dots, q_{j-1}$ en zetten we $r_{jj} = 0$. Dit geeft aan dat elke matrix A een gereduceerde QR factorisatie heeft.

Een volledige factorisatie kan vervolgens bekomen worden door $q_1, \dots, q_n$ aan te vullen met een set van $m - n$ vectoren $q_{n+1}, \dots, q_m$ zodat $S = \{q_1, \dots, q_m\}$ een basis vormt voor $\mathbb{C}^m$ (dit kan steeds omwille van de Uitbreidingsstelling, zie cursus Lineaire Algebra). $\square$

Volledige QR factorisaties zijn in het algemeen niet uniek (bv. de keuze van q_{n+1} tot q_m is vrij). Wanneer we ons echter beperken tot gereduceerde QR factorisaties van matrices $A \in \mathbb{C}^{m,n}$ met rang n , dan kan men tonen dat er een unieke QR factorisatie bestaat waarbij alle diagonaal elementen van R strict positieve reële getallen zijn. Wanneer we de i -de kolom van $\hat{Q}$ zouden vermenigvuldigen met een $z \in \mathbb{C}$ waarvoor $|z| = 1$ en de i -de rij van $\hat{R}$ met z^{-1} dan verkrijgen we eveneens een QR factorisatie van A , maar dan is r_{ii} geen strict positief reëel getal meer.

2.4 Algoritmes

In deze sectie bekijken we drie verschillende algoritmes om een QR factorisatie van een matrix A te berekenen. We starten met de aangepaste (modified) Gram-Schmidt methode, daarna bekijken we Householder reflecties en Givens rotaties.

2.4.1 Modified Gram-Schmidt

Het klassieke Gram-Schmidt algoritme wordt in de praktijk niet gebruikt voor het berekenen van een QR factorisatie omdat dit numeriek onstabiel blijkt te zijn. We kunnen het algoritme echter eenvoudig aanpassen om de stabiliteit te verbeteren.

Klassieke Gram-Schmidt $\sim 2mn^2$ flops for $j = 1$ to n $\quad v_j = a_j$ for $j = 1$ to n $\quad$ for $i = 1$ to $j - 1$ $\quad\quad r_{ij} = q_i^* a_j$ $\quad\quad v_j = v_j - r_{ij} q_i$ $\quad r_{jj} = \ v_j\ _2$ $\quad q_j = v_j / r_{jj}$	Aangepaste Gram-Schmidt $\sim 2mn^2$ flops for $i = 1$ to n $\quad v_i = a_i$ for $i = 1$ to n $\quad r_{ii} = \ v_i\ _2$ $\quad q_i = v_i / r_{ii}$ $\quad$ for $j = i + 1$ to n $\quad\quad r_{ij} = q_i^* v_j$ $\quad\quad v_j = v_j - r_{ij} q_i$
--	---

Figure 2.3: Klassieke en aangepaste Gram-Schmidt algoritme voor rang n matrix A met kolommen $a_1, \dots, a_n$.

Figuur 2.3 toont het klassieke en aangepaste algoritme, waarbij we de enkele lus van het klassieke algoritme hebben opgesplitst, wat niets verandert aan het resultaat of de afrondingsfouten. Beide algoritmes starten met $v_j = a_j$, voor $j = 1, \dots, n$ en berekenen q_1 als $a_1 / \|a_1\|_2$. Het aangepaste algoritme zal vervolgens de vectoren v_j , met $j > 1$, al aanpassen door er $r_{1j}q_1$ van af te trekken, wat in het klassieke algoritme pas later gebeurt. Op het eerste zicht lijkt het aangepaste algoritme dus enkel de volgorde van de operaties te wijzigen.

Echter, wanneer we kijken naar de lijn voor het berekenen van r_{ij} dan zien we dat $r_{ij} = q_i^* a_j$ in het klassieke algoritme en $r_{ij} = q_i^* v_j$ in het aangepaste algoritme. Op het ogenblik dat r_{ij} berekend wordt door het aangepaste algoritme zien we dat

$$v_j = a_j - r_{1j}q_1 - r_{2j}q_2 - \dots - r_{(i-1)j}q_{i-1},$$

aangezien $r_{ij}q_i$ van v_j wordt afgetrokken tijdens iteratie i (voor $i < j$) en we gestart zijn met $v_j = a_j$. Bijgevolg is $r_{ij} = q_i^* v_j$ gelijk aan

$$r_{ij} = q_i^* v_j = q_i^* a_j - r_{1j}(q_i^* q_1) - r_{2j}(q_i^* q_2) - \dots - r_{(i-1)j}(q_i^* q_{i-1}).$$

Aangezien q_i orthogonaal is met $q_1, \dots, q_{i-1}$, zien we dat $r_{ij} = q_i^* a_j$, wat aangeeft dat beide algoritmes in een exacte rekenomgeving identiek zijn. De numerieke afrondingen die optreden in een floating point omgeving zijn echter heel verschillend.

Wanneer we het aangepaste Gram-Schmidt algoritme implementeren, kunnen we wat geheugen besparen door de a_j vectoren te overschrijven met v_j en deze vervolgens te overschrijven met q_j . Wat de tijdscomplexiteit betreft, zien we dat deze gedomineerd wordt door de laatste twee lijnen code (voor het aangepaste algoritme). Deze twee lijnen worden ongeveer $n^2/2$ keer uitgevoerd (daar $\sum_{i=1}^n i = n(n-1)/2$). De lijn $q_i^* v_j$ vergt m vermenigvuldigingen en $m-1$ optellingen. De lijn $v_j - r_{ij} q_i$ telt m vermenigvuldigingen en m optellingen. Samen zijn de 2 laatste lijnen dus ongeveer goed voor $4m$ flops (in het reële geval), wat het totaal op $2mn^2$ brengt. Voor een vierkante $m \times m$ matrix A resulteert dit in $2m^3$ flops (in het reële geval).

2.4.2 Householder reflecties

Wanneer $v \in \mathbb{C}^m$ een willekeurige vector is verschillend van nul, dan definiëren we de Householder reflectie matrix (of transformatie matrix) P als

$$P = I - \frac{2}{v^* v} v v^*.$$

De matrix P is Hermitisch daar

$$P^* = I^* - \frac{2}{(v^* v)^*} (v v^*)^* = I - \frac{2}{v^* v} v v^* = P.$$

Verder is P ook unitair aangezien

$$P^* P = P^2 = I - 2 \left(\frac{2}{v^* v} v v^* \right) + \frac{4}{(v^* v)^2} v (v^* v) v^* = I.$$

De volgende eigenschap geeft aan vermenigvuldigen met P in het reële geval overeenstemt met een reflectie op het (hyper)vlak dat orthogonaal is aan v en dat door de oorsprong gaat. Kortom, P zal elk punt aan de ene kant van het vlak afbeelden op zijn spiegelbeeld aan de andere kant.

EIGENSCHAP 2.4.1. *Laat $P = I - \frac{2}{v^* v} v v^*$ en $x \in \mathbb{C}^m$, dan is*

1. $v^*(Px) = -v^* x$,
2. $x - Px = \frac{2v^* x}{v^* v} v$,
3. $\|Px\| = \|x\|$.

Bewijs. De matrix P is unitair waardoor de laatste gelijkheid volgt uit Eigenschap 2.1.4. Een eenvoudige berekening toont verder dat

$$v^*(Px) = v^*(I - \frac{2}{v^*v}vv^*)x = v^*x - \frac{2}{v^*v}(v^*v)v^*x = v^*x - 2v^*x = -v^*x$$

en dat

$$x - Px = x - (x - \frac{2}{v^*v}vv^*x) = \frac{2v^*x}{v^*v}v.$$

□

Merk op, de eerste gelijkheid geeft aan dat Px aan de andere kant van het hypervlak gelegen is. De tweede gelijkheid stelt dat de vector die Px met x verbindt, loodrecht staat op het hypervlak (want het is een veelvoud van v) en de laatste geeft aan dat Px dezelfde lengte heeft als x . Kortom, Px is het spiegelbeeld van x .

Voor $x, y \in \mathbb{C}^m$ kunnen we ons de vraag stellen of er een v bestaat die ervoor zorgt dat $Px = y$. In het kader van de QR factorisatie zijn we voornamelijk geïnteresseerd in het geval waarbij de vector y enkel in de eerste positie verschilt van nul. Daar P unitair is, is $\|Px\|_2 = \|x\|_2$, wat aangeeft dat y volgende vorm moet hebben $y = z\|x\|_2 e_1$, met $z \in \mathbb{C}$ en $|z| = 1$. Indien we ons beperken tot reële y vectoren hebben we dus twee keuzes voor y , namelijk $y = \pm \|x\|_2 e_1$. Volgende eigenschap toont aan dat hiermee twee mogelijkheden voor v overeenstemmen: $v = x \pm \|x\|_2 e_1$. Dit resultaat wordt in Figuur 2.4 ook nog eens grafisch weergegeven wanneer $x \in \mathbb{R}^2$.

EIGENSCHAP 2.4.2. *Laat $x \in \mathbb{R}^m$ en $P = I - \frac{2}{v^*v}vv^*$ met $v = x - \kappa\|x\|_2 e_1$, dan is $Px = \kappa\|x\|_2 e_1$ met $\kappa = \pm 1$.*

Bewijs. We starten met v^*v te berekenen

$$\begin{aligned} v^*v &= (x - \kappa\|x\|_2 e_1)^*(x - \kappa\|x\|_2 e_1) \\ &= x^*x - x^*\kappa\|x\|_2 e_1 - \kappa^*\|x\|_2 e_1^*x + \kappa^*\kappa\|x\|_2^2 e_1^*e_1 \\ &= 2(x^*x - \kappa\|x\|_2 x_1), \end{aligned}$$

daar $\kappa^*\kappa = 1$, $e_1^*e_1 = 1$, $x^*x = \|x\|_2^2$ en $e_1^*x = x_1 = x^*e_1$ want x is reëel. Aangezien

$$v^*x = x^*x - \kappa^*\|x\|_2 e_1^*x = x^*x - \kappa\|x\|_2 x_1,$$

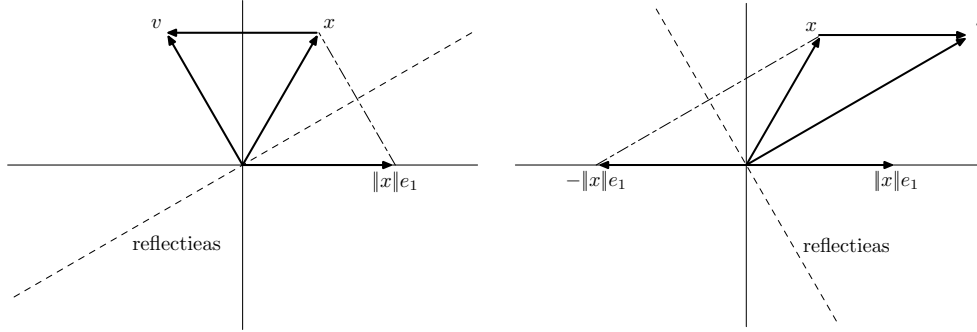


Figure 2.4: Grafische weergave van Householder reflectie in $\mathbb{R}^2$: links is $v = x - \|x\| e_1$, rechts is $v = x + \|x\| e_1$.

volgt hieruit dat $v^*x = v^*v/2$. Bijgevolg kunnen we besluiten dat

$$Px = x - \frac{2}{v^*v}v(v^*x) = x - v = \kappa \|x\|_2 e_1.$$

□

Indien $x \in \mathbb{C}^m$ (i.p.v. $\mathbb{R}^m$) zit er mogelijk een probleem in het bovenstaande bewijs: indien x_1 en x_1^* niet gelijk zijn, zullen x^*e_1 en e_1^*x verschillen. Merk op, van zodra $x_1 \in \mathbb{R}$ gaat het bovenstaande bewijs wel op. In het algemeen kan men dit resultaat wel veralgemenen door $\kappa = \pm e^{i \arg x_1}$ te nemen (met $\arg x_1 = \theta$ wanneer $x_1 = |x_1| e^{i\theta} = |x_1| (\cos(\theta) + i \sin(\theta))$). Het introduceren van de complexe factor $\pm e^{i\theta}$ zorgt ervoor dat $x_1^* \kappa = x_1 \kappa^* = |x_1|$ (daar $(e^{i\theta})^* = e^{-i\theta}$), waardoor het bovenstaande bewijs weer opgaat.

Vanuit mathematisch oogpunt zijn beide keuzes voor v goed. Echter, wanneer we werken met floating point getallen, dan willen we vermijden dat er *bit cancellation* optreedt wanneer we $v = x - \kappa \|x\|_2 e_1$ berekenen. Aangezien e_1 enkel in de eerste component verschilt van nul, willen we dat x_1 en $-\kappa \|x\|_2 e_1$ hetzelfde teken hebben. Voor het reële geval kiezen we dus best $\kappa = -\text{sign}(x_1)$, waarbij $\text{sign}(x_1)$ het teken weergeeft van x_1 :

$$\text{sign}(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases},$$

merk op, we spreken af dat $\text{sign}(0) = 1$. Dit resulteert in

$$v = x + \text{sign}(x_1) \|x\|_2 e_1.$$

In het complexe geval nemen we $\kappa = -e^{i \arg x_1}$, of nog

$$v = x + e^{i \arg x_1} \|x\|_2 e_1.$$

We gaan nu kijken hoe we met behulp van deze matrices P een volledige QR factorisatie kunnen berekenen van een $m \times n$ matrix A met kolommen $a_1, \dots, a_n$. Tijdens stap 1 stellen we $x = a_1$, waardoor we

$$v_1 = a_1 + e^{i \arg a_{11}} \|a_1\|_2 e_1$$

en $P_1 = I - \frac{2}{v_1^* v_1} v_1 v_1^*$. Wanneer we nu $P_1 A$ uitrekenen, zien we dat de voorgaande eigenschap aangeeft dat de eerste kolom van A een veelvoud is van e_1 , b.v., voor een 5×4 matrix A zien we dat

$$P_1 A = \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix},$$

waarbij een $\times$ een willekeurig complex getal voorstelt. We willen nu een P_2 vinden zodat

$$P_2 P_1 A = P_2 \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix} = \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}.$$

Indien we de tweede kolom van $P_1 A$, die we noteren als $a_2^{(1)}$, schrijven als

$$a_2^{(1)} = \begin{bmatrix} a_{12}^{(1)} \\ a_{.2}^{(1)} \end{bmatrix},$$

met $a_{12}^{(1)} \in \mathbb{C}$, dan zien we dat $\hat{P}_2 = I - \frac{2}{\hat{v}_2^* \hat{v}_2} \hat{v}_2 \hat{v}_2^*$ met $\hat{v}_2$ gelijk aan

$$\hat{v}_2 = a_{.2}^{(1)} + e^{i \arg a_{22}^{(1)}} \|a_{.2}^{(1)}\|_2 e_1,$$

ervoor zorgt dat $\hat{P}_2 a_{.2}^{(1)}$ een veelvoud is van $e_1 \in \mathbb{C}^{m-1}$. Indien we nu

$$P_2 = \begin{bmatrix} 1 & 0 \\ 0 & \hat{P}_2 \end{bmatrix},$$

Householder QR factorisatie (impliciete Q) $\sim 2mn^2 - 2n^3/3$ flops**for** $k = 1$ **to** n

$$x = A_{k:m,k}$$

$$\hat{v}_k = x + e^{i \arg x_1} \|x\|_2 e_1$$

$$\hat{v}_k = \hat{v}_k / \|\hat{v}_k\|_2$$

$$A_{k:m,k:n} = A_{k:m,k:n} - 2\hat{v}_k(\hat{v}_k^* A_{k:m,k:n})$$

Figure 2.5: Householder algoritme voor volledige QR factorisatie van de matrix A .

dan verkrijgen we de gezochte matrix P_2 , want vermenigvuldigen met P_2 bewaart de nullen op de eerste kolom van $P_1 A$ (alook de elementen op de eerste rij). Merk op dat $\hat{P}_2$ een Householder reflectie in $\mathbb{C}^{m-1}$ voorstelt. We kunnen eenvoudig inzien dat P_2 de Householder reflectie in $\mathbb{C}^m$ voorstelt met

$$v_2 = \begin{bmatrix} 0 \\ \hat{v}_2 \end{bmatrix}.$$

Wanneer we deze procedure n maal uitvoeren, bekomen we $P_1, \dots, P_n$ zodat

$$P_n P_{n-1} \dots P_1 A = R$$

met R een bovendriehoeksmatrix. Dit geeft aan dat $Q^* A = R$ met

$$Q^* = P_n P_{n-1} \dots P_1,$$

wat equivalent is aan $A = QR$ met

$$Q = P_1 P_2 \dots P_n,$$

aangezien $P_j^* = P_j$, voor $j = 1, \dots, n$. Merk ook op dat $Q^* Q = I$, wat aangeeft dat Q unitair is.

Dit alles resulteert in het algoritme weergegeven in Figuur 2.5, dat R berekent en de n vectoren $\hat{v}_1, \dots, \hat{v}_n$ (met $\hat{v}_1 = v_1$). Het algoritme in Figuur 2.6 geeft aan dat we ook de producten $Q^* b$ en Qx snel kunnen berekenen door gebruik te maken van de impliciete representatie $\hat{v}_1, \dots, \hat{v}_n$ van de unitaire matrix Q . Indien we de matrix Q toch expliciet willen voorstellen, dan

Impliciete berekening van Q^*b for $k = 1$ to n $b_{k:m} = b_{k:m} - 2\hat{v}_k(\hat{v}_k^* b_{k:m})$	Impliciete berekening van Qx for $k = n$ down to 1 $x_{k:m} = x_{k:m} - 2\hat{v}_k(\hat{v}_k^* x_{k:m})$
---	--

Figure 2.6: Algoritme voor de impliciete berekening van Q^*b en Qx .

kunnen we bijvoorbeeld m maal het algoritme voor Qx gebruiken met $x = e_1, \dots, e_m$. De m bekomen vectoren vormen dan de m kolommen van $QI = Q$. Analooog kan men ook m maal het algoritme voor Q^*b gebruiken met $b = e_1, \dots, e_m$. De m bekomen vectoren vormen dan de m kolommen van $Q^*I = Q^*$, waardoor we Q bekomen door hiervan de complex geconjugeerde te nemen.

Wanneer we kijken naar de tijdscomplexiteit van het algoritme in Figuur 2.5, dan mag duidelijk zijn dat de laatste lijn bepalend is. Tijdens de k -de iteratie, vergt deze (in het reële geval):

- $(m - k + 1)(n - k + 1)$ flops voor de aftrekking,
- $(m - k + 1)(n - k + 1)$ vermenigvuldigingen en $(m - k)(n - k + 1)$ optellingen voor het scalaire product met $\hat{v}_k^*$ en
- $(m - k + 1)(n - k + 1)$ vermenigvuldigingen voor het product met $\hat{v}_k$.

De vermenigvuldiging met 2 kan veel sneller gebeuren en tellen we dan ook niet als flop. In totaal bedraagt het aantal flops dus plus minus

$$4 \sum_{k=1}^n (m - k)(n - k) = 4 \left(\sum_{k=1}^n m(n - k) - \sum_{k=1}^n kn + \sum_{k=1}^n k^2 \right).$$

Deze drie sommen kunnen we benaderen door

$$\sim 4m \frac{n^2}{2} - 4n \frac{n^2}{2} + 4 \frac{n^3}{3} = 2mn^2 - \frac{2}{3}n^3,$$

waarbij we gebruik hebben gemaakt van de gelijkheid $\sum_{k=1}^n k^2 = n(n+1)(2n+1)/6 \sim n^3/3$. Voor een vierkante $m \times m$ matrix A resulteert dit in $\sim \frac{4}{3}m^3$ flops.

2.4.3 Givens rotaties

Een reële Givens rotatie wordt weergegeven door een reële 2×2 matrix van de vorm

$$G(c, s) = \begin{bmatrix} c & s \\ -s & c \end{bmatrix},$$

waarbij $c = \cos \theta$ en $s = \sin \theta$ voor een $\theta \in [0, 2\pi]$. Merk op, dit is een 2×2 rotatiematrix zoals we die reeds gebruikt hebben in de cursus Computer Graphics. Verder is $\det G(c, s) = c^2 + s^2 = 1$ en

$$G(c, s)G(c, s)^* = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} c^2 + s^2 & 0 \\ 0 & c^2 + s^2 \end{bmatrix} = I,$$

wat aangeeft dat $G(c, s)$ een unitaire matrix is.

Wanneer we nu een willekeurige vector $x = (x_1, x_2)^* \in \mathbb{R}^2$ nemen, dan kunnen we het tweede element van $G(c, s)x$ nul maken door $c = x_1/\|x\|_2$ en $s = x_2/\|x\|_2$ aangezien

$$G(c, s)x = \frac{1}{\|x\|_2} \begin{bmatrix} x_1 & x_2 \\ -x_2 & x_1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \|x\|_2 \\ 0 \end{bmatrix}.$$

We kunnen dit idee ook gebruiken om een willekeurig element van een reële vector $x \in \mathbb{R}^m$ gelijk te maken aan nul door $G(c, s, i, j)$ te definiëren als

$$G(c, s, i, j) = \begin{bmatrix} 1 & \dots & 0 & \dots & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \dots & c & \dots & s & \dots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \dots & -s & \dots & c & \dots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{bmatrix},$$

waarbij we $G(c, s)$ bekomen indien we alle rijen en kolommen verwijderen op de i - en j -de na. Het verwijderen van de i - en j -de rij en kolom resulteert dan weer in de eenheidsmatrix. Wanneer we een vector $x \in \mathbb{R}^m$ vermenigvuldigen met $G(c, s, i, j)$ waarbij

$$c = x_i/\sqrt{x_i^2 + x_j^2} \quad \text{en} \quad s = x_j/\sqrt{x_i^2 + x_j^2},$$

dan zal het i -de element van $G(c, s, i, j)x$ gelijk zijn aan $r = \sqrt{x_i^2 + x_j^2}$ en het j -de element gelijk aan nul. Alle overige elementen van x blijven ongewijzigd.

Het mag duidelijk zijn dat we hiermee ook een procedure kunnen maken om x af te beelden op een veelvoud van de vector e_1 zoals bij de Householder reflectie. Bijvoorbeeld, door x eerst met $G_{m-1} = G(\cdot, \cdot, m-1, m)$ te vermenigvuldigen, wordt het laatste element nul. Vervolgens met $G_{m-2} = G(\cdot, \cdot, m-2, m-1)$ vermenigvuldigen maakt het voorlaatste element nul, enzovoort. Let wel de vereiste c en s waarde moet steeds berekend worden. Kortom, we kunnen x afbeelden op een veelvoud van e_1 door te vermenigvuldigen met $G_1 G_2 \dots G_{m-1}$.

Net zoals bij de Householder transformaties kunnen we hier ook kiezen of we x afbeelden op re_1 of $-re_1$, door het teken van c en s te wijzigen. Het beste resultaat wordt bekomen wanneer we

$$c = \text{sign}(x_i)x_i/\sqrt{x_i^2 + x_j^2} \quad \text{en} \quad s = \text{sign}(x_j)x_j/\sqrt{x_i^2 + x_j^2},$$

indien $x_i \neq 0$. Anders geeft $c = 0$ en $s = \text{sign}(x_j)$ het beste resultaat. Voor vectoren $x \in \mathbb{C}^m$ kan men analoog te werk gaan, maar de formules voor c en s zijn nog wat ingewikkelder.

Dit idee kunnen we nu ook hanteren om een QR factorisatie van een matrix $A \in \mathbb{C}^{m,n}$ te bepalen, we beperken ons hier weer tot het reële geval. In stap 1 vermenigvuldigen we A met $G^{(1)} = G_1 \dots G_{m-1}$ zodat de eerste kolom een veelvoud wordt van e_1 :

$$G^{(1)}A = G_1 G_2 \dots G_{m-1} A = \begin{bmatrix} \times & \times & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \end{bmatrix},$$

waarbij de elementen aangeduid met $\times$ en $\star$ willekeurige reële getallen zijn. Tijdens iteratie twee bepalen we $G^{(2)}$ op basis van de laatste $m-1$ elementen van de tweede kolom van $G^{(1)}A$ (deze zijn aangeduid met een $\star$ in het bovenstaande voorbeeld), zodat de laatste $m-2$ elementen van de tweede kolom van $G^{(2)}G^{(1)}A$ gelijk zijn aan nul.

$$G^{(2)}G^{(1)}A = G^{(2)} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \\ 0 & \star & \times & \times \end{bmatrix} = \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}.$$

Merk op, $G^{(2)}$ is het product van $m-2$ rotatiematrices $G(\cdot, \cdot, 2, 3)$ tot $G(\cdot, \cdot, m-1, m)$. Verder wijzigt de vermenigvuldiging met $G^{(2)}$ de eerste rij en kolom van $G^{(1)}A$ niet.

Wanneer we deze procedure geen 2 stappen, maar n stappen uitvoeren, zien we dat

$$R = G^{(n)}G^{(n-1)} \dots G^{(1)}A,$$

een bovendriehoeksmatrix is en QR met $Q = (G^{(1)})^*(G^{(2)})^* \dots (G^{(n)})^*$ een volledige QR factorisatie van A . De matrix Q gaan we net zoals bij de Householder transformatie impliciet opslaan door enkel de c en s waarden te onthouden die horen bij de vereiste rotatiematrices.

Het is niet moeilijk na te gaan (zie oefeningen), dat de totale complexiteit van de QR factorisatie op basis van de Givens rotaties ongeveer gelijk is aan

$$6 \sum_{k=1}^n (m-k)(n-k) \sim 3mn^2 - n^3,$$

waarbij we de uitdrukking voor de bovenstaande som reeds hebben uitgerekend in de voorgaande sectie. Kortom, deze aanpak vergt 50% meer flops dan de Householder methode. Een voordeel van de Givens rotaties is wel dat je deze methode makkelijker kan paralleliseren (zo kan je verschillende rotaties die geen rijen gemeenschappelijk hebben in parallel uitvoeren). De complexiteit zoals hierboven aangegeven telt de berekeningen voor c en s niet mee omdat deze enkel bijdragen tot de term van de vorm mn .

2.5 Toepassingen

2.5.1 Oplossen van lineaire stelsels

Stel dat $A \in \mathbb{C}^{m,m}$ een matrix is met rang m (A is dus inverteerbaar) en dat we het lineaire stelsel $Ax = b$ wensen op te lossen met $b \in \mathbb{C}^m$. Dan kunnen we door gebruik te maken van de QR factorisatie van A , deze vergelijking herschrijven als $QRx = b$, of nog

$$Rx = Q^*b.$$

Indien we over de QR factorisatie van A beschikken, dan kunnen we Q^*b eenvoudig berekenen en x bepalen door het bovenstaande lineaire stelsel op

Lineaire stelsels via QR factorisatie $\sim 4m^3/3$ flops (via Householder reflecties)

1. De QR factorisatie van A te berekenen.
2. De vector $y = Q^*b$ te bepalen.
3. Het stelsel $Rx = y$ op te lossen (via achterwaartse substitutie).

Figure 2.7: QR factorisatie voor het oplossen van een lineair stelsel.

te lossen. Hierbij is R een bovendriehoeksmatrix, wat ons toelaat $Rx = Q^*b$ erg snel op te lossen (door achterwaartse substitutie, zie Sectie 4.2.3). Samengevat kunnen we $Ax = b$ dus oplossen door het algoritme in Figuur 2.7.

Dit blijkt een uitstekende numerieke manier te zijn om $Ax = b$ op te lossen (zie Sectie 4.2.7), maar deze wordt in de praktijk niet vaak gebruikt omdat Gauss eliminatie (met pivotering) slechts half zoveel flops vereist (zie Sectie 5.1).

We merken ook op dat we Q niet expliciet hoeven te berekenen in de bovenstaande procedure, het volstaat dat we Q^*b snel kunnen berekenen. Dit geeft aan dat we gebruik kunnen maken van de algoritmes in Figuur 2.5 en 2.6.

2.5.2 Kleinste kwadraten

Stel dat we wederom kijken naar de vergelijking $Ax = b$, maar nu met A een $m \times n$ matrix met $m > n$ en $b \in \mathbb{C}^m$. Dan zal deze vergelijking doorgaans geen oplossing hebben. Er is namelijk enkel een oplossing indien b een lineaire combinatie is van de kolommen van A , d.i., $b \in \langle a_1, \dots, a_n \rangle$ wat slechts een deelruimte van $\mathbb{C}^m$ is. We kunnen echter steeds trachten een x te vinden zodat het verschil tussen Ax en b zo klein mogelijk is. Dit verschil zullen we uitdrukken als de 2-norm

$$\|Ax - b\|_2.$$

We zoeken dus een x zodat $r = Ax - b$ een zo klein mogelijke 2-norm heeft. Indien $A = QR$ een volledige QR factorisatie van A voorstelt, dan wensen

Kleinste kwadraten via QR factorisatie $\sim 2mn^2 - 2n^3/3$ flops

1. Bereken de gereduceerde QR factorisatie van $A = \hat{Q}\hat{R}$.
2. Bereken de vector $\hat{Q}^*b$.
3. Los het stelsel $\hat{R}x = \hat{Q}^*b$ op naar x (m.b.v. achterwaardse substitutie).

Figure 2.8: QR factorisatie voor het kleinste kwadraten probleem.

we

$$\|r\|_2 = \|Ax - b\|_2 = \|QRx - b\|_2 = \|Rx - Q^*b\|_2,$$

te minimaliseren, want Q^* is een unitaire matrix (d.i., $Q^*Q = I$), waardoor de norm bewaard blijft bij de vermenigvuldiging met Q^* (zie Eigenschap 2.1.4). Merk op, indien $A = \hat{Q}\hat{R}$ een gereduceerde QR factorisatie is van A , dan kunnen we Q^*b en R schrijven als

$$R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} \quad \text{en} \quad Q^*b = \begin{bmatrix} \hat{Q}^* \\ Q_2^* \end{bmatrix} b,$$

met $Q_2 \in \mathbb{C}^{m,m-n}$. Bijgevolg is

$$\|r\|_2^2 = \|Rx - Q^*b\|_2^2 = \left\| \begin{bmatrix} \hat{R}x \\ 0 \end{bmatrix} - \begin{bmatrix} \hat{Q}^*b \\ Q_2^*b \end{bmatrix} \right\|_2^2 = \|\hat{R}x - \hat{Q}^*b\|_2^2 + \|Q_2^*b\|_2^2.$$

Indien we nu veronderstellen dat A rang n heeft dan is $\hat{R}$ inverteerbaar (zie ook oefeningen), waardoor $\|r\|_2^2$ wordt geminimaliseerd door de unieke oplossing van $\hat{R}x = \hat{Q}^*b$. De minimale waarde van $\|r\|_2$ is dan gelijk aan $\|Q_2^*b\|_2$.

Samengevat, indien $A = \hat{Q}\hat{R}$ een gereduceerde QR factorisatie voorstelt van $A \in \mathbb{C}^{m,n}$ met rang n , dan zal de unieke oplossing van $\hat{R}x = \hat{Q}^*b$ resulteren in een minimum voor $\|Ax - b\|_2$. Merk ook nu weer op dat we $\hat{Q}$ niet expliciet moeten bepalen, het volstaat dat we $\hat{Q}^*b$ snel kunnen berekenen. Dit resulteert in het Algoritme in Figuur 2.8.

Indien we niet zozeer wensen te weten welke x overeenstemt met een zo klein mogelijke $\|Ax - b\|_2$, maar op zoek zijn de kleinst mogelijke waarde voor $\|Ax - b\|_2$, dan moeten we enkel $\|Q_2^*b\|_2$ uitrekenen, waarbij $Q = [\hat{Q} \ Q_2] \in \mathbb{C}^{m,m}$ de Q matrix voorstelt van de volledige QR factorisatie.

De kleinst mogelijke waarde voor $\|Ax - b\|_2$ kan eveneens berekend worden door op te merken dat $x = \hat{R}^{-1}\hat{Q}^*b$ deze norm minimaliseert, wat aangeeft dat

$$\|r\|_2^2 = \|Ax - b\|_2^2 = \left\| \hat{Q} \hat{R} (\hat{R}^{-1} \hat{Q}^* b) - b \right\|_2^2 = \left\| \hat{Q} \hat{Q}^* b - b \right\|_2^2.$$

Deze laatste uitdrukking voor $\|r\|_2^2$ is sneller uit te rekenen dan $\|Q_2^*b\|_2^2$ indien $m \gg n$.

2.5.3 Hessenberg decompositie

Deze laatste toepassing is geen toepassing van de QR factorisatie, maar geeft aan dat de Householder reflecties ook voor andere doeleinden erg nuttig kunnen zijn. In deze sectie zullen we een algoritme bespreken dat gegeven $A \in \mathbb{C}^{m,m}$, toelaat een unitaire matrix Q te bepalen zodat

$$A = QHQ^*,$$

waarbij H een boven-Hessenberg matrix is. Een matrix $H \in \mathbb{C}^{m,m}$ is een boven-Hessenberg matrix indien $h_{ij} = 0$ voor $i > j + 1$, voor alle $i = 3, \dots, m$.

Dit algoritme zal bestaan uit $m - 2$ stappen. We starten met A te noteren als A_2 . Tijdens de eerste stap bepalen we de Householder reflectie matrix $\hat{P}_2 \in \mathbb{C}^{m-1, m-1}$ die ervoor zorgt dat

$$P_2 A_2 = \begin{bmatrix} 1 & 0 \\ 0 & \hat{P}_2 \end{bmatrix} A_2,$$

een matrix is waarvan de laatste $m - 2$ elementen van de eerste kolom gelijk zijn aan nul. Vervolgens berekenen we $A_3 = (P_2 A_2) P_2^*$, waarbij $P_2^* = P_2$, daar $\hat{P}_2$ Hermitisch is. Omwille van de structuur van de eerste kolom van P_2 zien we dat de eerste kolom van $A_3 = (P_2 A_2) P_2$ en $P_2 A_2$ identiek zijn. Met andere woorden de $m - 2$ nullen die P_2 heeft gecreëerd blijven behouden.

Tijdens de tweede stap bepalen we de Householder reflectie matrix $\hat{P}_3 \in \mathbb{C}^{m-2, m-2}$ die ervoor zorgt dat

$$P_3 A_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \hat{P}_3 \end{bmatrix} A_3,$$

een matrix is waarvan de laatste $m - 3$ elementen van de tweede kolom gelijk zijn aan nul. Merk op, de $m - 2$ nullen op de eerste kolom blijven eveneens behouden. De matrix A_4 berekenen we dan als $A_4 = P_3 A_3 P_3^* = P_3 A_3 P_3$.

Wanneer we deze procedure in totaal $m - 2$ keer uitvoeren, dan zal $H = A_m$ gegeven door

$$H = (P_{m-1} P_{m-2} \dots P_2) A (P_2 P_3 \dots P_{m-1}),$$

een boven-Hessenberg matrix zijn. Kortom indien we $Q = P_2 \dots P_{m-1}$ dan hebben we de gevraagde Hessenberg decompositie

$$A = Q H Q^*,$$

gevonden, daar Q unitair is.

2.6 Oefeningen

OEFENING 2.6.1. Gram-Schmidt orthogonalisatie

1. Schrijf een Matlab functie `[Q,R]=QR_clasGS(A)` die de QR factorisatie op basis van het klassieke Gram-Schmidt algoritme implementeert, alsook een functie `[Q,R]=QR_modGS(A)` voor het aangepaste algoritme.
2. Laat A een $m \times m$ matrix zijn met zijn j -de kolom gelijk aan a_j , voor $j = 1, \dots, m$. Toon de ongelijkheid van Hadamard aan

$$|\det(A)| \leq \prod_{j=1}^m \|a_j\|_2.$$

Maak gebruik van Eigenschap 2.1.6.

3. Laat A een $m \times n$ matrix zijn en $A = \hat{Q} \hat{R}$ een gereduceerde QR factorisatie van A .

- a. Argumenteer dat A rang n heeft als en slechts als alle elementen op de diagonaal van R verschillen van nul.
- b. Indien er op de diagonaal van R exact k elementen staan verschillend van nul, wat weet je dan over de rang van A (is deze gelijk aan k , minstens k of hoogstens k ?).

OEFENING 2.6.2. Householder reflecties

1. Laat $P = I - \frac{vv^*}{v^*v}$, voor $v \in \mathbb{C}^m$ (merk op, er staat hier geen 2 zoals bij de Householder reflectie). Ga na of P Hermitisch en/of unitair is. Toon aan dat het beeld Px van een vector x onder P gelegen is in het hypervlak orthogonaal aan v (door aan te geven dat $v^*Px = 0$). Bewijs dat P een vector x loodrecht projecteert op dit hypervlak (door aan te geven dat $x - Px$ een veelvoud is van v).
2. Implementeer een Matlab functie `[W,R]=QR_house(A)` die de QR factorisatie van A berekent met behulp van de Householder reflectie. W is een benedendriehoeksmatrix die de vectoren $\hat{v}_1$ tot $\hat{v}_n$ bevat. Zorg dat je code ook werkt voor complexe matrices A .
3. Schrijf een Matlab functie `formQ_house(W)` die de W matrix uit de voorgaande opgave omzet in de unitaire Q matrix van de QR factorisatie van A .
4. Gebruik je functies `QR_modGS`, `QR_house` en `formQ_house` om een gereduceerde QR factorisatie van Z te berekenen met

$$Z = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 7 \\ 4 & 2 & 3 \\ 4 & 2 & 2 \end{bmatrix}.$$

Vergelijk de resultaten met de output van de ingebouwde Matlab functie `qr`.

OEFENING 2.6.3. Givens rotaties

1. Implementeer een Matlab functie `[C,S,R]=QR_givens(A)` die de QR factorisatie van A berekent met behulp van de Givens rotatie. C

en S zijn benedendriehoeksmatrices die de c en s waarden van de gehanteerde Givens rotaties bevatten. Wat is de tijdscomplexiteit van je implementatie?

2. Schrijf een Matlab functie `formQ_givens(C,S)` die de C en S matrices uit de voorgaande opgave omzet in de unitaire Q matrix van de QR factorisatie van A .

OEFENING 2.6.4. Hessenberg decompositie

1. Implementeer een Matlab functie `[W,H]=hess_house(A)` die een Hessenberg decompositie van A berekent op basis van Householder reflecties. $W \in \mathbb{C}^{m-1,m-2}$ is een benedendriehoeksmatrix die de vectoren $\hat{v}_1$ tot $\hat{v}_{m-2}$ bevat die met de reflectie matrices $\hat{P}_2, \dots, \hat{P}_{m-1}$ overeenkomen. Zorg dat je code ook werkt voor complexe matrices A . Toon aan dat de complexiteit van dit algoritme ongeveer gelijk is aan $10m^3/3$ (in het reële geval). Schrijf eveneens een functie die Q expliciet uitrekent op basis van W . Vergelijk je resultaten met de ingebouwde Matlab functie `hess(A)`.

SINGULIERE WAARDEN ONTBINDING

3.1 Definitie en geometrische interpretatie

In het voorgaande hoofdstuk zagen we dat elke matrix $A \in \mathbb{C}^{m,n}$ met $m \geq n$, kon geschreven worden als $A = QR$ met Q unitair en R een bovendriehoeksmatrix. In dit hoofdstuk zullen we aangeven dat A eveneens ontbonden kan worden als

$$A = U\Sigma V^*,$$

waarbij $U \in \mathbb{C}^{m,m}$ unitair is, $\Sigma \in \mathbb{R}^{m,n}$ diagonaal, met diagonaalelementen $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$, en $V^* \in \mathbb{C}^{n,n}$ eveneens unitair. Merk op dat $m \geq n$ niet echt een beperking is in dit geval, daar we eerst de ontbinding van A^* kunnen bepalen en de complex geconjugeerde kunnen nemen van dit resultaat. Een ontbinding of factorisatie van A die voldoet aan de bovenstaande definitie noemen we een singuliere waarden ontbinding of *singular value decomposition (SVD)* in het Engels. Daar de afkorting SVD zeer veel gehanteerd wordt in de literatuur, zullen wij deze eveneens hanteren als afkorting voor de singuliere waarden ontbinding.

Voor we overgaan tot het bewijs dat elke matrix $A \in \mathbb{C}^{m,n}$ een SVD heeft, zullen we eerst even stilstaan bij de geometrische interpretatie van deze ontbinding **in het reële geval**. Wanneer we een reële $m \times n$ matrix A nemen, dan weten we dat die overeenstemt met een lineaire afbeelding van $\mathbb{R}^n$ naar $\mathbb{R}^m$. Wanneer we nu de eenheidssfeer S in $\mathbb{R}^n$ bekijken (S is de verzameling punten op afstand 1 van de oorsprong), dan kunnen we ons afvragen wat het beeld is van deze sfeer S onder A .

Het beeld van deze sfeer S zal precies overeenstemmen met een hyperellips in $\mathbb{R}^m$, omwille van het bestaan van de SVD van A zoals zo dadelijk zal blijken. Een hyperellips is een veralgemening van de klassieke ellips in $\mathbb{R}^2$, die bestaat uit een lange as met lengte $2a$ en een korte as met lengte $2b$ (beide assen kunnen ook even lang zijn, in dit geval verkrijgen we een cirkel met

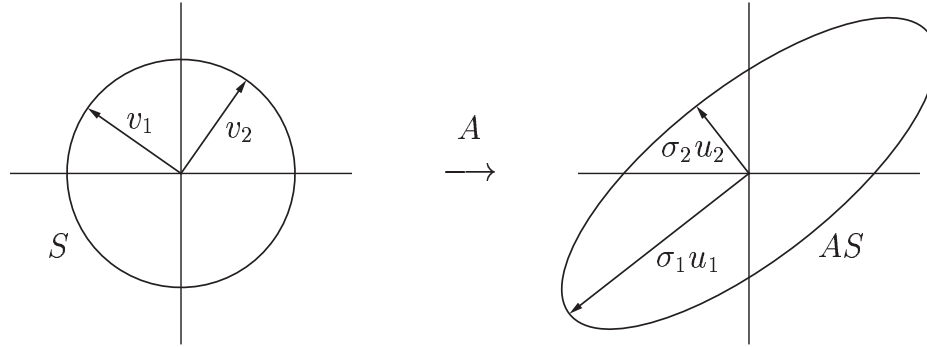


Figure 3.9: SVD van een 2×2 matrix.

straal $a = b$). Kortom, een ellips in $\mathbb{R}^2$ wordt volledig gekarakteriseerd door twee orthonormale vectoren u_1 en u_2 die de richting van de lange en korte as aangeven, alsook door de lengtes $\sigma_1 = a$ en $\sigma_2 = b$ (met $\sigma_1 \geq \sigma_2$).

Een hyperellips in $\mathbb{R}^m$ wordt volledig analoog gedefinieerd door een set $\{u_1, \dots, u_m\}$ van orthonormale vectoren en lengtes $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$. Merk op, een aantal van deze lengtes kunnen gelijk zijn aan nul. We zullen zien dat er precies r lengtes $\sigma_1, \dots, \sigma_r$ zullen verschillen van nul wanneer A rang r heeft.

Om nu te begrijpen waarom het beeld van S onder A een hyperellips is wanneer $A = U\Sigma V^*$ moeten we opmerken dat we het beeld van S onder A kunnen bekomen door eerst het beeld onder V^* te nemen, vervolgens Σ hierop los te laten en tenslotte U toe te passen. De afbeelding V^* zal S afbeelden op zichzelf want V^* is unitair en bewaart bijgevolg de 2-norm (zie Eigenschap 2.1.4). Σ zal de sfeer S uitrekken tot een hyperellips waarvan de assen samenvallen met $e_1, \dots, e_m$ en de unitaire map U zal deze hypersfeer roteren of reflecteren (afhankelijk van het teken van de determinant van U), waardoor de assen samen zullen vallen met $u_1, \dots, u_m$.

3.1.1 Gereduceerde SVD

Indien we veronderstellen dat $m \geq n$ en dat A rang n heeft, dan kunnen we volgende definities invoeren. Deze worden eveneens grafisch weergegeven in Figuur 3.9 wanneer $m = n = 2$.

Ten eerste zullen we verwijzen naar de n lengtes $\sigma_1, \dots, \sigma_n$ als de n singuliere waarden van A , waarbij $\sigma_1 \geq \dots \geq \sigma_n$ (daar $m \geq n$ zal $\sigma_{n+1}, \dots, \sigma_m = 0$). Verder definiëren we de n orthonormale assen $u_1, \dots, u_n$ van de hyper-

ellips als de n *linkse* singuliere vectoren van A (links omdat U links staat in de ontbinding $A = U\Sigma V^*$). Tot slot definiëren we de n *rechtse* singuliere vectoren van A als de n vectoren $\{v_1, \dots, v_n\} \in S$ die worden afgebeeld op de assen van de hyperellips, d.i., $Av_j = \sigma_j u_j$ voor $j = 1, \dots, n$. Deze set van vectoren $\{v_1, \dots, v_n\} \in S$ zal eveneens een orthonormale set van vectoren vormen.

Merk op dat V^* de set $\{v_1, \dots, v_n\}$ eerst zal afbeelden op $\{e_1, \dots, e_n\}$, dat Σ deze afbeeldt op $\{\sigma_1 e_1, \dots, \sigma_n e_n\} \in \mathbb{R}^m$ en dat U tenslotte $\sigma_j e_j$ afbeeldt op $\sigma_j u_j$ voor $j = 1, \dots, n$.

De gelijkheden $Av_j = \sigma_j u_j$, voor $j = 1, \dots, n$, kunnen we ook in matrixvorm weergeven

$$A \begin{bmatrix} | & | & & | \\ v_1 & v_2 & \dots & v_n \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ u_1 & u_2 & \dots & u_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \end{bmatrix}.$$

of kortweg als

$$AV = \hat{U}\hat{\Sigma},$$

waarbij $\hat{\Sigma} \in \mathbb{R}^{n,n}$ een diagonaalmatrix is, $\hat{U}$ een $m \times n$ matrix bestaande uit n orthonormale kolommen en V een unitaire $n \times n$ matrix. Bijgevolg is $V^*V = I$, waardoor we de bovenstaande vergelijking kunnen herschrijven als

$$A = \hat{U}\hat{\Sigma}V^*,$$

waarbij de kolommen van $\hat{U}$ een orthonormale set vormen, V unitair is en $\hat{\Sigma}$ een reële $n \times n$ diagonaalmatrix, waarvan de elementen $\sigma_1, \dots, \sigma_n$ voldoen aan $\sigma_1 \geq \dots \geq \sigma_n \geq 0$. Een factorisatie van deze vorm voor een $m \times n$ matrix A , met $m \geq n$ en $\text{rang}(A) = n$, noemen we de gereduceerde SVD van A . Deze definitie geldt eveneens voor complexe matrices A .

3.1.2 Volledige SVD

Net zoals bij de QR factorisatie, kunnen we de gereduceerde SVD eenvoudig uitbreiden tot een volledige SVD door de orthonormale set van vectoren

$u_1, \dots, u_n$ uit te breiden tot een basis $u_1, \dots, u_m$ voor $\mathbb{C}^m$. Om te komen tot een volledige SVD van de vorm

$$A = U\Sigma V,$$

waarbij U een unitaire $m \times m$ matrix is met zijn j -de kolom gelijk aan u_j , kunnen we Σ definiëren door aan $\hat{\Sigma}$ eveneens $m - n$ rijen bestaande uit nullen toe te voegen. Met andere woorden, we zetten

$$U = \left[\begin{array}{c|c|c|c} \hat{U} & & & \\ \hline & u_{n+1} & \dots & u_m \end{array} \right] \quad \text{en} \quad \Sigma = \left[\begin{array}{cccc} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \end{array} \right].$$

Tot nu toe hebben we veronderstelt dat de rang van A gelijk was aan n (en $m \geq n$, maar dit laatste is geen echte beperking).

Wanneer nu de rang van A gelijk is aan $r < n$, dan zal $\sigma_{r+1}, \dots, \sigma_n = 0$, waardoor de hyperellips volledig bepaald wordt door $u_1, \dots, u_r$ en $\sigma_1 \geq \dots \geq \sigma_r \geq 0$. In dit geval bestaat de gereduceerde SVD uit de $m \times r$ matrix $\hat{U}$ gevormd door $u_1, \dots, u_r$, de $r \times r$ diagonaalmatrix $\hat{\Sigma}$ met diagonaalelementen $\sigma_1 \geq \dots \geq \sigma_r$, terwijl V een $n \times r$ matrix is bestaande uit de r rechtse singuliere vectoren $v_1, \dots, v_r$ die worden afgebeeld op de assen van de ellips, d.i., $Av_j = \sigma_j u_j$, voor $j = 1, \dots, r$. Een volledige SVD van een rang $r < n$ matrix A wordt dan bekomen door $m - r$ vectoren $u_{r+1}, \dots, u_m$ toe te voegen aan $\hat{U}$, $n - r$ vectoren $v_{r+1}, \dots, v_n$ aan V en $m - r$ rijen en $n - r$ kolommen met nullen aan $\hat{\Sigma}$.

3.2 Existentie

In deze sectie zullen we bewijzen dat er een volledige SVD bestaat voor elke matrix $A \in \mathbb{C}^{m,n}$. Het belang van deze ontbinding wordt besproken in de volgende sectie.

EIGENSCHAP 3.2.1. *Elke matrix $A \in \mathbb{C}^{m,n}$ heeft een volledige singuliere waarden ontbinding $A = U\Sigma V$ (en bijgevolg ook een gereduceerde).*

Bewijs. Laat σ_1 gelijk zijn aan de matrixnorm van A geïnduceerd door de 2-norm, d.i., $\sigma_1 = \|A\|_2 = \sup_{x \in \mathbb{C}, \|x\|_2=1} \|Ax\|_2$. Daar de eenheidssfeer gesloten is en A een continue afbeelding, mogen we dit supremum vervangen door een maximum. Er bestaat dus een $v_1 \in \mathbb{C}^n$ met $\|v_1\|_2 = 1$, zodat $\|Av_1\|_2 = \sigma_1$. Laat $u_1 = (Av_1)/\sigma_1$, dan is $u_1 \in \mathbb{C}^m$, $\|u_1\|_2 = 1$ en $Av_1 = \sigma_1 u_1$.

Kies nu $\{v_2, \dots, v_n\} \in \mathbb{C}^n$ willekeurige, zodat $\{v_1, \dots, v_n\}$ een orthonormale basis is voor $\mathbb{C}^n$ en kies $\{u_2, \dots, u_m\} \in \mathbb{C}^m$ zodat $\{u_1, \dots, u_m\}$ een orthonormale basis is voor $\mathbb{C}^m$. De matrices U_1 en V_1 waarvan respectievelijk de j -de kolom gelijk is aan u_j en v_j , zijn dan unitair. Het product $U_1^* Av_1$ is dan gelijk aan

$$U_1^* Av_1 = \sigma_1 U_1^* u_1 = \sigma_1 e_1,$$

daar $u_j^* u_1 = 0$ voor $j > 1$ en $u_1^* u_1 = 1$. Bijgevolg is $S = U_1^* AV_1$ van de vorm

$$S = U_1^* AV_1 = \begin{bmatrix} \sigma_1 & w^* \\ 0 & B \end{bmatrix},$$

waarbij w^* een rijvector is van lengte $n-1$ en $B \in \mathbb{C}^{m-1, n-1}$. Verder zien we dat

$$\left\| \begin{bmatrix} \sigma_1 & w^* \\ 0 & B \end{bmatrix} \begin{bmatrix} \sigma_1 \\ w \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} \sigma_1^2 + w^* w \\ Bw \end{bmatrix} \right\|_2 \geq \sigma_1^2 + w^* w = (\sigma_1^2 + w^* w)^{1/2} \left\| \begin{bmatrix} \sigma_1 \\ w \end{bmatrix} \right\|_2.$$

Per definitie van de geïnduceerde matrixnorm mogen we hieruit besluiten dat $\|S\|_2 \geq (\sigma_1^2 + w^* w)^{1/2}$. Echter, $S = U_1^* AV_1$, met U_1^* en V_1 unitair, wat aangeeft $\|S\|_2 = \|A\|_2 = \sigma_1$ (zie Eigenschap 2.1.3). Het kan dus niet anders dan dat $w = 0$.

Wanneer $n = 1$ of $m = 1$ dan verkrijgen we de gevraagde SVD door $U = U_1$ en $V^* = V_1^*$. Anders mogen we per inductie aannemen dat B een volledige SVD heeft van de vorm $B = U_2 \Sigma_2 V_2^*$. Hierdoor kunnen we A schrijven als

$$A = U_1 S V_1^* = \underbrace{U_1 \begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix}}_U \underbrace{\begin{bmatrix} \sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix}}_\Sigma \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & V_2 \end{bmatrix}^*}_{V^*} V_1^*,$$

wat een SVD is van A . Tot slot merken we op dat $\|A\|_2 = \|S\|_2$ met $w^* = 0$, wat aangeeft dat $\|A\|_2 \geq \|B\|_2$. Hierdoor mogen we per inductie besluiten dat $\sigma_1 \geq \dots \geq \sigma_n \geq 0$. $\square$

Men kan ook aantonen dat alle mogelijke SVDs van A gebruik maken van dezelfde matrix Σ , kortom elke matrix $A \in \mathbb{C}^{m,n}$ heeft exact n (niet noodzakelijk verschillende) singuliere waarden groter of gelijk aan nul. Net zoals elke matrix $A \in \mathbb{C}^{m,m}$ exact m (niet noodzakelijk verschillende) eigenwaarden heeft (zie Cursus Lineaire Algebra). Wanneer $A \in \mathbb{C}^{m,m}$ en alle m singuliere waarden zijn verschillend, dan zullen ook de vectoren u_j en v_j (en dus de matrices U en V) uniek bepaald zijn op vermenigvuldiging met $z \in \mathbb{C}$ waarvoor $|z| = 1$ na. Dit laatste is in het reële geval eigenlijk een direct gevolg van onze geometrische interpretatie: als alle assen van de hyperellips een verschillende lengte hebben dan kunnen we enkel de richting van de assen omkeren. Is echter $\sigma_i = \sigma_{i+1}$ voor een $i < m$, dan kan er steeds gekozen worden voor een andere orthonomale basis die dezelfde ruimte opspant als de vectoren u_i en u_{i+1} , waardoor de matrix U (en V) niet uniek is, zelfs al rekenen we de eerder vermelde vermenigvuldigingen met z niet mee.

We eindigen deze sectie met aan te geven dat het bestaan van de SVD voor elke matrix $A \in \mathbb{C}^{m,n}$ eigenlijk wil zeggen dat alle matrices diagonaalmatrices zijn indien we een geschikte basis kiezen voor $\mathbb{C}^n$ en $\mathbb{C}^m$. In Sectie 2.1, vlak na Eigenschap 2.1.3, merken we reeds op dat Q^*b , met Q een unitaire matrix, eigenlijk de coördinaten zijn van b ten opzichte van de basis gevormd door de kolommen van Q . Indien we nu

$$\tilde{b} = U^*b \quad \text{en} \quad \tilde{x} = V^*x,$$

dan geeft $\tilde{b}$ dus de coördinaten weer van $b \in \mathbb{C}^m$ ten opzichten van de basis $\{u_1, \dots, u_m\}$, terwijl $\tilde{x}$ de coördinaten bevat van $x \in \mathbb{C}^n$ ten opzichten van de basis $\{v_1, \dots, v_n\}$. Wanneer we nu de gelijkheid $Ax = b$ nemen en we gebruiken de SVD van $A = U\Sigma V^*$ dan zien we dat

$$Ax = b \Leftrightarrow U\Sigma V^*x = b \Leftrightarrow \Sigma\tilde{x} = U^*b = \tilde{b}.$$

Kortom, $Ax = b$ is hetzelfde als $\Sigma\tilde{x} = \tilde{b}$, wat aangeeft dat de afbeelding A met Σ overeenstemt indien we de juiste basis nemen voor $\mathbb{C}^m$ en $\mathbb{C}^n$.

Merk op, analoog zien we dat het bestaan van een QR factorisatie van $A = QR$, met $A \in \mathbb{C}^{m,n}$, aangeeft dat we elke matrix kunnen schrijven als een bovendriehoeksmatrix door Q als basis te kiezen voor $\mathbb{C}^m$ (de beeldruimte). Namelijk, wanneer $b = Ax = QRx$, dan is $\tilde{b} = Rx$, waarbij $\tilde{b} = Q^*b$ de coördinaten van b bevat ten opzichte van de basis Q .

3.3 Eigenschappen

In deze sectie bespreken we enkele belangrijke eigenschappen van de SVD van een matrix A .

EIGENSCHAP 3.3.1. *De rang van een matrix $A \in \mathbb{C}^{m,n}$ is gelijk aan het aantal singuliere waarden dat verschilt van nul.*

Bewijs. In de cursus Lineaire Algebra werd aangetoond dat de rang van CB gelijk is aan de rang van B , indien B een $m \times n$ matrix is en C een inverteerbare $m \times m$ matrix. Dit geldt natuurlijk ook voor de rang van B en BD wanneer D een inverteerbare $n \times n$ matrix is. Kortom, A links of rechts vermenigvuldigen met een inverteerbare matrix wijzigt de rang van A niet. Aangezien $A = U\Sigma V^*$ met U^* en V inverteerbaar, is de rang van A gelijk aan deze van Σ , een diagonaalmatrix. De rang van Σ is duidelijk gelijk aan het aantal diagonaalelementen dat verschilt van nul. $\square$

De bovenstaande eigenschap geeft aan dat we de (numerieke) rang van een matrix A kunnen bepalen door de SVD te berekenen. Merk wel op dat we in een numerieke omgeving een goedgekozen $\epsilon > 0$ zullen moeten hanteren, waarbij we zullen stellen dat een singuliere waarde als nul wordt aanzien indien deze kleiner is dan ϵ .

We zullen de beeldruimte van A noteren als $\text{range}(A) = \{Ax | x \in \mathbb{C}^n\}$ en de kern als $\text{null}(A) = \{x \in \mathbb{C}^n | Ax = 0\}$. Merk op, in de cursus Lineaire Algebra werden deze genoteerd als $\text{Im}(A)$ en $\text{Ker}(A)$. De volgende eigenschap geeft aan dat de beeldruimte en de kern van A makkelijk uit te drukken zijn op basis van de SVD van A .

EIGENSCHAP 3.3.2. *Laat $A = U\Sigma V^*$ een rang r matrix zijn, dan is $\text{range}(A) = \langle u_1, \dots, u_r \rangle$ en $\text{null}(A) = \langle v_{r+1}, \dots, v_n \rangle$.*

Bewijs. V^* is unitair waardoor $\text{range}(V^*) = \mathbb{C}^n$, de beeldruimte van Σ is gelijk aan $\langle e_1, \dots, e_r \rangle$ (dankzij de vorige eigenschap), terwijl U de vector e_j afbeeldt op u_j , voor $j = 1, \dots, m$. Hieruit volgt dat $\text{range}(A) = \langle u_1, \dots, u_r \rangle$. Daar U unitair is, is de kern van A gelijk aan deze van ΣV^* en $\text{null}(\Sigma) = \langle e_{r+1}, \dots, e_n \rangle$, terwijl V^* de vector v_j afbeeldt op e_j , voor $j = 1, \dots, n$. $\square$

De 2-norm en Frobenius norm van A zijn ook eenvoudig uit te drukken in functie van de singuliere waarden $\sigma_1 \geq \dots \geq \sigma_r$:

EIGENSCHAP 3.3.3. $\|A\|_2 = \sigma_1$ en $\|A\|_F = \sqrt{\sigma_1^2 + \dots + \sigma_r^2}$

Bewijs. De eerste gelijkheid volgt uit het bewijs van Eigenschap 3.2.1. Door gebruik te maken van Eigenschap 2.1.5, kunnen we tonen dat $\|A\|_F = \|\Sigma\|_F$ en per definitie van de Frobenius norm zien we dat $\|\Sigma\|_F = \sqrt{\sigma_1^2 + \dots + \sigma_r^2}$. $\square$

EIGENSCHAP 3.3.4. *Laat $A \in \mathbb{C}^{m,n}$, dan is $\|A\|_* = \sum_{i=1}^n \sigma_i$ een matrixnorm. Deze norm wordt de nuclear of trace norm genoemd.*

Bewijs. We zien dat $\|A\|_* \geq 0$ en $\|A\|_* = 0$ als en slechts als $A = 0$. Het is ook eenvoudig om na te gaan dat $\|zA\|_* = |z| \|A\|_*$. We moeten nu nog aantonen dat $\|A + B\|_* \leq \|A\|_* + \|B\|_*$. Hiervoor tonen we eerst dat

$$\|A\|_* = \sup_{Q: \|Q\|_2 \leq 1} \operatorname{tr}(Q^* A),$$

met $Q \in \mathbb{C}^{m,n}$. Uit het feit dat

$$\operatorname{tr}(CD) = \sum_i (CD)_{ii} = \sum_i \sum_j c_{ij} d_{ji} = \sum_j \sum_i d_{ji} c_{ij} = \sum_j (DC)_{jj} = \operatorname{tr}(DC),$$

volgt dat

$$\operatorname{tr}(Q^* A) = \operatorname{tr}(Q^* U \Sigma V^*) = \operatorname{tr}((V^* Q^* U) \Sigma) = \sum_{i=1}^n (V^* Q^* U)_{ii} \sigma_i,$$

gezien Σ een diagonaalmatrix is. Verder is $\|V^* Q^* U\|_2 = \|Q\|_2$, waardoor we weten dat $(V^* Q^* U)_{ii} \leq 1$. Kortom,

$$\sup_{Q: \|Q\|_2 \leq 1} \operatorname{tr}(Q^* A) = \sup_{Q: \|Q\|_2 \leq 1} \sum_{i=1}^n (V^* Q^* U)_{ii} \sigma_i \leq \sup_{Q: \|Q\|_2 \leq 1} \sum_{i=1}^n \sigma_i = \|A\|_*.$$

Deze ongelijkheid is echter een gelijkheid indien we $Q = UV^*$ aangezien $V^* Q^* U$ dan gelijk is aan I . We besluiten bijgevolg dat

$$\|A + B\|_* = \sup_{Q: \|Q\|_2 \leq 1} \operatorname{tr}(Q^* (A + B)) = \sup_{Q: \|Q\|_2 \leq 1} (\operatorname{tr}(Q^* A) + \operatorname{tr}(Q^* B)) \quad (3.4)$$

$$\leq \sup_{Q: \|Q\|_2 \leq 1} \operatorname{tr}(Q^* A) + \sup_{Q: \|Q\|_2 \leq 1} \operatorname{tr}(Q^* B) = \|A\|_* + \|B\|_*. \quad (3.5)$$

$\square$

De singuliere waarden van A kunnen ook gerelateerd worden aan de eigenwaarden van de Hermitische matrix A^*A (of AA^* want AA^* en A^*A hebben dezelfde eigenwaarden).

EIGENSCHAP 3.3.5. *De singuliere waarden $\sigma_1 \geq \dots \geq \sigma_r$ zijn de vierkantswortels van de eigenwaarden van A^*A die verschillen van nul.*

Bewijs. Gebruik makend van de SVD van A zien we

$$A^*A = (U\Sigma V^*)^*U\Sigma V^* = V\Sigma U^*U\Sigma V^* = V\Sigma^2 V^*.$$

De matrices A^*A en Σ^2 zijn dus equivalent waardoor ze dezelfde karakteristieke veelterm hebben en dus ook dezelfde eigenwaarden (zie cursus Lineaire Algebra). De eigenwaarden van Σ^2 die verschillen van nul, zijn duidelijk gelijk aan $\sigma_1^2, \dots, \sigma_r^2$. $\square$

Een onmiddellijk gevolg hiervan is dat als A Hermitisch is, d.i., $A = A^*$, dan zijn de singuliere waarden van A de absolute waarde van de eigenwaarden van A . Dit volgt uit het feit dat de eigenwaarden van Σ^2 dan identiek zijn aan deze van A^2 en deze laatste zijn duidelijk de kwadraten van de eigenwaarden van A , daar $x A = \lambda x$ impliceert dat $x A^2 = \lambda^2 x$. Verder zien we ook dat Eigenschap 1.2.3 volgt uit Eigenschap 3.3.3 en Eigenschap 3.3.5.

Tot slot kunnen we ook de absolute waarde van de determinant van A uitdrukken met behulp van de singuliere waarden:

EIGENSCHAP 3.3.6. *Voor $A \in \mathbb{C}^{m,m}$ zien we dat $|\det(A)| = \prod_{j=1}^m \sigma_j$.*

Bewijs. We weten dat $\det(AB) = \det(A)\det(B)$, bijgevolg is $\det(A) = \det(U)\det(\Sigma)\det(V^*)$, d.i.,

$$|\det(A)| = |\det(U)| \left(\prod_{j=1}^m \sigma_j \right) |\det(V^*)|.$$

Verder is $|\det(Q)| = 1$ met Q unitair, omwille van Eigenschap 2.1.6. $\square$

3.4 Toepassingen

3.4.1 Lage-rang approximaties

Hoewel de voorgaande eigenschappen erg nuttig zijn en de link leggen tussen de SVD en heel wat fundamentele begrippen uit de lineaire algebra, zijn de

meeste toepassingen van de SVD waarschijnlijk gebaseerd de volgende twee eigenschappen.

EIGENSCHAP 3.4.1. *Laat A een rang r matrix zijn met SVD $A = U\Sigma V^*$, dan is*

$$A = \sum_{j=1}^r \sigma_j u_j v_j^*,$$

waarbij σ_j de j -de singuliere waarde, u_j de j -de kolom van U en v_j^* de j -de rij van V^* voorstelt.

Bewijs. Dankzij Eigenschap 3.3.1 weten we dat $\sigma_r > \sigma_{r+1} = \dots = \sigma_n = 0$. Laat $\Sigma = \Sigma_1 + \dots + \Sigma_r$, waarbij $\Sigma_j = \sigma_j e_j e_j^*$ (Σ_j bevat slechts één element verschillend van nul), dan is $A = \sum_{j=1}^r U \Sigma_j V^*$. Het product $U \Sigma_j V^* = \sigma_j (U e_j)(e_j^* V^*) = \sigma_j u_j (V e_j)^* = \sigma_j u_j v_j^*$, wat het gestelde bewijst. $\square$

Deze eigenschap geeft aan dat we A kunnen schrijven als een som van r rang één matrices. Op zich is dit niets bijzonders, namelijk als A een $m \times n$ matrix is, dan kunnen we A ook schrijven als een som van n rang één matrices:

$$A = \sum_{j=1}^n a_j e_j^*,$$

met a_j de j -de kolom van A . Wat de bovenstaande ontbinding echter bijzonder maakt, is dat deze ons ook de best mogelijke benadering van A oplevert met behulp van een matrix met een rang $k < r$, zoals aangegeven in de volgende eigenschap.

EIGENSCHAP 3.4.2. *Laat A een rang r matrix zijn met SVD $A = U\Sigma V^*$. Definieer A_k , voor $1 \leq k \leq r$ als*

$$A_k = \sum_{j=1}^k \sigma_j u_j v_j^*,$$

waarbij σ_j de j -de singuliere waarde, u_j de j -de kolom van U en v_j^* de j -de rij van V^* voorstelt. Dan is

$$\|A - A_k\|_2 = \min_{B \in \mathbb{C}^{m,n}, \text{rang}(B) \leq k} \|A - B\|_2 = \sigma_{k+1},$$

met $\sigma_{r+1} = 0$ indien $r = n$.

Bewijs. Omwille van Eigenschap 3.3.3 weten we dat $\|A - A_k\|_2 = \sigma_{k+1}$. Stel dat $B \in \mathbb{C}^{m,n}$ met $\text{rang}(B) \leq k$ zodat $\|A - B\|_2 < \|A - A_k\|_2 = \sigma_{k+1}$. Aangezien $\text{rang}(B) \leq k$, volgt uit Eigenschap 3.3.2 dat $\text{null}(B)$ minstens dimensie $n - k$ heeft. Er bestaat dus een $n - k$ dimensionale deelruimte W van $\mathbb{C}^n$, zodat $Bw = 0$ voor $w \in W$. Bijgevolg is $Aw = (A - B)w$, voor $w \in W$, wat impliceert dat

$$\|Aw\|_2 = \|(A - B)w\|_2 \leq \|A - B\|_2 \|w\|_2 < \sigma_{k+1} \|w\|_2,$$

voor $w \in W$, waar de eerste ongelijkheid volgt uit Eigenschap 1.2.4. Kortom, er bestaat een $n - k$ dimensionale deelruimte W van $\mathbb{C}^n$ zodat $\|Aw\|_2 < \sigma_{k+1} \|w\|_2$.

Anderzijds weten we dat $\langle v_1, \dots, v_{k+1} \rangle$ een $k + 1$ dimensionale deelruimte W' vormt van $\mathbb{C}^n$. Indien $w \in W'$, dan kunnen we deze schrijven als $w = Vx$ met $x \in \mathbb{C}^m$ en $x_{k+2} = \dots = x_m = 0$. Bijgevolg is $Aw = U\Sigma x \geq \sigma_{k+1} Ux$, zodat

$$\|Aw\|_2 \geq \sigma_{k+1} \|Ux\|_2 = \sigma_{k+1} \|x\|_2 = \sigma_{k+1} \|Vx\|_2 = \sigma_{k+1} \|w\|_2,$$

voor $w \in W'$, aangezien U en V unitaire matrices zijn.

De som van de dimensies van de twee deelruimtes W en W' kan echter niet groter zijn dan n , waardoor het gestelde (zijnde het bestaan van B) niet klopt. $\square$

3.4.2 Kleinste kwadraten

In deze sectie bekijken we net zoals in Sectie 2.5.2 de vergelijking $Ax = b$ met A een $m \times n$ matrix met $m > n$ en $b \in \mathbb{C}^m$, dewelke doorgaans geen oplossing heeft. Onze betrachting is opnieuw om de 2-norm

$$\|Ax - b\|_2,$$

te minimaliseren. Indien $A = U\Sigma V^*$ een volledige SVD van A voorstelt, dan wensen we

$$\|r\|_2 = \|Ax - b\|_2 = \|U\Sigma V^*x - b\|_2 = \|\Sigma(V^*x) - U^*b\|_2,$$

te minimaliseren, want U^* is een unitaire matrix, waardoor de norm bewaard blijft bij de vermenigvuldiging met U^* .

Kleinste kwadraten via SVD ($\text{rang}(A) = r$) $\sim 2mn^2 + 11n^3$ flops

1. Bereken de gereduceerde SVD van $A = U_r \Sigma_r V_r^*$.
2. Bereken de vector $U_r^* b$.
3. Los het diagonaalstelsel $\Sigma_r y = U_r^* b$ op naar y .
4. Zet $x = V_r y$.

Figure 3.10: SVD voor het kleinste kwadraten probleem.

Laat $\text{rang}(A) = r \leq n$, dan kunnen we Σ , $V^* x$ en $U^* b$ schrijven als

$$\Sigma = \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix}, \quad V^* x = \begin{bmatrix} V_r^* \\ V_2^* \end{bmatrix} x \quad \text{en} \quad U^* b = \begin{bmatrix} U_r^* \\ U_2^* \end{bmatrix} b,$$

waarbij Σ_r een diagonaalmatrix is die de r singuliere waarden bevat, U_r de eerste r kolommen van $U = [U_r \ U_2]$ bevat en V_r^* de eerste r rijen van V^* . Bijgevolg is

$$\|r\|_2^2 = \left\| \begin{bmatrix} \Sigma_r(V_r^* x) \\ 0 \end{bmatrix} - \begin{bmatrix} U_r^* b \\ U_2^* b \end{bmatrix} \right\|_2^2 = \|\Sigma_r(V_r^* x) - U_r^* b\|_2^2 + \|U_2^* b\|_2^2.$$

Daar Σ_r inverteerbaar is, wordt $\|r\|_2^2$ geminimaliseerd door de unieke oplossing van $\Sigma_r(V_r^* x) = U_r^* b$. De minimale waarde van $\|r\|_2$ is gelijk aan $\|U_2^* b\|_2$. Het stelsel $\Sigma(V_r^* x) = U_r^* b$ lossen we op door eerst het triviale stelsel $\Sigma_r y = U_r^* b$ op te lossen (met y onbekend) en dan $x = V_r y$ (daar we willen dat $V_r^* x = y$).

Samengevat, indien $A = U_r \Sigma_r V_r^*$ een gereduceerde SVD voorstelt van $A \in \mathbb{C}^{m,n}$ met rang r , dan zal de unieke oplossing van $\Sigma_r(V_r^* x) = U_r^* b$ resulteren in een minimum voor $\|Ax - b\|_2$. Dit resulteert in het algoritme in Figuur 3.10. Indien we niet zozeer wensen te weten welke x overeenstemt met een zo klein mogelijke $\|Ax - b\|_2$, maar op zoek zijn de kleinst mogelijke waarde voor $\|Ax - b\|_2$, dan moeten we enkel $\|U_2^* b\|_2$ uitrekenen, waarbij $U = [U_r \ U_2] \in \mathbb{C}^{m,m}$ de U matrix voorstelt van de volledige SVD.

Analoog aan Sectie 2.5.2 kunnen we de kleinst mogelijke waarde voor $\|Ax - b\|_2$ eveneens uitdrukken met behulp van U_r door op te merken dat

$x = V_r \Sigma_r^{-1} U_r^* b$ deze norm minimaliseert. Daar $V_r^* V_r = I$ (terwijl $U_r U_r^*$ doorgaans verschilt van I), resulteert dit in

$$\|r\|_2^2 = \|Ax - b\|_2^2 = \|U_r \Sigma_r V_r^* (V_r \Sigma_r^{-1} U_r^* b) - b\|_2^2 = \|U_r U_r^* b - b\|_2^2,$$

en deze uitdrukking voor $\|r\|_2^2$ is sneller uit te rekenen dan $\|U_2^* b\|_2^2$ indien $m \gg n$.

Wanneer we de algoritmes in Figuur 2.8 en 3.10 met elkaar vergelijken, zien we dat de tijdscomplexiteit voornamelijk wordt bepaald door respectievelijk de QR factorisatie en de SVD van A . De QR factorisatie op basis van de Householder reflecties heeft een complexiteit van $2mn^2 - 2n^3/3$ (zie Sectie 2.4.2). De complexiteit van de SVD is doorgaans wat moeilijker uit te drukken, maar wordt vaak geschat op $2mn^2 + 11n^3$. Kortom, tenzij $m \gg n$, zien we dat de QR factorisatie duidelijk sneller is. Echter, indien A niet veel verschilt van een matrix met een rang strict kleiner dan n , dan is het algoritme op basis van de QR factorisatie vaak minder nauwkeurig dan het SVD algoritme.

3.4.3 Herkennen van handgeschreven cijfers

In deze sectie geven we aan hoe we handgeschreven cijfers snel machinaal kunnen herkennen door gebruik te maken van een SVD. We starten met ons systeem te trainen op basis van enkele duizende handgeschreven cijfers $0, 1, \dots, 9$. Elk van deze handgeschreven cijfers zetten we om in een 16×16 bitmap, waarbij we elke pixel weergeven door een grijswaarde tussen 0 en 255. In Figuur 3.11 zien we de bitmap van een vijftal handgeschreven cijfers. Door de pixelwaardes als vector weer te geven, zien we dat elk handgeschreven cijfer wordt weergegeven door een vector in $\mathbb{R}^{256}$. Vervolgens plaatsen we alle vectoren die overeenstemmen met het handgeschreven cijfer i in een matrix D_i , voor $i = 0, \dots, 9$. Kortom, we stellen 10 matrices op die elk 256 rijen hebben en waarbij het aantal kolommen gelijk is aan het aantal cijfers gelijk aan i in de trainingsset (bv. een paar duizend).

Wanneer we nu een bepaald handgeschreven cijfer (dat niet tot de trainingsset behoort) machinaal willen herkennen, dan zullen we dit cijfer eveneens opslaan als 16×16 bitmap en omzetten naar een vector d in $\mathbb{R}^{256}$. Vervolgens gaan we voor $i \in \{0, \dots, 9\}$ de afstand ρ_i bepalen tussen de vector d en de ruimte opgespannen door de kolommen van D_i . De afstand tussen een punt $d \in \mathbb{R}^{256}$ en een deelruimte van $\mathbb{R}^{256}$, stemt overeen met de kleinst mogelijke

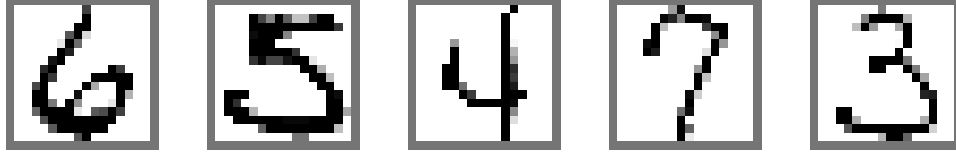


Figure 3.11: De bitmap van 5 handgeschreven cijfers.

afstand tussen d en een punt x binnen de deelruimte. Kortom, we kunnen de ρ_i -waarden bepalen door volgende 10 kleinste kwadraat problemen op te lossen:

$$\rho_i = \min_x \|D_i x - d\|_2.$$

Merk op, indien d geschreven kan worden als lineaire combinatie van de kolommen van D_i dan is ρ_i gelijk aan nul en maakt d deel uit van de deelruimte opgespannen door de kolommen van D_i .

In Sectie 3.4.2 zagen we dat we de minimale waarde voor $\|Ax - b\|_2^2$ konden bepalen via de SVD van A door middel van de uitdrukking

$$\|U_r U_r^* b - b\|_2^2.$$

Merk op, in Sectie 3.4.2 stelden we dat $A \in \mathbb{C}^{m,n}$ met $m > n$, maar dit resultaat gaat evenzeer op voor $m \leq n$. Met andere woorden indien $D_i = U_{i,r_i} \Sigma_{i,r_i} V_{i,r_i}^*$ een gereduceerde SVD van D_i is, waarbij $r_i = \text{rang}(D_i) \leq 256$, dan is

$$\rho_i = \|U_{i,r_i} U_{i,r_i}^* d - d\|_2.$$

We zullen de vector d catalogeren als het cijfer j indien ρ_i minimaal is voor $i = j$.

Wanneer we bovenstaande methode zonder meer zouden toepassen, dan treden er problemen op. De deelruimte die met een bepaald cijfer overeenstemt zal namelijk steeds groter worden naarmate we onze trainingsset vergroten. In het extreme geval zou de deelruimte zelfs kunnen samenvallen met $\mathbb{R}^{256}$ waardoor de afstand tot deze deelruimte steeds nul is. Verder zullen niet alle deelruimtes even groot zijn, d.i., dezelfde dimensie r_i hebben (wat niet noodzakelijk slecht hoeft te zijn). Tot slot zal ook de berekening van de afstanden ρ_i vertragen naarmate de dimensie r_i toeneemt.

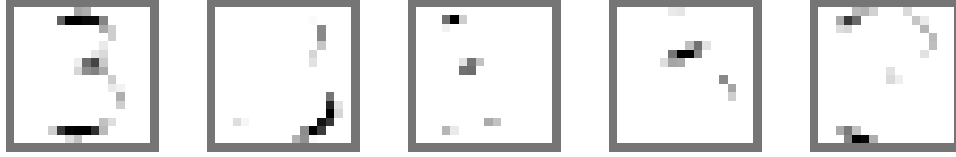


Figure 3.12: De bitmap van de eerste 5 kolommen van U_{3,r_3} .

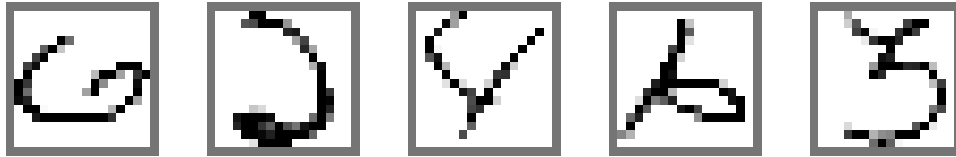


Figure 3.13: De bitmap van 5 foutief herkende cijfers.

Om deze problemen te omzeilen gaan we voor elk van de 10 cijfers gebruik maken van een deelruimte met dimensie k (bijvoorbeeld $k = 10$). Gezien onze eerdere discussies rond de geometrische interpretatie van de SVD, mag het duidelijk zijn dat we de k assen van deze deelruimte gelijk zullen nemen aan de k langste assen van de hyperellips en deze vallen natuurlijk precies samen met de eerste k kolommen van U_{i,r_i} . Kortom, we bepalen de afstanden ρ_i , voor $i = 0, \dots, 9$, als

$$\rho_i = \|U_{i,k}(U_{i,k}^*d) - d\|_2,$$

waarbij $U_{i,k}$ de eerste k kolommen van U_{i,r_i} bevat. Hierdoor verdwijnen de eerder vernoemde problemen en verloopt de berekening ook veel sneller.

In Tabel 3.1 zien we de resultaten voor een systeem waarbij de trainingsset bestond uit 1707 cijfers (die we ook in de oefeningen zullen hanteren) en de testset uit 2007 cijfers, terwijl $k = 10$. We zien dat de resultaten doorgaans vrij accuraat zijn en dat niet alle cijfers even makkelijk herkenbaar zijn. In Figuur 3.13 zien we enkele cijfers die niet correct werden herkend, het is duidelijk dat het hier gaat om zeer slecht geschreven cijfers. Het zou gaan om een 6, 2, 4, 6 en 3, terwijl het SVD algoritme deze catalogeerde als 0, 0, 7, 2 en 5.

Cijfer	Aantal	Juist	Fout	Succeskans
0	359	353	6	98.3287
1	264	261	3	98.8636
2	198	179	19	90.4040
3	166	146	20	87.9518
4	200	183	17	91.5000
5	160	139	21	86.8750
6	170	160	10	94.1176
7	147	138	9	93.8776
8	166	145	21	87.3494
9	177	166	11	93.7853

Table 3.1: Resultaten voor het herkennen van handgeschreven cijfers op basis van een SVD met $k = 10$.

3.5 Berekenen van de SVD

Ervan uitgaande dat we over numerieke methodes beschikken om de eigenwaarden van een Hermitische matrix nauwkeurig berekenen, is het op het eerste zicht erg verleidelijk om Eigenschap 3.3.5 te gebruiken voor de berekening van de singuliere waarden van een matrix A , daar A^*A Hermitisch is. Deze aanpak kan echter leiden tot numerieke problemen. Neem bijvoorbeeld de matrix A met

$$A = \begin{bmatrix} 1 & 1 \\ \mu & 0 \\ 0 & \mu \end{bmatrix}.$$

Als μ echter klein is zodat $1 + \mu^2$ numeriek gelijk is aan 1 (terwijl $1 + \mu$ numeriek verschilt van 1), dan zal de matrix

$$A^*A = \begin{bmatrix} 1 + \mu^2 & 1 \\ 1 & 1 + \mu^2 \end{bmatrix},$$

met eigenwaarden $2 + \mu^2$ en μ^2 , terwijl A^*A numeriek gelijk is aan de matrix

$$\mathbb{1}(A^*A) = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix},$$

met eigenwaarden 2 en 0. Kortom, de exacte singuliere waarden zijn $\sigma_1 = \sqrt{2 + \mu^2}$ en $\sigma_2 = |\mu|$, terwijl we numeriek besluiten dat $\sigma_1 = \sqrt{2}$ en $\sigma_2 = 0$, wat de fout op σ_2 zeer groot maakt.

Een numeriek betrouwbare methode voor de berekening van de SVD kunnen we bekomen via de eigenwaarden decompositie van

$$H = \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix},$$

zie oefeningen. De berekeningsmethode die we in deze sectie bespreken is eigenlijk op dit idee gebaseerd, maar is sneller en vermijdt ook dat we een matrix van grootte $m + n$ moeten opslaan in het geheugen. Deze procedure bestaat uit twee fases. Tijdens de eerste fase zoeken we unitaire matrices U_1 en V_1 zodat

$$B = U_1^* A V_1$$

een boven-bidiagonaalmatrix is. Een boven-bidiagonaalmatrix heeft enkel elementen verschillend van nul op rij i en kolom j indien j gelijk is aan i of $i + 1$. Tijdens fase 2 wordt er een iteratieve methode gebruikt om de unitaire matrices U_2 en V_2 te bepalen zodat

$$\Sigma = U_2^* B V_2$$

een niet-negatieve diagonaalmatrix is. Tot slot, stellen we de matrix $U = U_1 U_2$ en $V = V_1 V_2$ zodat

$$U^* A V = U_2^* (U_1^* A V_1) V_2 = U_2^* B V_2 = \Sigma,$$

waaruit we mogen besluiten dat $A = U \Sigma V^*$ een SVD is van A .

We besluiten dit hoofdstuk door aan te geven hoe we de eerste fase van dit algoritme kunnen uitvoeren met behulp van Householder reflectie matrices. Dit algoritme staat bekend als de Golub-Kahan bidiagonalisatie (1965) en is verwant aan de Hessenberg decompositie die we reeds bespraken in Sectie 2.5.3. Bij de berekening van de SVD zal de eerste fase doorgaans ook het meeste tijd in beslag nemen.

3.5.1 Golub-Kahan bidiagonalisatie

In deze sectie geven we aan hoe we voor een willekeurige matrix $A \in \mathbb{C}^{m,n}$, met $m \geq n$, twee unitaire matrices U en V kunnen vinden zodat $B = U^* A V$

een boven-bidiagonaalmatrix is. We maken hiervoor gebruik van de Householder reflectie matrices uit Sectie 2.4.2 en gaan analoog te werk als in Sectie 2.5.3 voor de Hessenberg decompositie. Er is echter één belangrijk verschil, namelijk voor de Hessenberg decompositie zoeken we een unitaire matrix Q zodat $H = Q^* A Q$ een boven-Hessenberg matrix is, terwijl we bij de bidiagonalisatie twee verschillende matrices U en V mogen gebruiken.

In Sectie 2.4.2 zagen we dat Px een veelvoud werd van e_1 door de vector v van de Householder reflectie matrix gelijk te stellen aan $x + e^{i \arg x_1} \|x\|_2 e_1$. Laat x^* een willekeurige rij vector zijn en stel v wederom gelijk aan $x + e^{i \arg x_1} \|x\|_2 e_1$, (waarbij x een kolom vector is gelijk aan $(x^*)^*$). Dan geldt dat Px een veelvoud is van e_1 , waardoor $(Px)^* = x^* P^* = x^* P$ een veelvoud is van e_1^* (want P is Hermitisch). Kortom, we kunnen door A rechts te vermenigvuldigen met de juiste Householder reflectie matrix P eveneens een bepaalde rij een veelvoud maken van e_1^* .

De bidiagonalisatie gaat nu als volgt te werk. Stel eerst $v_1 = a_1$, en laat P_1 de Householder reflectie matrix zijn met $v = v_1 + e^{i \arg(v_1)_1} \|v_1\|_2 e_1$, dan is de eerste kolom van $P_1 A$ een veelvoud van e_1 :

$$A_1 = P_1 A = \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix}.$$

Laat $w_1^* = (A_1)_{1,2:n}$ en definieer $\hat{Q}_1$ als de Householder reflectie matrix met $v = w_1 + e^{i \arg(w_1)_1} \|w_1\|_2 e_1$, dan is

$$C_1 = A_1 Q_1 = A_1 \begin{bmatrix} 1 & 0 \\ 0 & \hat{Q}_1 \end{bmatrix} = \begin{bmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & \times & \times & \times \end{bmatrix},$$

daar de eerste kolom van A_1 niet verandert wanneer we rechts vermenigvuldigen met Q_1 . Als we nu $v_2 = (C_1)_{2:m,2}$ en $\hat{P}_2$ is de Householder reflectie matrix

met $v = v_2 + e^{i \arg(v_2)_1} \|v_2\|_2 e_1$, dan resulteert dit in

$$A_2 = P_2 C_1 = \begin{bmatrix} 1 & 0 \\ 0 & \hat{P}_2 \end{bmatrix} C_1 = \begin{bmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix},$$

daar C_1 links vermenigvuldigen met P_2 de eerste rij ongewijzigd laat en eveneens de nullen op de eerste kolom bewaart. Wanneer we nu $w_2^* = (A_2)_{2,3:n}$ stellen en $\hat{Q}_2$ gelijk aan de Householder reflectie matrix met $v = w_2 + e^{i \arg(w_2)_1} \|w_2\|_2 e_1$, dan is

$$C_2 = A_2 Q_2 = A_2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \hat{Q}_2 \end{bmatrix} = \begin{bmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & 0 \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & \times & \times \end{bmatrix}.$$

Wanneer we deze procedure herhalen, dan zien we dat

$$A_{n-1} = P_{n-1} \dots P_2 P_1 A Q_1 Q_2 \dots Q_{n-2} = \begin{bmatrix} \times & \times & 0 & 0 \\ 0 & \times & \times & 0 \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \\ 0 & 0 & 0 & \times \end{bmatrix}.$$

Indien $m = n$, dan is A_{n-1} een bidiagonaalmatrix, anders volstaat het de laatste $m - n$ elementen van de n -de kolom van A_{n-1} eveneens nul te maken met behulp van de Householder reflectie matrix $\hat{P}_n$ waarvoor $v = v_n + e^{i \arg(v_n)_1} \|v_n\|_2 e_1$ met $v_n = (A_{n-1})_{n:m,n}$. De gevraagde unitaire matrices U^* en V zijn bijgevolg gelijk aan:

$$U^* = P_n \dots P_1, \quad \text{en} \quad V = Q_1 \dots Q_{n-2}.$$

De tijdscomplexiteit van deze procedure is ongeveer $4mn^2 - 4n^3/3$, daar we min of meer twee keer zoveel werk hebben als bij een QR factorisatie.

3.6 Oefeningen

OEFENING 3.6.1. Singuliere waarden ontbinding



Figure 3.14: Data compressie van een 300×300 bitmap met behulp van de SVD. Van links naar rechts: rang $k = 10$ approximatie, rang $k = 30$ approximatie en de originele foto.

1. Bepaal de SVD van volgende matrices (met de hand):

$$(a) \begin{bmatrix} 3 & 0 \\ 0 & -1 \end{bmatrix}, \quad (b) \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad (c) \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad (d) \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad (e) \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

2. Twee matrices $A, B \in \mathbb{C}^{m,m}$ noemen we **unitair equivalent** indien er een unitaire matrix Q bestaat, zodat $A = QBQ^*$. Toon aan dat A en B dezelfde singuliere waarden hebben indien ze unitair equivalent zijn. Geef aan dat het omgekeerde niet geldt aan de hand van een tegenvoorbeeld (neem $A, B \in \mathbb{R}^{2,2}$ zodat A en B diagonaalmatrices zijn).
3. Toon aan dat elke matrix $A \in \mathbb{C}^{m,n}$ als limiet kan geschreven worden van een reeks van matrices A_k met rang van A_k gelijk aan $\min(m, n)$. Gebruik de 2-norm in je bewijs.
4. Gebruik de Eigenschappen uit Sectie 3.3 om de 2-norm van de matrix

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix},$$

uit Figuur 1.1 met de hand te bepalen.

5. Laat $A \in \mathbb{C}^{m,m}$ met SVD $A = U\Sigma V^*$. Geef een eigenwaarden ontbinding voor de Hermitische matrix

$$H = \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix},$$

met $H \in \mathbb{C}^{2m,2m}$. Kortom, zoek een unitaire matrix $Q \in \mathbb{C}^{2m,2m}$, zodat $H = Q\Delta Q^*$, waarbij Δ een diagonaalmatrix is (die de eigenwaarden van H bevat). Bereken volgende producten om de eigenwaarde ontbinding te vinden:

$$H \begin{bmatrix} v_i \\ u_i \end{bmatrix} \quad \text{en} \quad H \begin{bmatrix} v_i \\ -u_i \end{bmatrix}.$$

OEFENING 3.6.2. SVD toepassingen

1. Data compressie is een vaak aangehaalde toepassing van de SVD. Wanneer een foto bestaat uit $m \times n$ pixels, dan kunnen we deze voorstellen als matrix $A \in \mathbb{R}^{m,n}$ (of als 3 zulke matrices voor een kleurenfoto in RGB). Door A te benaderen met behulp van een rang $k < \min(m, n)$ matrix kunnen we het aantal bytes vereist om deze op te slaan reduceren (ten koste van de kwaliteit van de foto). In Figuur 3.14 zien we hiervan een voorbeeld. Schrijf zelf een Matlab functie die toelaat afbeeldingen te comprimeren en experimenteer met de waarde van k . Maak gebruik van de Matlab functies `imread('foto.jpg')`, `rgb2gray`, `im2double`, `colormap(gray(256))`, etc.
2. In de cursus toonden we in Tabel 3.1 de resultaten van het SVD algoritme voor het herkennen van handgeschreven cijfers, wanneer de dimensie van de deelruimtes k gelijk was aan 10. In deze oefening is het de bedoeling deze resultaten te reproduceren en vervolgens na te gaan wat de invloed van k is op de nauwkeurigheid van de resultaten. Stel dat je eveneens in sommige gevallen mag antwoorden dat het resultaat **onbepaald** is. Hoe kan je dan de nauwkeurigheid verbeteren zonder al te vaak **onbepaald** te antwoorden (onderbouw je antwoord met experimenten).

OEFENING 3.6.3. Golub-Kahan bidiagonalisatie

1. Implementeer een Matlab functie `[W,B]=bidiag(A)` die de Golub-Kahan bidiagonalisatie van A berekent. W bevat zowel de vectoren $v_1, \dots, v_n$ als de vectoren $w_1^*, \dots, w_{n-2}^*$ zodat $W_{k:m,k} = v_k$ en $W_{s,s+1:n} = w_s^*$ voor $k = 1, \dots, n$ en $s = 1 : \dots, n - 2$. Zorg dat je code ook werkt voor complexe matrices A .

2. Schrijf een Matlab functie `formUV_bidiag(W)` die de W matrix uit de voorgaande opgave omzet in de unitaire matrices U en V zodat $B = U^*AV$, of nog, $A = UB V^*$.

CONDITIONERING EN STABILITEIT

Wanneer we met behulp van een algoritme trachten een oplossing te berekenen voor een opgegeven probleem dan kunnen hierbij numerieke problemen optreden. Deze problemen kunnen enerzijds aan het probleem zelf en anderzijds aan het gehanteerde algoritme toe te schrijven zijn.

Laten we ons probleem noteren als een afbeelding f van $\mathbb{C}^n$ naar $\mathbb{C}^m$, waarbij $f(x)$ het beeld van x weergeeft. Laat verder ons algoritme overeenstemmen met de afbeelding $\tilde{f}$ (wat eveneens een afbeelding is van $\mathbb{C}^n$ naar $\mathbb{C}^m$), met $\tilde{f}(x)$ het beeld van x . Ten eerste moeten we vaststellen dat we doorgaans niet $f(x)$ gaan uitrekenen, maar in het beste geval $f(\text{fl}(x))$, waarbij $\text{fl}(x)$ het *dichtsbijgelegen* floating point getal is. Kortom, indien $f(x)$ en $f(\text{fl}(x))$ al sterk van elkaar verschillen dan is er weinig hoop dat $\tilde{f}(x) = \tilde{f}(\text{fl}(x))$ een goede benadering zal vormen voor $f(x)$, ongeacht het algoritme $\tilde{f}$ dat we hanteren. Kortom, wanneer een kleine verandering/perturbatie in x aanleiding kan geven tot een grote verandering in $f(x)$ dan zal er vaak sprake zijn van een grote fout. In dit geval spreken we van een slecht geconditioneerd probleem. In Sectie 4.1 zullen we het conditiegetal $\kappa(x)$ invoeren dat zal aangeven hoe slecht of goed een probleem geconditioneerd is.

Zelfs wanneer het probleem goed geconditioneerd is, dan kan $\tilde{f}(\text{fl}(x))$ nog sterk verschillen van $f(x)$, daar $\tilde{f}$ afrondingsfouten introduceert tijdens de berekening. Wanneer deze fouten samen geen al te grote gevolgen hebben op ons eindresultaat dan noemen we een algoritme $\tilde{f}$ stabiel. Een precieze definitie van stabiliteit (en achterwaartse stabiliteit) wordt gegeven in Sectie 4.2.

In dit hoofdstuk zullen we ook vaak de notatie δx en δA hanteren terwijl er eveneens sprake is van een vector $x \in \mathbb{C}^m$ en een matrix $A \in \mathbb{C}^{m,n}$. De notatie δx geeft in dit geval eveneens een vector uit $\mathbb{C}^m$ weer en deze hoeft niet gerelateerd te zijn met x , analoog is $\delta A \in \mathbb{C}^{m,n}$ eveneens een willekeurige matrix van dezelfde grootte als A . Het gaat in beide gevallen dus *niet* om een product met $\delta \in \mathbb{C}$.

4.1 Conditiegetallen

4.1.1 Definitie en voorbeelden

DEFINITIE 4.1.1. Wanneer f een probleem weergeeft (van $\mathbb{C}^n$ naar $\mathbb{C}^m$), dan zullen we het **absolute conditiegetal** $\hat{\kappa}(x)$ in het punt $x \in \mathbb{C}^n$ definiëren als

$$\hat{\kappa}(x) = \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|f(x + \delta x) - f(x)\|}{\|\delta x\|},$$

waarbij de norm in de teller (noemer) overeenstemt met de gekozen norm in $\mathbb{C}^m$ ($\mathbb{C}^n$).

Merk op, indien f een afbeelding is van $\mathbb{R}$ naar zichzelf en f is afleidbaar in x , dan is $\hat{\kappa}(x)$ gelijk aan $|f'(x)|$. Verder zullen we doorgaans dezelfde vectornorm hanteren in $\mathbb{C}^m$ en $\mathbb{C}^n$.

Wanneer f een afbeelding is van $\mathbb{R}^n$ naar $\mathbb{R}^m$ dan bestaan er functies $f_1, \dots, f_m$ van $\mathbb{R}^n$ naar $\mathbb{R}$ zodat

$$f(x_1, \dots, x_n) = (f_1(x_1, \dots, x_n), \dots, f_m(x_1, \dots, x_n)).$$

Wanneer alle partiële afgeleides $\partial f_i(x)/\partial x_j = \partial f_i(x_1, \dots, x_m)/\partial x_j$ bestaan in $x = (x_1, \dots, x_n)$ dan noemen we f differentieerbaar in x en definiëren we de **Jacobiaan** $J(x)$ als de volgende $m \times n$ matrix

$$J(x) = \begin{bmatrix} \partial f_1(x)/\partial x_1 & \partial f_1(x)/\partial x_2 & \dots & \partial f_1(x)/\partial x_n \\ \partial f_2(x)/\partial x_1 & \partial f_2(x)/\partial x_2 & \dots & \partial f_2(x)/\partial x_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial f_m(x)/\partial x_1 & \partial f_m(x)/\partial x_2 & \dots & \partial f_m(x)/\partial x_n \end{bmatrix}.$$

Het belang van de Jacobiaan met betrekking tot de definitie van $\hat{\kappa}(x)$ heeft te maken met het feit dat we $f(x + \delta x)$ in dit geval kunnen schrijven als

$$f(x + \delta x) = f(x) + J(x)\delta x + o(\delta x),$$

waarbij de $o(\delta x)$ aangeeft dat het om termen gaat die sneller naar nul gaan dan $\|\delta x\|$. Dit houdt in dat

$$\lim_{\|\delta x\| \rightarrow 0} \frac{o(\delta x)}{\|\delta x\|} = 0.$$

Wanneer we de uitdrukking voor $f(x + \delta x)$ in onze definitie van $\hat{\kappa}(x)$ stoppen dan zien we dat

$$\hat{\kappa}(x) = \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|J(x)\delta x + o(\delta x)\|}{\|\delta x\|} = \|J(x)\|.$$

Merk op, de norm van de Jacobiaan $J(x)$ die hierboven verschijnt is de matrixnorm geïnduceerd door de vectornorm die we gebruikte in $\mathbb{C}^n$ en $\mathbb{C}^m$.

In het bijzondere geval dat f een lineaire afbeelding is, d.i., $f(x) = Ax$ met A een $m \times n$ matrix, zien we dat

$$f_i(x_1, \dots, x_n) = a_{i1}x_1 + \dots + a_{in}x_n,$$

wat aangeeft dat $J(x) = A$. Bijgevolg is

$$f(x + \delta x) = A(x + \delta x) = Ax + A\delta x = f(x) + A\delta x = f(x) + J(x)\delta x.$$

Kortom, de termen $o(\delta x)$ zijn gelijk aan nul en de matrixnorm van de Jacobiaan is eenvoudigweg de matrixnorm van A .

DEFINITIE 4.1.2. Wanneer f een probleem weergeeft (van $\mathbb{C}^n$ naar $\mathbb{C}^m$), dan zullen we het **relatieve conditiegetal** $\kappa(x)$ in het punt $x \in \mathbb{C}^n$ definiëren als $\kappa(x) = \hat{\kappa}(x)/(\|f(x)\|/\|x\|)$.

Het relatieve conditiegetal $\kappa(x)$ zal een bepalende rol spelen in de accurateheid van onze berekeningen. We zullen later zien dat het verlies aan nauwkeurigheid, d.i., het aantal correcte digits dat we verliezen, min of meer wordt bepaald door $\log_{10} \kappa(x)$. Kortom, een conditiegetal $\kappa(x) = 1, 2$ of zelfs 10 is prima, maar conditiegetallen in de grootte orde van 10^6 of 10^{10} zullen steeds aanleiding geven tot een groot verlies aan nauwkeurigheid.

VOORBEELD 4.1.1. Stel dat we de vierkantswortel van $x \in \mathbb{R}$ wensen te berekenen. Dan is $f(x) = x^{1/2}$, wat maakt dat $\hat{\kappa}(x) = |J(x)| = |f'(x)| = |x^{-1/2}/2|$. Het relatieve conditiegetal $\kappa(x)$ is gelijk aan $\hat{\kappa}(x)$ gedeeld door $|f(x)|/|x| = |x^{1/2}/x| = |x^{-1/2}|$. Hieruit volgt $\kappa(x) = 1/2$ waaruit we besluiten dat het nemen van de vierkantswortel een goed geconditioneerd probleem is.

VOORBEELD 4.1.2. Stel dat we het verschil van twee getallen wensen te berekenen. Dan is $f(x_1, x_2) = x_1 - x_2$. Dit is een lineaire afbeelding met

$A = J(x) = [1 \ -1]$. Indien we de ∞ -norm hanteren op $\mathbb{R}^2$, dan zien we dat $\|J(x)\|_\infty = 2$ omwille van Eigenschap 1.2.2. Bijgevolg is

$$\kappa(x) = \frac{2}{|x_1 - x_2| / \max(|x_1|, |x_2|)}.$$

Wanneer $x_1 - x_2 \approx 0$, dan is het probleem duidelijk slecht geconditioneerd.

VOORBEELD 4.1.3. Stel dat we de twee oplossingen voor y van de kwadratische vergelijking $y^2 - 2y + x = 0$ willen bepalen voor een zeker getal x . Deze oplossing zijn natuurlijk gelijk aan $1 \pm \sqrt{1-x}$. Laat $f_1(x) = 1 - \sqrt{1-x}$ en $f_2(x) = 1 + \sqrt{1-x}$.

Dan is de Jacobiaan $J(x) = (1/(2\sqrt{1-x}), -1/(2\sqrt{1-x}))^*$. Wanneer we x naar 1 laten gaan, zien we dat $\|J(x)\|_1 = 1/\sqrt{1-x}$ oneindig wordt, terwijl dat $\|f(x)\|_1 / \|x\|_1$ naar 2 gaat. Dit maakt dat het relatieve conditiegetal $\kappa(x) = \|J(x)\| \|x\| / \|f(x)\|$ oneindig groot wordt, waaruit we mogen besluiten dat het bepalen van de twee wortels een slecht geconditioneerd probleem is (voor x dicht bij 1). De functie $f(x)$ is in dit geval niet differentieerbaar in $x = 1$ (omdat de polynoom dan een dubbele wortel heeft).

In het algemeen is het bepalen van de wortels van een polynoom vaak een slecht geconditioneerd probleem, zelfs wanneer de wortels ver uit elkaar gelegen zijn. Kortom, kleine wijzigingen in één of meer coëfficiënten van een polynoom kan de wortels sterk doen veranderen. Een klassiek voorbeeld hiervan is de Wilkinson polynoom $p(y) = \prod_{i=1}^{20} (y - i)$.

VOORBEELD 4.1.4. Een ander (nog extremer) voorbeeld zijn de eigenwaarden van een matrix A in functie van zijn elementen a_{ij} . Vergelijk bijvoorbeeld de matrices

$$A_1 = \begin{bmatrix} 1 & 1000 \\ 0 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 1000 \\ 0.001 & 1 \end{bmatrix},$$

die slechts verschillen met 0.001 in een enkele entry. De eigenwaarden van A_1 zijn duidelijk gelijk aan 1 en 1, terwijl 0 en 2 de eigenwaarden van A_2 zijn. Het bepalen van de eigenwaarden van een symmetrische matrix is echter wel een goed geconditioneerd probleem. Zo kan men bewijzen dat indien A met eigenwaarde λ , vervangen wordt door $A + \delta A$, waarvan we de overeenkomstige eigenwaarde noteren als $\lambda + \delta\lambda$, dan zal $|\delta\lambda| \leq \|\delta A\|_2$.

4.1.2 Conditiegetal van een matrix

In deze sectie bekijken we het conditiegetal $\kappa(x)$ voor de functie $f(x)$ van $\mathbb{C}^n$ naar $\mathbb{C}^m$ met $f(x) = Ax$, waar $A \in \mathbb{C}^{m,n}$. Kortom, we gaan na of de matrix-vector vermenigvuldiging een goed geconditioneerd probleem is wanneer we de elementen van de vector x perturberen. Via de definitie van $\hat{\kappa}(x)$ en het feit dat $A(x + \delta x) - Ax = A\delta x$, zien we dat

$$\hat{\kappa}(x) = \lim_{\delta \rightarrow 0} \sup_{\|\delta x\| \leq \delta} \frac{\|A\delta x\|}{\|\delta x\|}.$$

Verder is $\|A\delta x\| / \|\delta x\| = \|A(\delta x / \|\delta x\|)\|$, waardoor we zien dat $\hat{\kappa}(x) = \|A\|$. Merk op, voor de reële matrix-vector vermenigvuldiging zien we dit ook onmiddellijk uit de gelijkheid $\hat{\kappa}(x) = \|J(x)\| = \|A\|$. Bijgevolg is

$$\kappa(x) = \hat{\kappa}(x) / (\|f(x)\| / \|x\|) = \|A\| \|x\| / \|Ax\|.$$

Indien A een inverteerbare matrix is, dan is $x = A^{-1}(Ax)$, met als gevolg dat

$$\|x\| = \|A^{-1}(Ax)\| \leq \|A^{-1}\| \|Ax\|,$$

onwille van Eigenschap 1.2.4. Kortom, $\|x\| / \|Ax\| \leq \|A^{-1}\|$, waaruit we mogen besluiten dat

$$\kappa(x) \leq \|A\| \|A^{-1}\|,$$

voor een inverteerbare matrix $A \in \mathbb{C}^{m,m}$. Wanneer we de 2-norm hanteren is $\|A\|_2 = \sigma_1$, de grootste singuliere waarde van A (zie Eigenschap 3.3.3). Daar de singuliere waarden van A^{-1} de inverse zijn van die van A is verder $\|A^{-1}\|_2 = 1/\sigma_m$, met σ_m de kleinste singuliere waarde van A . Dat $\kappa(x)$ voor sommige x waarden gelijk is aan $\|A\| \|A^{-1}\| = \sigma_1/\sigma_m$ zien we door $x = v_m$ met $Av_m = \sigma_m u_m$ en $\|u_m\|_2 = \|v_m\|_2 = 1$, waardoor

$$\|x\|_2 / \|Ax\|_2 = \|v_m\|_2 / \|\sigma_m u_m\|_2 = 1/\sigma_m = \|A^{-1}\|_2.$$

In de bovenstaande discussie keken we naar de mate waarin $b = Ax$ wijzigt wanneer we de vector x vervangen door $x + \delta x$. We kunnen echter ook de vraag stellen in welke mate de oplossing van het stelsel $Ax = b$ verandert indien we b vervangen door $b + \delta b$. Daar $x = A^{-1}b$, hebben we deze vraag eigenlijk ook al beantwoord. Dit alles resulteert in volgende eigenschap:

EIGENSCHAP 4.1.1. *Laat $A \in \mathbb{C}^{m,m}$ een inverteerbare matrix zijn en beschouw het stelsel $Ax = b$. Het probleem om b te bepalen in functie van x heeft als conditiegetal*

$$\kappa(x) = \|A\| \|x\| / \|Ax\| \leq \|A\| \|A^{-1}\|,$$

terwijl het probleem om x te bepalen als functie van b heeft als conditiegetal

$$\kappa(b) = \|A^{-1}\| \|b\| / \|A^{-1}b\| \leq \|A\| \|A^{-1}\|.$$

Voor de 2-norm is $\kappa(\cdot) = \|A\| \|A^{-1}\| = \sigma_1/\sigma_m$ indien $x = v_m$ met $Av_m = \sigma_m u_m$ voor het eerste probleem en indien $b = u_1$ met $Av_1 = \sigma_1 u_1$ voor het tweede.

De voorgaande eigenschap geeft aanleiding tot de volgende definitie:

DEFINITIE 4.1.3. *Het **conditiegetal** $\kappa(A)$ van een inverteerbare matrix $A \in \mathbb{C}^{m,m}$ wordt gegeven door $\|A\| \|A^{-1}\|$. Voor de 2-norm is $\kappa(A) = \sigma_1/\sigma_m$.*

Indien we deze definitie van het conditiegetal $\kappa(A)$ veralgemenen door te stellen dat $\kappa(A) = \sigma_1/\sigma_n$ voor een willekeurige $A \in \mathbb{C}^{m,n}$ met $m \geq n$, dan zal het conditiegetal van de twee problemen uit Eigenschap 4.1.1 eveneens begrensd blijven door $\kappa(A)$ (met gelijkheid in sommige gevallen). Merk op, voor een singuliere matrix $A \in \mathbb{C}^{m,m}$ is $\sigma_m = 0$, waardoor het conditiegetal oneindig is. Voor unitaire matrices Q hebben we dan weer dat $\|Q\| = \|Q^{-1}\| = 1$, bijgevolg is $\kappa(Q) = 1$. Daar de transformatie matrices uit de cursus Computer Graphics unitair zijn, zien we dat de transformatie naar het Eye-coördinaat systeem (en terug) een goed geconditioneerd probleem is.

We kunnen ons ook de vraag stellen in welke mate perturbaties in A een invloed hebben op de oplossing van het stelsel $Ax = b$. In dit geval is het probleem f een functie van $\mathbb{C}^{m,n}$ naar $\mathbb{C}^n$ die A afbeeldt op x zodat b constant blijft. Indien we A dus wijzigen in $A + \delta A$ dan zal de oplossing x veranderen in $x + \delta x$ zodat

$$(A + \delta A)(x + \delta x) = b.$$

Daar $Ax = b$ zien we dat $A(\delta x) + (\delta A)(x + \delta x) = 0$. Wanneer A inverteerbaar is, volgt hieruit dat

$$\|\delta x\| = \|-A^{-1}(\delta A)(x + \delta x)\| \leq \|A^{-1}\| \|\delta A\| \|x + \delta x\|.$$

Dit impliceert dat

$$\|A\| \|A^{-1}\| \geq \|A\| \|\delta x\| / (\|\delta A\| \|x + \delta x\|) = \frac{\|\delta x\| / \|\delta A\|}{\|x + \delta x\| / \|A\|}.$$

Merk op dat $x = f(A)$, $x + \delta x = f(A + \delta A)$ en dat $\delta x = f(A + \delta A) - f(A)$, waaruit volgt

$$\|A\| \|A^{-1}\| \geq \frac{\|f(A + \delta A) - f(A)\| / \|\delta A\|}{\|f(A + \delta A)\| / \|A\|}.$$

We concluderen dat het conditiegetal $\kappa \leq \|A\| \|A^{-1}\|$ (want $\|f(A + \delta A)\|$ gaat naar $\|f(A)\|$ wanneer $\|\delta A\|$ naar nul gaat). Merk op, we laten hier bewust de input parameter A weg bij het conditiegetal κ daar we $\kappa(A)$ reeds introduceerde voor het conditiegetal $\|A\| \|A^{-1}\|$ van de matrix A . We besluiten met volgende eigenschap:

EIGENSCHAP 4.1.2. *Laat $A \in \mathbb{C}^{m,m}$ een inverteerbare matrix zijn en beschouw het stelsel $Ax = b$. Het probleem om x te bepalen in functie van A (met b vast), heeft als conditiegetal*

$$\kappa \leq \|A\| \|A^{-1}\|.$$

Het is ook mogelijk om te bewijzen dat er perturbaties δA bestaan zodat $\kappa = \|A\| \|A^{-1}\|$.

4.2 Stabiliteit

Een algoritme $\tilde{f}$, dat dient om de oplossing van een probleem f numeriek te berekenen, zal doorgaans tal van afrondingsfouten introduceren. In principe zouden we graag hebben dat $f(x)$ en $\tilde{f}(x)$ dicht bij elkaar gelegen zijn of nog dat

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|}$$

klein is. Dit is echter een onrealistische verwachting, zelfs wanneer $\tilde{f}(x) = f(x)$ voor alle floating point getallen x , want $\tilde{f}(x)$ is gelijk aan $\tilde{f}(\text{fl}(x))$ en $f(\text{fl}(x))$ kan sterk verschillen van $f(x)$ indien het probleem $f(x)$ slecht geconditioneerd is.

Vandaar dat we voor stabiliteit gaan eisen dat

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|}$$

klein is, voor een $\tilde{x}$ die dicht bij x ligt. Kortom, we eisen dat *de berekende oplossing ongeveer correct is, voor ongeveer de juiste input*. Om deze definitie formeel te maken, introduceren we eerst de machineprecisie $\epsilon_{\text{machine}}$.

4.2.1 Floating point rekenen en de machineprecisie

Wanneer we een algoritme $\tilde{f}$ uitvoeren, zullen we geen gebruik maken van de mathematische operaties $+$, $-$, $\times$ en $\div$, maar van hun overeenkomstige floating point operaties, die we zullen noteren als $\oplus$, $\ominus$, $\otimes$ en $\odiv$.

DEFINITIE 4.2.1. *De **machineprecisie** $\epsilon_{\text{machine}}$ is het kleinste reële getal waarvoor geldt dat voor elke $x \in \mathbb{R}$ er een ϵ bestaat met $|\epsilon| \leq \epsilon_{\text{machine}}$ zodat*

$$\text{fl}(x) = x(1 + \epsilon) \quad (4.6)$$

en voor elke twee floating point getallen x, y , bestaat er eveneens een ϵ met $|\epsilon| \leq \epsilon_{\text{machine}}$ zodat

$$x \otimes y = (x * y)(1 + \epsilon), \quad (4.7)$$

waarbij $$ = $+$, $-$, $\times$ of $\div$.*

We eisen dus dat de relatieve afstand tussen een willekeurig gekozen reëel getal en het dichtsbijgelegen floating point getal kleiner is dan $\epsilon_{\text{machine}}$. Deze eis is natuurlijk enkel van toepassing voor getallen die gelegen zijn tussen het kleinste en het grootste floating point getal. Verder eisen we ook dat de floating point operaties exact zijn op een relatieve fout niet groter dan $\epsilon_{\text{machine}}$ na.

Wanneer onze machine de floating point getallen van het *IEEE 754 Double Floating Point* formaat ondersteunt, dan zal de machineprecisie $\epsilon_{\text{machine}}$ gelijk zijn aan 2^{-53} . In dat geval kan de voorwaarde in (4.7) zelfs vervangen worden door de sterkere voorwaarde die stelt dat

$$x \otimes y = \text{fl}(x * y),$$

voor elke twee floating point getallen x, y . Voor meer details rond floating point getallen en de IEEE formaten verwijzen we naar de cursus Computer-systemen en -architectuur.

4.2.2 Definitie

Nu we de machineprecisie $\epsilon_{\text{machine}}$ hebben ingevoerd, kunnen we een formele definitie van stabiliteit invoeren als volgt:

DEFINITIE 4.2.2. *Een algoritme $\tilde{f}$ voor een probleem f van $\mathbb{C}^n$ naar $\mathbb{C}^m$ is stabiel voor elke $x \in \mathbb{C}^n$ indien*

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = O(\epsilon_{\text{machine}}),$$

voor een bepaalde $\tilde{x}$ waarvoor geldt dat

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}).$$

Laten we deze definitie even van naderbij bekijken. Ten eerste merken we op dat er een constante $C_1 > 0$ en $\epsilon_1 > 0$ moet bestaan zodat voor alle $\epsilon_{\text{machine}} \leq \epsilon_1$ geldt

$$\|\tilde{f}(x) - f(\tilde{x})\| \leq C_1 \epsilon_{\text{machine}} \|f(\tilde{x})\|.$$

Analoog moet er een $C_2 > 0$ en $\epsilon_2 > 0$ bestaan zodat

$$\|\tilde{x} - x\| \leq C_2 \epsilon_{\text{machine}} \|x\|,$$

voor alle $\epsilon_{\text{machine}} \leq \epsilon_2$. De constanten C_1, C_2, ϵ_1 en ϵ_2 moeten tevens geldig zijn voor alle $x \in \mathbb{C}^n$. Merk ook op dat $\|x\| = 0$ impliceert dat $x = \tilde{x}$ voor voldoende kleine $\epsilon_{\text{machine}}$.

Kortom, wanneer we schrijven dat $\|g(x)\|$ gelijk is aan $O(\epsilon_{\text{machine}})$, voor een bepaalde functie g , dan beschouwen we eigenlijk een (oneindige) reeks van machines waarvan de machineprecisie $\epsilon_{\text{machine}}$ steeds kleiner wordt. De waarde van $\|g(x)\|$ zal dan minstens even snel als $\epsilon_{\text{machine}}$ naar nul dalen wanneer we deze zouden berekenen op deze reeks machines.

Deze definitie en met name het gebruik van de *big-O* notatie lijkt op het eerste zicht eigenaardig en misschien zelfs wat ongepast. De constante C_1 (en C_2) mag bijvoorbeeld erg groot zijn, we eisen enkel dat deze strict positief is. Mocht C_1 inderdaad gelijk zijn aan 2^{100} dan geeft de bovenstaande definitie geen enkele garantie voor een machine met bijvoorbeeld $\epsilon_{\text{machine}} = 2^{-53}$. Hoewel de waarde van C_1 (en C_2) dus zeker niet onbelangrijk is, zijn

er tal van redenen waarom men deze vaak toch niet bepaalt. Ten eerste zijn deze constanten afhankelijk van de gehanteerde norm en van de dimensies m en n (waarbij de constanten groter worden in functie van m en n). Daarnaast zouden we bij het bepalen van C_1 (en C_2) ook steeds rekening moeten houden met het slechtst denkbare scenario, waardoor deze bovengrenzen soms erg pessimistisch zijn en daardoor weinig inzicht verschaffen over het typisch gedrag van een algoritme $\tilde{f}$. In de praktijk blijkt dat een uitdrukking van de vorm $O(\epsilon_{\text{machine}})$ vaak overeenkomt met een waarde die niet meer dan 100 tot 1000 maal groter is dan $\epsilon_{\text{machine}}$. Een gekende uitzondering hierop is de Gauss eliminatie met gedeeltelijke pivotering (zie Sectie 5.3.2).

4.2.3 Achterwaartse stabiliteit

Een iets eenvoudigere, maar strictere definitie voor de stabiliteit van een algoritme $\tilde{f}$ is de volgende:

DEFINITIE 4.2.3. Een algoritme $\tilde{f}$ voor een probleem f van $\mathbb{C}^n$ naar $\mathbb{C}^m$ is **achterwaarts stabiel** voor elke $x \in \mathbb{C}^n$ indien $\tilde{f}(x) = f(\tilde{x})$ voor een bepaalde $\tilde{x}$ waarvoor geldt dat

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}).$$

In dit geval eisen we dus een exact antwoord voor een ongeveer correcte input. We merken ook op dat het stabiel of achterwaarts stabiel zijn van een algoritme $\tilde{f}$ voor een probleem f onafhankelijk is van de gehanteerde norm, want alle normen zijn equivalent (voor n en m eindig), zie Sectie 1.1.

Volgende eigenschap geeft aan dat wanneer een algoritme achterwaarts stabiel is, dan zal de relatieve nauwkeurigheid van $\tilde{f}(x)$ proportioneel zijn aan het relatieve conditiegetal $\kappa(x)$:

EIGENSCHAP 4.2.1. Laat $\tilde{f}$ een achterwaarts stabiel algoritme zijn voor een probleem f van $\mathbb{C}^n$ naar $\mathbb{C}^m$ dan

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\kappa(x)\epsilon_{\text{machine}}).$$

Bewijs. Per definitie van de achterwaartse stabiliteit bestaat er voor elke $x \in \mathbb{C}^n$ een $\tilde{x}$ en een zodanig $\tilde{f}(x) = f(\tilde{x})$ met

$$\frac{\|\tilde{x} - x\|}{\|x\|} = O(\epsilon_{\text{machine}}).$$

Uit de definitie van het absolute conditiegetal $\hat{\kappa}(x)$ zien we

$$\|f(\tilde{x}) - f(x)\| \leq \hat{\kappa}(x) \|\tilde{x} - x\| + o(\|\tilde{x} - x\|).$$

Daar $\kappa(x) = \hat{\kappa}(x) \|x\| / \|f(x)\|$ en $\tilde{f}(x) = f(\tilde{x})$, zien we dat

$$\|\tilde{f}(x) - f(x)\| \leq \kappa(x) \|f(x)\| \frac{\|\tilde{x} - x\|}{\|x\|} + o(\|\tilde{x} - x\|).$$

Er bestaat dus een $C > 0$ en een $\epsilon > 0$ zodat voor $\epsilon_{\text{machine}} \leq \epsilon$

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} \leq \kappa(x) C \epsilon_{\text{machine}} + o(\epsilon_{\text{machine}}) = O(\kappa(x) \epsilon_{\text{machine}}).$$

Merk op, daar de constante in de big-O notatie onafhankelijk moet zijn van x mogen we niet stellen dat $\|\tilde{f}(x) - f(x)\| / \|f(x)\| = O(\epsilon_{\text{machine}})$. $\square$

De bovenstaande eigenschap bevestigt onze eerdere uitspraak in verband met het conditiegetal: indien $\log_{10} \kappa(x) = k$ dan kunnen we tot k digits aan nauwkeurigheid verliezen.

4.2.4 Eenvoudige voorbeelden

In deze sectie bekijken we enkele voorbeelden van stabiele en achterwaarts stabiele algoritmes. We geven ook enkele eenvoudige regels mee die aangeven wanneer het weinig realistisch is om te verwachten dat een algoritme achterwaarts stabiel is.

Floating point operaties: In dit voorbeeld tonen we dat de 4 floating point operaties $\oplus, \ominus, \otimes$ en $\odot$ achterwaarts stabiel zijn. Laten we starten met de optelling. In dit geval is $f(x, y) = x + y$ en $\tilde{f}(x, y) = \text{fl}(x) \oplus \text{fl}(y)$. Dankzij (4.6) weten we dat

$$\text{fl}(x) \oplus \text{fl}(y) = x(1 + \epsilon_1) \oplus y(1 + \epsilon_2),$$

voor een $|\epsilon_1|, |\epsilon_2| \leq \epsilon_{\text{machine}}$. Wanneer we nu (4.7) hierop toepassen verkrijgen we

$$\text{fl}(x) \oplus \text{fl}(y) = (x(1 + \epsilon_1) + y(1 + \epsilon_2))(1 + \epsilon_3),$$

met $|\epsilon_3| \leq \epsilon_{\text{machine}}$. Kortom $\tilde{f}(x, y) = f(\tilde{x}, \tilde{y})$ met $\tilde{x} = x(1 + \epsilon_1)(1 + \epsilon_3)$ en $\tilde{y} = y(1 + \epsilon_2)(1 + \epsilon_3)$. Hieruit volgt

$$|\tilde{x} - x|/|x| = |\epsilon_1 + \epsilon_3 + \epsilon_1\epsilon_3| = O(\epsilon_{\text{machine}})$$

en

$$|\tilde{y} - y|/|y| = |\epsilon_2 + \epsilon_3 + \epsilon_2\epsilon_3| = O(\epsilon_{\text{machine}}),$$

waaruit de achterwaartse stabiliteit volgt. Voor het verschil is het bewijs volledig analoog. Voor de deling zien we dat $f(x, y) = x/y$ en $\tilde{f}(x, y) = \text{fl}(x_1) \oplus \text{fl}(y)$. Via (4.6) en (4.7) verkrijgen we

$$\text{fl}(x) \oplus \text{fl}(y) = \frac{x(1 + \epsilon_1)}{y(1 + \epsilon_2)}(1 + \epsilon_3),$$

waaruit het resultaat opnieuw volgt door $\tilde{x} = x(1 + \epsilon_1)(1 + \epsilon_3)$ en $\tilde{y} = y(1 + \epsilon_2)$. Voor het product vinden we tenslotte dat

$$\text{fl}(x) \otimes \text{fl}(y) = x(1 + \epsilon_1)y(1 + \epsilon_2)(1 + \epsilon_3),$$

waardoor we verschillende mogelijkheden hebben voor $\tilde{x}$ en $\tilde{y}$.

Plus één: Stel dat we $f(x) = x+1$ en $\tilde{f}(x) = \text{fl}(x) \oplus 1$ dan verwachten we op basis van het voorgaande waarschijnlijk dat ook dit algoritme $\tilde{f}$ achterwaarts stabiel is. Echter, wanneer we $\tilde{f}(x)$ uitwerken met behulp van (4.6) en (4.7), merken we op dat

$$\text{fl}(x) \oplus 1 = x(1 + \epsilon_1)(1 + \epsilon_2) + \epsilon_2 + 1,$$

met $|\epsilon_1|, |\epsilon_2| \leq \epsilon_{\text{machine}}$. Waardoor $\tilde{f}(x) = f(\tilde{x})$ voor $\tilde{x} = x(1 + \epsilon_1)(1 + \epsilon_2) + \epsilon_2$, wat aangeeft dat

$$|\tilde{x} - x|/|x| = |\epsilon_1 + \epsilon_2 + \epsilon_1\epsilon_2 + \epsilon_2/x| \neq O(\epsilon_{\text{machine}}),$$

omwille van de term $\epsilon_2/|x|$, die willekeurig groot kan zijn kan zijn. De big-O notatie gaat dus niet op omdat deze impliceert dat er een C moet bestaan zodat $|\epsilon_2/|x|| \leq C\epsilon_{\text{machine}}$ voor alle x . Wanneer $x \notin [-1/2, 1/2]$, dan is $|\tilde{x} - x|/|x|$ wel $O(\epsilon_{\text{machine}})$.

Het algoritme $\tilde{f}(x) = \text{fl}(x) \oplus 1$ is wel stabiel door $\tilde{x} = x(1 + \epsilon_1)(1 + \epsilon_2)$ te nemen voor $x \in [-1/2, 1/2]$, waardoor de term $\epsilon_2/|x|$ verdwijnt in de

uitdrukking voor $|\tilde{x} - x|/|x|$, wat maakt dat deze weer wel $O(\epsilon_{\text{machine}})$ is. Verder is

$$|\tilde{f}(x) - f(\tilde{x})|/|f(\tilde{x})| = |\epsilon_2|/|1 + \tilde{x}| \leq |\epsilon_2| = O(\epsilon_{\text{machine}}),$$

wat bewijst dat $\tilde{f}$ stabiel is. Merk op dat de bovenstaande discussie eveneens opgaat voor $f(x) = x + a$ en $\tilde{f}(x) = \text{fl}(x) \oplus a$ voor een willekeurige $a \neq 0$.

Inwendig en uitwendig product: Het inwendige product $f(x, y)$ van twee vectoren $x, y \in \mathbb{R}^n$ is gelijk aan $\sum_{i=1}^n x_i y_i$ en $\tilde{f}(x, y) = \oplus_{i=1}^n (\text{fl}(x_i) \otimes \text{fl}(y_i))$ is achterwaarts stabiel (zie Oefeningen). Het uitwendige product van $x, y \in \mathbb{R}^n$ is een $n \times n$ matrix A met $a_{ij} = x_i y_j$. Het voor de handliggende algoritme dat $a_{ij} = \text{fl}(x_i) \otimes \text{fl}(y_j)$ is echter niet achterwaarts stabiel. Bijvoorbeeld voor $n = 2$ verkrijgen we dat

$$\tilde{f}(x, y) = \begin{bmatrix} \bar{x}_1 \bar{y}_1 (1 + \epsilon_5) & \bar{x}_1 \bar{y}_2 (1 + \epsilon_6) \\ \bar{x}_2 \bar{y}_1 (1 + \epsilon_7) & \bar{x}_2 \bar{y}_2 (1 + \epsilon_8) \end{bmatrix},$$

met $\bar{x}_i = x_i(1 + \epsilon_i)$ en $\bar{y}_i = y_i(1 + \epsilon_{2+i})$. In het speciale geval dat $\epsilon_5 = \epsilon_6$ en $\epsilon_7 = \epsilon_8$ (of $\epsilon_5 = \epsilon_7$ en $\epsilon_6 = \epsilon_8$), kunnen we dit schrijven als $f(\tilde{x}, \tilde{y})$, maar in het algemeen kan dit niet daar $\tilde{f}(x, y)$ doorgaans geen rang één matrix is. Het algoritme is dus niet achterwaarts stabiel (maar wel stabiel). Als een algemene regel kan men stellen dat afbeeldingen f van $\mathbb{R}^n$ naar $\mathbb{R}^m$ met $m > n$, zelden achterwaarts stabiel zijn.

Sinus en cosinus: Wat kunnen we voor $f(x) = \cos(x)$ en $f(x) = \sin(x)$ verwachten? Voor de cosinus merken we op dat $\cos(x) = 1 + g(x)$ met $g(x) = o(x^2)$ daar

$$\cos(x) = \cos(0) + \sum_{n=1}^{\infty} \frac{d^n}{dx^n} \cos(0) \frac{x^n}{n!} = 1 - x^2/2! + x^4/4! - x^6/6! + \dots$$

omwille van de Taylorreeks expansie in 0. Een algoritme $\tilde{f}$ zal doorgaans overeenstemmen met $\tilde{f}(x) = 1 \oplus \tilde{g}(x)$, waardoor we uit onze discussie van de *Plus Eén* mogen besluiten dat achterwaartse stabiliteit niet opgaat. Kortom, wanneer $f(0) \neq 0$, dan is achterwaartse stabiliteit niet haalbaar. Daar $\sin(0) = 0$ hebben we misschien meer geluk met de sinus. Echter, de Taylorreeks expansie van $f(x) = \sin(x)$ geeft aan dat

$$f(x) = f(\tilde{x}) + f'(\tilde{x})(x - \tilde{x}) + o(x - \tilde{x}),$$

of nog

$$|x - \tilde{x}| \approx \left| \frac{f(x) - f(\tilde{x})}{f'(\tilde{x})} \right|.$$

Stel verder dat ons algoritme $\tilde{f}$ het exacte resultaat geeft afgerond naar het dichtbijgelegen floating point getal, d.i., $\tilde{f}(x) = \text{fl}(\sin x) = \text{fl}(f(x)) = f(x)(1 + \epsilon)$ met $|\epsilon| \leq \epsilon_{\text{machine}}$. Dan impliceert de gelijkheid $\tilde{f}(x) = f(\tilde{x})$ dat

$$|x - \tilde{x}| \approx \left| \frac{f(x) - \tilde{f}(x)}{f'(\tilde{x})} \right| = O(\epsilon_{\text{machine}} |f(x)/f'(\tilde{x})|).$$

Kortom, als de afgeleide van $f(x)$ gelijk is aan nul voor bepaalde x waarden, dan kan het algoritme $\tilde{f}$ voor f niet achterwaarts stabiel zijn.

4.2.5 Achterwaartse substitutie

In deze sectie onderzoeken we de achterwaartse stabiliteit van de achterwaartse substitutie die we doorgaans gebruiken om een lineair stelsel $Rx = b$ op te lossen, waarbij R een bovendriehoeksmatrix is, d.i.

$$\begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1m} \\ & r_{22} & & \vdots \\ & & \ddots & \vdots \\ 0 & & & r_{mm} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix},$$

met $b \in \mathbb{C}^m$ en $R \in \mathbb{C}^{m,m}$. Dit stelsel gaan we oplossen met behulp van achterwaartse substitutie zoals beschreven in Figuur 4.15. We zullen de reeks verschillen berekenen van links naar rechts (een andere volgorde is evengoed m.b.t. de achterwaartse stabiliteit, zie oefeningen).

EIGENSCHAP 4.2.2. *Het achterwaartse substitutie algoritme is achterwaarts stabiel in de zin dat de numerieke oplossing $\tilde{x}$ voldoet aan $(R + \delta R)\tilde{x} = b$ met δR een bovendriehoeksmatrix met $\|\delta R\| = O(\epsilon_{\text{machine}}) \|R\|$. In het bijzonder geldt voor elke i, j dat*

$$|\delta r_{ij}| \leq (m\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)) |r_{ij}|.$$

Merk op, deze laatste uitdrukking geeft aan dat $\|\delta R\| = O(\epsilon_{\text{machine}}) \|R\|$ voor de F_1 -norm. We bewijzen dit voor oplopende m waarden. We stellen voor de eenvoud dat de elementen van b en R floating point getallen zijn. Het bewijs voor willekeurige b en R loopt volledig analoog, maar vereist een aantal extra ϵ 's.

Achterwaartse substitutie $\sim m^2$ flops

$$\begin{aligned}
x_m &= b_m / r_{mm} \\
x_{m-1} &= (b_{m-1} - x_m r_{m-1,m}) / r_{m-1,m-1} \\
x_{m-2} &= (b_{m-2} - x_{m-1} r_{m-2,m-1} - x_m r_{m-2,m}) / r_{m-2,m-2} \\
&\vdots \\
x_j &= \left(b_j - \sum_{k=j+1}^m x_k r_{jk} \right) / r_{jj}
\end{aligned}$$

Figure 4.15: Het achterwaartse substitutie algoritme voor het oplossen van het stelsel $Rx = b$.

m = 1: In dit geval is $\tilde{x}_1 = \text{fl}(b_1) \oplus \text{fl}(r_{11}) = b_1 \oplus r_{11}$, waardoor (4.7) impliceert dat

$$\tilde{x}_1 = \frac{b_1}{r_{11}}(1 + \epsilon_1),$$

met $|\epsilon_1| \leq \epsilon_{\text{machine}}$. Daar we willen dat $\tilde{x}_1$ de exacte oplossing is voor een stelsel waarin R is geperturbeerd, hadden we graag de factor $(1 + \epsilon_1)$ in de noemer staan in plaats van in de teller. We kunnen $1 + \epsilon_1$ als $1/(1 + \epsilon'_1)$ schrijven door $\epsilon'_1 = -\epsilon_1/(1 + \epsilon_1) = -\epsilon_1 \sum_{i=0}^{\infty} (-\epsilon_1)^i = -\epsilon_1 + O(\epsilon_{\text{machine}}^2)$. Bijgevolg is

$$\tilde{x}_1 = \frac{b_1}{r_{11}(1 + \epsilon'_1)},$$

met $|\epsilon'_1| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$. Met andere woorden $(r_{11} + \delta r_{11})\tilde{x}_1 = b_1$ en $\delta r_{11} = r_{11}\epsilon'_1$, waardoor

$$|\delta r_{11}| \leq (\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)) |r_{11}|.$$

m = 2: De eerste stap is analoog als het geval $m = 1$, kortom

$$\tilde{x}_2 = \frac{b_2}{r_{22}(1 + \epsilon_1)},$$

met $|\epsilon_1| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$. Voor $\tilde{x}_1 = (b_1 \ominus (\tilde{x}_2 \otimes r_{12})) \oplus r_{11}$ bekomen we via (4.7) dat

$$\tilde{x}_1 = \frac{(b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_2))(1 + \epsilon_3)}{r_{11}}(1 + \epsilon_4),$$

met $|\epsilon_2|, |\epsilon_3|, |\epsilon_4| \leq \epsilon_{\text{machine}}$. Door net zoals in het geval van $m = 1$ de factoren $(1 + \epsilon_3)$ en $(1 + \epsilon_4)$ naar de noemer te verplaatsen door $\epsilon'_i = -\epsilon_i/(1 + \epsilon_i)$ voor $i = 3$ en 4 , stellen we vast dat

$$\tilde{x}_1 = \frac{b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_2)}{r_{11}(1 + \epsilon'_3)(1 + \epsilon'_4)},$$

met $|\epsilon'_3|, |\epsilon'_4| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$. Tot slot kunnen we dit vereenvoudigen tot

$$\tilde{x}_1 = \frac{b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_2)}{r_{11}(1 + 2\epsilon_5)},$$

met $|\epsilon_5| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$. Kortom, wanneer we r_{11}, r_{12} en r_{22} vervangen door $r_{11}(1 + 2\epsilon_5), r_{12}(1 + \epsilon_2)$ en $r_{22}(1 + \epsilon_1)$ dan is $\tilde{x}$ een exacte oplossing, d.i., $(R + \delta R)\tilde{x} = b$ met

$$\begin{bmatrix} |\delta r_{11}|/|r_{11}| & |\delta r_{12}|/|r_{12}| \\ 0 & |\delta r_{22}|/|r_{22}| \end{bmatrix} \leq \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

m = 3: De eerste twee stappen zijn analoog aan het geval $m = 2$:

$$\begin{aligned} \tilde{x}_3 &= \frac{b_3}{r_{33}(1 + \epsilon_1)}, \\ \tilde{x}_2 &= \frac{b_2 - \tilde{x}_3 r_{23}(1 + \epsilon_2)}{r_{22}(1 + 2\epsilon_3)}, \end{aligned}$$

met

$$\begin{bmatrix} 2|\epsilon_3| & 1|\epsilon_2| \\ 0 & 1|\epsilon_1| \end{bmatrix} \leq \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

Voor $\tilde{x}_1 = [(b_1 \ominus (\tilde{x}_2 \otimes r_{12})) \ominus (\tilde{x}_3 \otimes r_{13})] \oplus r_{11}$ bekomen we via (4.7) dat

$$\tilde{x}_1 = \frac{[(b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_4))(1 + \epsilon_6) - \tilde{x}_3 r_{13}(1 + \epsilon_5)](1 + \epsilon_7)}{r_{11}}(1 + \epsilon_8),$$

met $|\epsilon_i| \leq \epsilon_{\text{machine}}$ voor $i = 4, \dots, 8$. Door ϵ'_7 en ϵ'_8 in te voeren, verkrijgen we dat

$$\tilde{x}_1 = \frac{(b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_4))(1 + \epsilon_6) - \tilde{x}_3 r_{13}(1 + \epsilon_5)}{r_{11}(1 + \epsilon'_7)(1 + \epsilon'_8)},$$

met $|\epsilon'_7|, |\epsilon'_8| \leq \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$. Het enige probleem in deze uitdrukking is de term $b_1(1 + \epsilon_6)$ vandaar dat we ook nog eens ϵ'_6 invoeren om te komen tot

$$\tilde{x}_1 = \frac{(b_1 - \tilde{x}_2 r_{12}(1 + \epsilon_4)) - \tilde{x}_3 r_{13}(1 + \epsilon_5)(1 + \epsilon'_6)}{r_{11}(1 + \epsilon'_6)(1 + \epsilon'_7)(1 + \epsilon'_8)}.$$

Kortom, $\tilde{x}$ is de exacte oplossing van $(R + \delta R)\tilde{x} = b$ met

$$\begin{bmatrix} |\delta r_{11}|/|r_{11}| & |\delta r_{12}|/|r_{12}| & |\delta r_{13}|/|r_{13}| \\ & |\delta r_{22}|/|r_{22}| & |\delta r_{23}|/|r_{23}| \\ & & |\delta r_{33}|/|r_{33}| \end{bmatrix} \leq \begin{bmatrix} 3 & 1 & 2 \\ & 2 & 1 \\ & & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

m > 3: In het algemene geval verloopt alles gelijkaardig als voor $m = 3$, waardoor men kan tonen dat $\tilde{x}$ de oplossing is van $(R + \delta R)\tilde{x} = b$ waarbij $|\delta r_{ij}|/|r_{ij}|$ begrensd wordt door het (i, j) -de element van onderstaande matrix (voor $m = 5$):

$$\begin{bmatrix} 5 & 1 & 2 & 3 & 4 \\ & 4 & 1 & 2 & 3 \\ & & 3 & 1 & 2 \\ & & & 2 & 1 \\ & & & & 1 \end{bmatrix} \epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2).$$

Dit patroon komt als volgt tot stand. De vermenigvuldigingen $\tilde{x}_k r_{jk}$ veroorzaken $\epsilon_{\text{machine}}$ perturbaties volgens het patroon

$$\otimes : \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ & 0 & 1 & 1 & 1 \\ & & 0 & 1 & 1 \\ & & & 0 & 1 \\ & & & & 0 \end{bmatrix}.$$

De delingen door r_{kk} resulteren in $\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$ perturbaties op de diagonaal

$$\oplus : \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ & 1 & 0 & 0 & 0 \\ & & 1 & 0 & 0 \\ & & & 1 & 0 \\ & & & & 1 \end{bmatrix}.$$

Terwijl de verschillen (doordat we deze van links naar rechts uitvoerde) ervoor zorgen dat we initieel $(1 + \epsilon_i)$ factoren hebben op de zelfde posities als waar de producten voor een perturbatie zorgde, maar deze worden omgezet in $1/(1 + \epsilon'_i)$ factoren die elk een $\epsilon_{\text{machine}} + O(\epsilon_{\text{machine}}^2)$ perturbatie veroorzaken in de noemer (d.i., op de diagonaal), als ook op elke rechtse term. Het patroon ziet er dus uit als

$$\ominus : \begin{bmatrix} 4 & 0 & 1 & 2 & 3 \\ & 3 & 0 & 1 & 2 \\ & & 2 & 0 & 1 \\ & & & 1 & 0 \\ & & & & 0 \end{bmatrix}.$$

4.2.6 Matrixvermenigvuldiging

Volgende eigenschap geeft aan dat het product tussen een unitaire matrix Q en een willekeurige matrix $A \in \mathbb{C}^{m,n}$ achterwaarts stabiel is. De notatie $\widetilde{QA}$ geeft het floating point resultaat aan van het product tussen Q en A wanneer we het matrixproduct op de voor de handliggende manier berekenen.

EIGENSCHAP 4.2.3. *Laat $Q \in \mathbb{C}^{m,m}$ een unitaire matrix zijn en $A \in \mathbb{C}^{m,n}$, dan bestaat er een $\delta A \in \mathbb{C}^{m,n}$ zodat $\widetilde{QA} = Q(A + \delta A)$ met $\|\delta A\| = \|A\| O(\epsilon_{\text{machine}})$.*

Bewijs. We zoeken een δA zodat $\widetilde{QA} = Q(A + \delta A)$ met $\|\delta A\| = \|A\| O(\epsilon_{\text{machine}})$. Neem $\delta A = Q^*(\widetilde{QA} - QA)$ dan is

$$\|\delta A\|_2 = \left\| \widetilde{QA} - QA \right\|_2,$$

omwille van Eigenschap 2.1.5. We moeten dus tonen dat de *voorwaartse fout* voldoende klein is. Via (4.6) en (4.7) kunnen we eenvoudig tonen dat

$$\oplus_{k=1}^m (\text{fl}(q_{ik}) \otimes \text{fl}(a_{kj})) = \sum_k q_{ik} a_{kj} (1 + mO(\epsilon_{\text{machine}})).$$

Kortom, de fout op $\sum_k q_{ik} a_{kj}$, wat het (i, j) -de element van $\widetilde{QA} - QA$ weergeeft, is begrensd door $mO(\epsilon_{\text{machine}})(\sum_k |a_{kj}|)$ daar $|q_{ik}| < 1$ (want Q is unitair). Nu is

$$\begin{aligned} mO(\epsilon_{\text{machine}}) \left(\sum_k |a_{kj}| \right) &\leq mO(\epsilon_{\text{machine}}) \left(\sum_{k,j} |a_{kj}| \right) \\ &= mO(\epsilon_{\text{machine}}) \|A\|_{F_1} \\ &\leq mO(\epsilon_{\text{machine}}) \|A\|_2, \end{aligned}$$

daar alle matrixnormen equivalent zijn (zie Sectie 1.3). Bijgevolg is

$$\|\delta A\|_2 = \|\widetilde{QA} - QA\|_2 \leq K \|\widetilde{QA} - QA\|_{F_1} \leq m^3 O(\epsilon_{\text{machine}}) \|A\|_2,$$

voor een bepaalde constante K , die mogelijk ook afhankelijk is van m , waardoor het bewijs compleet is. $\square$

Merk op dat de m^3 in het bovenstaande bewijs geen probleem vormt daar de constante in de big-O notatie niet moet gelden voor alle m (maar wel voor alle A). Met behulp van bovenstaande eigenschap kunnen we ook het volgende resultaat aantonen. De notatie $\widetilde{Q_k \dots Q_1 A}$ geeft aan dat we eerst $\widetilde{Q_1 A}$ berekenen, vervolgens $\widetilde{Q_2 Q_1 A}$, etc.

EIGENSCHAP 4.2.4. *Laat $Q_1, \dots, Q_k \in \mathbb{C}^{m,m}$ unitaire matrices zijn en $A \in \mathbb{C}^{m,n}$, dan bestaat er een $\delta A \in \mathbb{C}^{m,n}$ zodat $\widetilde{Q_k \dots Q_1 A} = Q_k \dots Q_1 (A + \delta A)$ met $\|\delta A\| = \|A\| O(\epsilon_{\text{machine}})$.*

Bewijs. Voor $k = 1$ is het resultaat reeds bewezen door de voorgaande eigenschap. We bewijzen de eigenschap per inductie op k . Laat $B = Q_k \dots Q_1 A$ en stel dat $\tilde{B} = Q_k \dots Q_1 (A + \delta A_k)$ met $\|\delta A_k\| = O(\epsilon_{\text{machine}}) \|A\|$ per inductie. We moeten tonen dat $\widetilde{Q_{k+1} \tilde{B}} = Q_{k+1} Q_k \dots Q_1 (A + \delta A)$ met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$.

Uit de voorgaande eigenschap weten we dat $\widetilde{Q_{k+1} \tilde{B}} = Q_{k+1}(\tilde{B} + \delta \tilde{B})$ met

$$\|\delta \tilde{B}\| = O(\epsilon_{\text{machine}}) \|\tilde{B}\| = O(\epsilon_{\text{machine}}) \|A + \delta A_k\| = O(\epsilon_{\text{machine}}) \|A\|,$$

omwille van Eigenschap 2.1.5. Verder is

$$\begin{aligned} \widetilde{Q_{k+1} \tilde{B}} &= Q_{k+1}(\tilde{B} + \delta \tilde{B}) \\ &= Q_{k+1}(Q_k \dots Q_1 (A + \delta A_k) + \delta \tilde{B}) \\ &= Q_{k+1} Q_k \dots Q_1 (A + \delta A_k + Q_1^* \dots Q_k^* \delta \tilde{B}). \end{aligned}$$

Door $\delta A = \delta A_k + Q_1^* \dots Q_k^* \delta \tilde{B}$ bekomen we het gevraagde daar $\|\delta A\| \leq \|\delta A_k\| + \|\delta \tilde{B}\| = O(\epsilon_{\text{machine}}) \|A\|$. $\square$

Het is belangrijk op te merken dat het product tussen een gegeven matrix $B \in \mathbb{C}^{m,m}$ en $A \in \mathbb{C}^{m,n}$ in het algemeen niet achterwaarts stabiel is. Neem bijvoorbeeld voor B een rang één matrix, d.i., $B = xy^*$. Dan heeft $BA = x(y^*A)$ eveneens rang één voor gelijk welke matrix A . Echter, het floating point resultaat $\widetilde{BA}$ zijn rang is meestal niet gelijk aan één. Wanneer de matrix B inverteerbaar is, dan kunnen we wel aantonen dat er een $\delta A \in \mathbb{C}^{m,n}$ bestaat zodat $\widetilde{BA} = B(A + \delta A)$ met $\|\delta A\| = \|A\| O(\kappa(B)\epsilon_{\text{machine}})$ (zie oefeningen). Kortom, het product $\widetilde{BA}$ is in wezen wel achterwaarts stabiel, maar wanneer het conditiegetal $\kappa(B)$ van B groot is dan kan het resultaat erg onnauwkeurig zijn.

4.2.7 Householder triangularisatie

In deze sectie bespreken we de achterwaartse stabiliteit van de QR -factorisatie wanneer we deze berekenen met behulp van Householder reflecties zoals beschreven in Sectie 2.4.2. Op het eerste zicht lijkt achterwaartse stabiliteit weinig haalbaar, want we wensen een $A + \delta A$ te vinden, zodat de berekende $\tilde{Q}$ en $\tilde{R}$ voldoen aan de vergelijking

$$(A + \delta A) = \tilde{Q}\tilde{R},$$

met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$. De matrix $\tilde{Q}$ is echter niet exact unitair, terwijl de Q -matrix van de QR -factorisatie van gelijk welke matrix B (en dus ook van $A + \delta A$) wel exact unitair is. We moeten echter opmerken dat we de matrix $\tilde{Q}$ niet hoeven te construeren. In Sectie 2.4.2 zagen we dat $Q = P_1 \dots P_n$, waarbij P_k een Householder reflectie matrix is. Kortom, $P_k = I - 2v_k v_k^* / (v_k^* v_k)$ voor een bepaalde vector $\hat{v}_k$. Het volstaat dus dat we de vectoren $\hat{v}_1, \dots, \hat{v}_n$ berekenen om een impliciete weergave van Q te hebben. Indien we $\tilde{v}_k$ noteren voor de floating point berekening van v_k , voor $k = 1, \dots, n$, dan zullen we met $\tilde{Q}$ verwijzen naar de *exact unitaire* matrix die gelijk is aan

$$\tilde{Q} = \prod_{k=1}^n P_k = \prod_{k=1}^n \left(I - \frac{2}{\tilde{v}_k^* \tilde{v}_k} \tilde{v}_k \tilde{v}_k^* \right). \quad (4.8)$$

Hierdoor behoort achterwaartse stabiliteit weer wel tot de mogelijkheden. Volgende eigenschap geeft aan dat achterwaartse stabiliteit inderdaad geldt met deze definitie van $\tilde{Q}$ (we bewijzen deze niet).

EIGENSCHAP 4.2.5. *Laat $\tilde{v}_1, \dots, \tilde{v}_n \in \mathbb{C}^m$ en $\tilde{R} \in \mathbb{C}^{n,n}$ de output zijn van Algoritme 2.5 met $A \in \mathbb{C}^{m,n}$ als input, dan*

$$(A + \delta A) = \tilde{Q}\tilde{R},$$

met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$ en $\tilde{Q}$ gegeven door (4.8).

De QR -factorisatie van een matrix is doorgaans geen doel op zich. Zo zagen we in Sectie 2.5.1 dat we deze kunnen gebruiken om een linear stelsel $Ax = b$ op te lossen door middel van volgende drie stappen:

1. Bereken de QR factorisatie van A .
2. Bepaal de vector $y = \tilde{Q}^*b$.
3. Los het stelsel $\tilde{R}x = \tilde{y}$ op door achterwaartse substitutie.

Merk op dat we $\tilde{Q}$ hanteren in stap 2 en $\tilde{R}$ en $\tilde{y}$ in stap 3, daar we verder werken met de berekende matrices $\tilde{Q}$, $\tilde{R}$ en de vector $\tilde{y}$. Verder kunnen we, door middel van Algoritme 2.6, stap 2 uitvoeren zonder dat we $\tilde{Q}$ expliciet moeten uitrekenen.

Uit de voorgaande eigenschap weten we reeds dat er een δA bestaat zodat

$$(A + \delta A) = \tilde{Q}\tilde{R}, \tag{4.9}$$

met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$. Wanneer we $\tilde{y} = \tilde{Q}^*b$ berekenen met behulp van Algoritme 2.6 dan kan men eveneens aantonen dat er een $\delta\tilde{Q}$ bestaat zodat

$$(\tilde{Q} + \delta\tilde{Q})\tilde{y} = b, \tag{4.10}$$

met $\|\delta\tilde{Q}\| = O(\epsilon_{\text{machine}})$, daar $\|\tilde{Q}\|_2 = 1$. Tot slot weten we uit Eigenschap 4.2.2 dat er een $\delta\tilde{R}$ bestaat zodat

$$(\tilde{R} + \delta\tilde{R})\tilde{x} = \tilde{y}, \tag{4.11}$$

met $\|\delta\tilde{R}\| = O(\epsilon_{\text{machine}}) \|\tilde{R}\|$. Deze drie vergelijkingen kunnen we combineren om te komen tot volgende eigenschap:

EIGENSCHAP 4.2.6. *Laat $A \in \mathbb{C}^{m,m}$ en $b \in \mathbb{C}^m$. De oplossing van het stelsel $Ax = b$ met behulp van de QR factorisatie en Householder reflecties is achterwaarts stabiel, waardoor er een ΔA bestaat zodat $(A + \Delta A)\tilde{x} = b$, met $\|\Delta A\| = O(\epsilon_{\text{machine}}) \|A\|$.*

Bewijs. Vergelijkingen (4.10) en (4.11) samen geven aan dat

$$b = (\tilde{Q} + \delta\tilde{Q})(\tilde{R} + \delta\tilde{R})\tilde{x} = [\tilde{Q}\tilde{R} + \tilde{Q}(\delta\tilde{R}) + (\delta\tilde{Q})\tilde{R} + (\delta\tilde{Q})(\delta\tilde{R})]\tilde{x}.$$

Omwillen van (4.9) zien we dat $b = (A + \Delta A)\tilde{x}$ met

$$\Delta A = \delta A + \tilde{Q}(\delta\tilde{R}) + (\delta\tilde{Q})\tilde{R} + (\delta\tilde{Q})(\delta\tilde{R}),$$

met

$$\|\Delta A\| \leq \|\delta A\| + \|\tilde{Q}(\delta\tilde{R})\| + \|(\delta\tilde{Q})\tilde{R}\| + \|(\delta\tilde{Q})(\delta\tilde{R})\|.$$

Het volstaat dus te tonen dat elk van deze vier termen $O(\epsilon_{\text{machine}}) \|A\|$ is (van de eerste weten we dit reeds dankzij (4.9)). Daar $\tilde{Q}$ exact unitair is en $\tilde{Q}\tilde{R} = A + \delta A$, hebben we

$$\|\tilde{R}\| \leq \|\tilde{Q}^*\| \|A + \delta A\| \leq \|\tilde{Q}^*\| (\|A\| + \|\delta A\|) = \|A\| O(1).$$

De term $O(1)$ is gelijk aan $1 + O(\epsilon_{\text{machine}})$ indien we de 2-norm gebruiken, want in dat geval is $\|\tilde{Q}^*\|_2 = 1$. Voor een willekeurige andere matrixnorm $\|\cdot\|_\alpha$ weten we uit de equivalentie van de matrixnormen dat er een K bestaat (die afhankelijk is van m , maar niet van Q), zodat $\|\tilde{Q}^*\|_\alpha \leq K \|\tilde{Q}^*\|_2 = K$.

De uitdrukking voor $\|\tilde{R}\|$ impliceert dat

$$\|\tilde{Q}(\delta\tilde{R})\| \leq \|\tilde{Q}\| \frac{\|\delta\tilde{R}\|}{\|\tilde{R}\|} \|\tilde{R}\| = O(\epsilon_{\text{machine}}) \|A\|,$$

en

$$\|(\delta\tilde{Q})\tilde{R}\| \leq \|\delta\tilde{Q}\| \|\tilde{R}\| = O(\epsilon_{\text{machine}}) \|A\|.$$

De laatste term $\delta\tilde{Q}\delta\tilde{R}$ kunnen we analoog afhandelen door

$$\|(\delta\tilde{Q})(\delta\tilde{R})\| \leq \|\delta\tilde{Q}\| \frac{\|\delta\tilde{R}\|}{\|\tilde{R}\|} \|\tilde{R}\| = O(\epsilon_{\text{machine}}^2) \|A\|.$$

□

OEFENING 4.2.1. Stabiliteit

1. Ga na of volgende algoritmen stabiel of achterwaarts stabiel zijn:
 - (a) Input: $x \in \mathbb{C}$; $f(x) = 2x$ en $\tilde{f}(x) = \text{fl}(x) \oplus \text{fl}(x)$.
 - (b) Input: $x \in \mathbb{C}$; $f(x) = x^2$ en $\tilde{f}(x) = \text{fl}(x) \otimes \text{fl}(x)$.
 - (c) Input: $x \in \mathbb{C} \setminus \{0\}$; $f(x) = 1$ en $\tilde{f}(x) = \text{fl}(x) \oplus \text{fl}(x)$.
 - (d) Input: $x \in \mathbb{C}$; $f(x) = 0$ en $\tilde{f}(x) = \text{fl}(x) \ominus \text{fl}(x)$.
2. Toon aan dat het inwendige product $\sum_{i=1}^m x_i y_i$ achterwaarts stabiel is voor $x, y \in \mathbb{C}^m$, waarbij zowel x als y input parameters zijn. Wat als enkel x een input parameter is?
3. Toon aan dat het matrix-vector product Ab achterwaarts stabiel is met $A \in \mathbb{C}^{m,n}$ als input en $b \in \mathbb{C}^n$ vast. Met andere woorden, toon aan dat er een δA bestaat zodat $(A + \delta A)b = \tilde{A}b$ met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$. Maak gebruik van de voorgaande eigenschap.
4. Toon aan dat de achterwaartse substitutie eveneens achterwaarts stabiel is indien we de verschillen van rechts naar links uitrekenen. Pas eerst het bewijs aan voor $m = 3$, geef daarna aan hoe het patroon voor algemene m hierdoor wijzigt.
5. Laat $B \in \mathbb{C}^{m,m}$ een inverteerbare matrix zijn en $A \in \mathbb{C}^{m,n}$, bewijs dan dat er een $\delta A \in \mathbb{C}^{m,n}$ bestaat zodat $\widetilde{BA} = B(A + \delta A)$ met $\|\delta A\| = \|A\| O(\kappa(B)\epsilon_{\text{machine}})$ en $\kappa(B) = \|B^{-1}\| \|B\|$. Gebruik dezelfde redenering als in Eigenschap 4.2.3 en merk op dat

$$\left| \sum_k b_{ik} a_{kj} \right| \leq \sum_k |b_{ik}| |a_{kj}| \leq \left(\sum_{i,k} |b_{ik}| \right) \left(\sum_{k,j} |a_{kj}| \right).$$

GAUSS ELIMINATIE

In dit laatste hoofdstuk bespreken we de Gauss eliminatie die veelal in de praktijk gebruik wordt voor het oplossen van lineaire stelsels $Ax = b$ met $A \in \mathbb{C}^{m,m}$. Deze kwam reeds deels aan bod in de cursus Lineaire Algebra (d.i., onder de hoofding *Gauss reductie*). We tonen eerst dat de Gauss eliminatie eigenlijk overeenstemt met een LU factorisatie, wat aangeeft dat we A schrijven als $A = LU$ met L een beneden- en R een bovendriehoeksmatrix. We beperken ons steeds tot vierkante matrices.

5.1 LU factorisatie

De Gauss eliminatie zal de matrix $A \in \mathbb{C}^{m,m}$ omvormen tot een bovendriehoeksmatrix $U \in \mathbb{C}^{m,m}$ door kolom per kolom nullen te introduceren, volledig analoog aan de QR factorisatie op basis van de Householder reflecties of Givens rotaties (zie Secties 2.4.2 en 2.4.3). Tijdens de eerste stap bepalen we een matrix $L_1 \in \mathbb{C}^{m,m}$ zodat de eerste kolom van $L_1 A$ een veelvoud is van e_1 :

$$L_1 A = \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix},$$

Daarna bepalen we $L_2 \in \mathbb{C}^{m,m}$ zodat

$$L_2 L_1 A = \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

Deze procedure herhalen we tot we de matrices $L_1, \dots, L_{m-1}$ hebben bepaald zodat $U = L_{m-1} \dots L_2 L_1 A$ een bovendriehoeksmatrix is, net zoals bij de QR

factorisatie met dat verschil dat de matrices $L_1, \dots, L_{m-1}$ niet unitair zijn. Waaruit volgt dat

$$A = \underbrace{L_1^{-1} L_2^{-1} \dots L_{m-1}^{-1}}_L U.$$

We zullen zien dat L een benedendriehoeksmatrix is die we kunnen bepalen zonder veel rekenwerk.

Om de vereiste nullen te creëren in de eerste kolom van A gaan we α_i keer de eerste rij van A optellen bij de i -de rij van A . Wanneer we $\alpha_i = -a_{i1}/a_{11}$ dan verschijnt er een nul op positie $(i, 1)$. We kunnen dit idee in matrixvorm weergeven als

$$\underbrace{\begin{bmatrix} 1 & & & & \\ -a_{21}/a_{11} & 1 & & & \\ -a_{31}/a_{11} & & 1 & & \\ \vdots & & & \ddots & \\ -a_{m1}/a_{11} & & & & 1 \end{bmatrix}}_{L_1} A = \begin{bmatrix} \times & \times & \times & \dots & \times \\ 0 & \times & \times & \dots & \times \\ 0 & \times & \times & \dots & \times \\ \vdots & \vdots & \vdots & \ddots & \times \\ 0 & \times & \times & \times & \times \end{bmatrix}.$$

Noteer de tweede kolom van $L_1 A$ als x_2 met als i -de element x_{i2} . De volgende stap bestaat erin de tweede rij van $L_1 A$ een aantal maal op te tellen bij rij i voor $i = 3, \dots, m$, zodat element $(i, 2)$ eveneens nul wordt. We kunnen dit eenvoudig doen door rij 2 precies $-x_{i2}/x_{22}$ keer op te tellen bij rij i . Merk op, aangezien element $(2, 1)$ van $L_1 A$ gelijk is aan nul, blijven de nullen op de eerste kolom behouden. In matrixvorm gaan we $L_1 A$ voorvermenigvuldigen met

$$L_2 = \begin{bmatrix} 1 & & & & \\ & 1 & & & \\ & -x_{32}/x_{22} & 1 & & \\ & \vdots & & \ddots & \\ & -x_{m2}/x_{22} & & & 1 \end{bmatrix}.$$

Wanneer we x_k in het algemeen noteren als de k -de kolom van $L_{k-1} \dots L_1 A$ met als i -de element x_{ik} en we stellen $\ell_{ik} = x_{ik}/x_{kk}$ dan ziet de matrix L_k

eruit als

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_{mk} & & & 1 \end{bmatrix}.$$

Nu we de matrices $L_1, \dots, L_{m-1}$ beschreven hebben, gaan we kijken hoe we de matrix $L = L_1^{-1} \dots L_{m-1}^{-1}$ kunnen bepalen zonder dat we de matrices $L_1, \dots, L_{m-1}$ expliciet moeten opstellen. Laten we eerst de vectoren ℓ_k als volgt definiëren:

$$\ell_k = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k+1,k} \\ \vdots \\ \ell_{mk} \end{bmatrix}.$$

Dan kunnen we L_k schrijven als

$$L_k = I - \ell_k e_k^*.$$

Door op te merken dat $e_k^* \ell_k = 0$ zien we dat

$$(I - \ell_k e_k^*)(I + \ell_k e_k^*) = I - \ell_k \underbrace{e_k^* \ell_k}_0 e_k^* = I,$$

wat aangeeft dat

$$L_k^{-1} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & \ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & \ell_{mk} & & & 1 \end{bmatrix}.$$

Het berekenen van de inverse vereist dus helemaal geen rekenwerk. We tonen nu, per inductie, dat

$$L_1^{-1} \dots L_k^{-1} = \begin{bmatrix} 1 & & & & \\ \ell_{21} & 1 & & & \\ \vdots & \ddots & \ddots & & \\ \ell_{k+1,1} & \dots & \ell_{k+1,k} & 1 & \\ \vdots & \ddots & \vdots & & \ddots \\ \ell_{m1} & \dots & \ell_{mk} & & 1 \end{bmatrix} = I + \sum_{i=1}^k \ell_i e_i^*.$$

Voor $k = 1$ is de bovenstaande uitdrukking duidelijk correct. Verder merken we op dat

$$\begin{aligned} \left(\prod_{i=1}^k L_i^{-1} \right) L_{k+1}^{-1} &= (I + \sum_{i=1}^k \ell_i e_i^*) (I + \ell_{k+1} e_{k+1}^*) \\ &= I + \sum_{i=1}^k \ell_i e_i^* + \ell_{k+1} e_{k+1}^* + \sum_{i=1}^k \underbrace{\ell_i e_i^* \ell_{k+1}}_0 e_{k+1}^* \\ &= I + \sum_{i=1}^{k+1} \ell_i e_i^*, \end{aligned}$$

waaruit het gestelde volgt. We mogen dus besluiten dat

$$L = L_1^{-1} \dots L_{m-1}^{-1} = \begin{bmatrix} 1 & & & & \\ \ell_{21} & 1 & & & \\ \ell_{31} & \ell_{32} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{m1} & \ell_{m2} & \dots & \ell_{m,m-1} & 1 \end{bmatrix},$$

waarbij $\ell_{ik} = x_{ik}/x_{kk}$ was en x_{ik} het i -de element van de vector x_k , die op zijn beurt de k -de kolom van de matrix $L_k \dots L_1 A$ weergaf. Dit geeft aanleiding tot het algoritme in Figuur 5.16. Merk op, in deze figuur worden ook de elementen onder de hoofddiagonaal van U berekent, hoewel deze allemaal gelijk aan nul zijn. In de praktijk zal men de matrix A overschrijven met de matrices L en U om geheugen uit te sparen (waarbij we de elementen op de hoofddiagonaal van L niet hoeven op te slaan).

De complexiteit van dit algoritme wordt duidelijk bepaald door de laatste lijn uit de code, deze vergt

Gauss eliminatie zonder pivoting $\sim 2m^3/3$ flops $U = A, L = I$ for $k = 1$ to $m - 1$ $L_{k+1:m,k} = U_{k+1:m,k}/U_{k,k}$ $U_{k+1:m,k:m} = U_{k+1:m,k:m} - L_{k+1:m,k}U_{k,k:m}$
--

Figure 5.16: LU factorisatie van A .

- $(m - k)(m - k + 1)$ flops voor de uitwendige producten en
- $(m - k)(m - k + 1)$ flops voor het berekenen van de verschillen.

In totaal bedraagt het aantal flops dus plus minus

$$2 \sum_{k=1}^{m-1} (m - k)(m - k + 1) \sim 2m^3 - 4m \left(\sum_{k=1}^{m-1} k \right) + 2 \left(\sum_{k=1}^{m-1} k^2 \right) \sim \frac{2}{3}m^3,$$

waarbij we gebruik hebben gemaakt van de gelijkheid $\sum_{k=1}^m k^2 = m(m + 1)(2m + 1)/6 \sim m^3/3$.

Deze LU factorisatie van A zal falen van zodra één van de elementen $U_{k,k}$ gelijk is aan nul (voor $k < m$). We bewijzen volgende eigenschap met betrekking tot het bestaan van de LU factorisatie in de oefeningen. In deze eigenschap is L niet enkel een bovendriehoeksmatrix, maar zijn de diagonaalelementen van L eveneens allemaal gelijk aan één.

EIGENSCHAP 5.1.1. *Een matrix $A \in \mathbb{C}^{m,m}$ heeft een LU factorisatie indien $\det(A_{1:k,1:k}) \neq 0$ voor alle $k = 1, \dots, m - 1$. Indien A inverteerbaar is, dan is de LU factorisatie eveneens uniek.*

5.2 Oplossen van lineaire stelsels

Stel dat $A \in \mathbb{C}^{m,m}$ een matrix is met rang m (A is dus inverteerbaar) en dat we het lineaire stelsel $Ax = b$ wensen op te lossen met $b \in \mathbb{C}^m$. Dan kunnen we eveneens gebruik te maken van de LU factorisatie van A en deze vergelijking herschrijven als $LUx = b$, of nog

$$Ly = b, \quad \text{en} \quad Ux = y.$$

Lineaire stelsels via LU factorisatie $\sim 2m^3/3$ flops (via Gauss eliminatie)

1. Bepaal de LU factorisatie van A .
2. Los het stelsel $Ly = b$ op (via voorwaartse substitutie).
3. Los het stelsel $Ux = y$ op (via achterwaartse substitutie).

Figure 5.17: LU factorisatie voor het oplossen van een lineair stelsel.

We lossen dus eerst het stelsel $Ly = b$ op via voorwaartse substitutie, daar L een benedendriehoeksmatrix is. Daarna gebruiken we achterwaartse substitutie om het stelsel $Ux = y$ op te lossen (zie Sectie 4.2.3). Samengevat kunnen we $Ax = b$ dus oplossen door het algortime in Figuur 5.17.

De voorwaartse en achterwaartse substitutie vergen beide slechts $O(m^2)$ werk, waardoor het bepalen van de LU factorisatie doorslaggevend is voor de totale hoeveelheid rekenwerk. De LU factorisatie vergt $2m^3/3$ flops, wat maakt dat het oplossen van een stelsel lineaire vergelijkingen 2 maal sneller gaat met Gauss eliminatie dan via de QR factorisatie uit Sectie 2.5.1. Deze laatste is echter wel achterwaarts stabiel (zie Sectie 4.2.7) en de vraag is dan ook of dit eveneens opgaat voor de het oplossen van een lineair stelsel via de LU factorisatie. Van de achterwaartse substitutie weten we reeds dat deze achterwaarts stabiel is en volledig analoog kan men ook bewijzen dat de voorwaartse substitutie achterwaarts stabiel is. Volgende eigenschap werpt enig licht op de vraag of dit ook geldt voor de LU factorisatie in stap 1:

EIGENSCHAP 5.2.1. *Laat $A = LU$ een LU factorisatie zijn van $A \in \mathbb{C}^{m,m}$ en laat $\tilde{L} \in \mathbb{C}^{m,m}$ en $\tilde{U} \in \mathbb{C}^{m,m}$ de output zijn van Algoritme 5.16 met A als input, dan*

$$A + \delta A = \tilde{L}\tilde{U},$$

$$\text{met } \|\delta A\| = O(\epsilon_{\text{machine}}) \|L\| \|U\|.$$

Het is hierbij belangrijk op te merken dat $\|\delta A\| = O(\epsilon_{\text{machine}}) \|L\| \|U\|$, terwijl we voor achterwaartse stabiliteit eisen dat $\|\delta A\| = O(\epsilon_{\text{machine}}) \|A\|$. Kortom indien $\|L\| \|U\| = O(\|A\|)$ dan is dit algoritme achterwaarts stabiel.

Het volgende voorbeeld toont echter aan dat $\|L\| \|U\|$ willekeurig veel groter kan zijn dan $\|A\|$. Neem

$$A = \begin{bmatrix} 10^{-n} & 1 \\ 1 & 1 \end{bmatrix},$$

met $n \geq 1$ een geheel getal. Dan is $\|A\|_\infty = 2$ terwijl

$$A = \underbrace{\begin{bmatrix} 1 & 0 \\ 10^n & 1 \end{bmatrix}}_L \underbrace{\begin{bmatrix} 10^{-n} & 1 \\ 0 & 1 - 10^n \end{bmatrix}}_U,$$

wat aangeeft dat $\|L\|_\infty = 10^n + 1$ en $\|U\|_\infty = 10^n - 1$. De Gauss eliminatie zonder pivoting is in het algemeen dan ook niet achterwaarts stabiel.

5.3 Pivoting

De k -de stap van de Gauss eliminatie bestond erin om de k -de rij van $L_{k-1} \dots L_1 A$ precies $\ell_{ik} = x_{ik}/x_{kk}$ keer af te trekken van de i -de rij (met $i > k$) om $m - k$ nullen te creëren, waarbij x_{ik} element (i, k) voorstelt van de matrix $L_{k-1} \dots L_1 A$:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & x_{kk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix} \xrightarrow{x_{kk} \text{ pivot}} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & x_{kk} & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

Het element x_{kk} speelt hierbij duidelijk een belangrijke rol en we noemen x_{kk} hierdoor de **pivot**. We hadden echter net zo goed $m - k$ nullen kunnen creëren door rij $j > k$ precies x_{ik}/x_{jk} keer af te trekken van de rijen $i \geq k$ en $i \neq j$:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & x_{jk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix} \xrightarrow{x_{jk} \text{ pivot}} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & x_{jk} & \times & \times & \times \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

In dit geval noemen we het element x_{jk} de pivot. In de cursus Lineaire Algebra werd er reeds met een pivot gewerkt, er werd namelijk steeds gekozen voor een pivot die verschilde van nul. Puur wiskundig is dit inderdaad voldoende om ervoor te zorgen dat we de deling door nul vermijden. Wanneer we echter werken met floating point getallen dan is het best een zo groot mogelijke pivot te kiezen in absolute waarde zodat $|x_{ik}|/|x_{jk}| \leq 1$ (zoals verder zal blijken).

5.3.1 Permutatiematrices

Voor we gaan kijken naar de Gauss eliminatie met pivotering, voeren eerst de definitie in van een permutatiematrix.

DEFINITIE 5.3.1. Een matrix $P \in \mathbb{R}^m$ is een **permutatiematrix** wanneer deze bekomen kan worden door de rijen van de eenheidsmatrix I te permuteren. Wanneer P bekomen werd door slechts twee rijen om te wisselen noemen we P een **verwisseling**.

We zien hieronder twee voorbeelden:

$$P_a = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad \text{en} \quad P_b = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

De matrix P_a komt overeen met de permutatie $(4, 1, 2, 5, 3)$ en P_b met de permutatie $(1, 5, 3, 4, 2)$. Merk op, P_b is een verwisseling daar enkel rij 2 en rij 5 zijn omgewisseld. We kunnen elke permutatiematrix $P \in \mathbb{R}^m$ schrijven als een product van maximaal $m - 1$ verwisselingen. Bijvoorbeeld voor de matrix P_a kunnen we de permutatie $(4, 1, 2, 5, 3)$ bekomen door volgende vier verwisseling na elkaar toe te passen:

$$(1, 2, 3, 4, 5) \rightarrow (4, 2, 3, 1, 5) \rightarrow (4, 1, 3, 2, 5) \rightarrow (4, 1, 2, 3, 5) \rightarrow (4, 1, 2, 5, 3).$$

Wanneer we een matrix A links vermenigvuldigen met een permutatiematrix P , dan zal dit de rijen van A herordenen volgens de permutatie waarmee P overeenstemt. De matrix A rechts vermenigvuldigen met een permutatiematrix P zal de kolommen van A permuteren.

Verder kunnen we ook opmerken dat de inverse van een verwisseling P , gelijk is aan de matrix P zelf. Dat is ook logisch, als je i en j twee maal met elkaar verwisselt staan ze terug in de originele volgorde. De inverse van een algemene permutatiematrix is gelijk aan zijn getransponeerde, d.i., $P^{-1} = P^*$, wat maakt dat P unitair is. We kunnen dit als volgt inzien. Indien element $p_{ij} = 1$, dan is $p_{ik} = 0$ voor $k \neq j$ en $p_{kj} = 0$ voor $k \neq i$. Anderzijds is $(P^*)_{ji} = p_{ij} = 1$ en $(P^*)_{jk} = p_{kj} = 0$ voor $k \neq i$.

Het unitair zijn van P geeft aan dat $|\det(P)| = 1$. We kunnen dit ook inzien door op te merken dat het verwisselen van twee rijen de determinant van een matrix A enkel van teken doet wijzigen (zie Cursus Lineaire Algebra). Daar P bekomen werd uit de eenheidsmatrix door maximaal $m - 1$ verwisselingen is zijn determinant 1 of -1 .

5.3.2 Gauss eliminatie met gedeeltelijke pivotering

De Gauss eliminatie met (gedeeltelijke) pivotering zal tijdens de eerste stap de pivot x_{j1} kiezen, zodat $|x_{j1}| = \max_{i \geq 1} |x_{i1}|$, waarbij x_1 de eerste kolom van de matrix A voorstelt. Daarna zal rij 1 en j omgewisseld worden, door A links te vermenigvuldigen met de permutatiematrix P_1 . Merk op, P_1 is de verwisseling die rij 1 en j omwisselt. Laat y_1 de eerste kolom van $P_1 A$ voorstellen met het i -de element gelijk aan y_{i1} . Wanneer we nu $P_1 A$ links vermenigvuldigen met

$$L_1 = \begin{bmatrix} 1 & & & & \\ -\ell_{21} & 1 & & & \\ -\ell_{31} & & 1 & & \\ \vdots & & & \ddots & \\ -\ell_{m1} & & & & 1 \end{bmatrix},$$

met $\ell_{ik} = y_{i1}/y_{j1} = y_{i1}/x_{j1}$, dan zal de eerste kolom opnieuw een veelvoud zijn van e_1 :

$$\begin{bmatrix} x_{11} & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ x_{j1} & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix} \xrightarrow{P_1} \begin{bmatrix} x_{j1} & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ x_{11} & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix} \xrightarrow{L_1} \begin{bmatrix} x_{j1} & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix}.$$

Laat x_k de k -de kolom van de matrix $L_{k-1}P_{k-1} \dots L_1P_1A$ voorstellen. Stap k van de Gauss eliminatie met pivotering bestaat er dan in om eerst de pivot

x_{jk} te kiezen (met $j \geq k$), zodat $|x_{jk}| = \max_{i \geq k} |x_{ik}|$. Daarna zal rij k en j omgewisseld worden, door links met de verwisseling P_k te vermenigvuldigen. Laat verder y_k de eerste kolom van $P_k L_{k-1} P_{k-1} \dots L_1 P_1 A$ voorstellen met het i -de element gelijk aan y_{ik} . Dan zal links vermenigvuldigen met

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_{mk} & & & 1 \end{bmatrix}$$

met $\ell_{ik} = y_{ik}/y_{kk} = y_{ik}/x_{jk}$ ervoor zorgen dat de $m - k$ elementen $(k + 1, k)$ tot (m, k) gelijk worden aan nul:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & x_{kk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & x_{jk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix} \xrightarrow{P_k} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & x_{jk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & x_{kk} & \times & \times & \times \\ 0 & \times & \times & \times & \times \end{bmatrix} \xrightarrow{L_k} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & x_{jk} & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

Indien we deze methode $m - 1$ stappen herhalen, dan zien we dat

$$L_{m-1} P_{m-1} \dots L_2 P_2 L_1 P_1 A = U,$$

waarbij U een bovendriehoeksmatrix is. Merk op, de matrix P_k is een verwisseling die element k met element $j > k$ omwisselt. De matrix $L_{m-1} P_{m-1} \dots L_1 P_1$ in de bovenstaande uitdrukking is echter in het algemeen geen benedendriehoeksmatrix, dus we kunnen nog niet spreken van een LU factorisatie. We gaan de matrix U echter trachten te herschrijven als

$$U = L_{m-1} P_{m-1} \dots L_2 P_2 L_1 P_1 A = L'_{m-1} \dots L'_2 L'_1 P_{m-1} \dots P_2 P_1 A,$$

zodat

$$P_{m-1} \dots P_2 P_1 A = (L'_1)^{-1} \dots (L'_{m-1})^{-1} U,$$

indien de inverse matrices bestaan. Laten we dit even doen voor $m = 4$, dan volstaat het dat we

$$L'_3 = L_3, \quad L'_2 = P_3 L_2 P_3^{-1}, \quad L'_1 = P_3 P_2 L_1 P_2^{-1} P_3^{-1}.$$

In het algemeen zal

$$L'_k = P_{m-1} \dots P_{k+1} L_k P_{k+1}^{-1} \dots P_{m-1}^{-1}.$$

We tonen nu dat L'_k dezelfde structuur heeft als L_k . Wanneer we een matrix X van de vorm

$$\begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & x_{k'+1,k'} & 1 & & \\ & & \vdots & & \ddots & \\ & & x_{mk'} & & & 1 \end{bmatrix},$$

links vermenigvuldigen met P_k met $k > k'$, dan worden de elementen $x_{kk'}$ en $x_{jk'}$ omgewisseld. Verder zullen de enen op posities (k, k) en (j, j) verhuizen naar (j, k) en (k, j) . Als we daarna $P_k X$ rechts vermenigvuldigen met $P_k^{-1} = P_k$, dan zullen kolom k en j omgewisseld worden, waardoor de enen terug op positie (k, k) en (j, j) terecht komen. Kortom, $P_k X P_k^{-1}$ heeft dezelfde structuur als X indien $k > k'$, enkel element $x_{kk'}$ en $x_{jk'}$ zijn omgewisseld.

Hieruit mogen we besluiten dat L'_k gelijk is aan L_k indien we de verwisselingen $P_{k+1}, \dots, P_{m-1}$ enkel toepassen op de elementen $-\ell_{ik}$. Bijgevolg zal de inverse van L'_k ook bekomen worden door $-\ell_{ik}$ te vervangen door ℓ_{ik} , terwijl $(L'_1)^{-1} \dots (L'_{m-1})^{-1}$ een benedendriehoeksmatrix is die we bekomen door de k -de kolom gelijk te stellen aan de k -de kolom van $(L'_k)^{-1}$.

Dit alles impliceert dat

$$\underbrace{P_{m-1} \dots P_1}_P A = \underbrace{(L'_1)^{-1} \dots (L'_{m-1})^{-1}}_L U.$$

De matrix PA , die gelijk is aan A op een permutatie van zijn rijen na, heeft dus een LU factorisatie. Dit geeft aan dat de Gauss eliminatie met gedeeltelijke pivotering eigenlijk overeenstemt met het permuteren van de rijen van A volgens P en vervolgens Gauss eliminatie zonder pivotering toe te passen. Daar we de permutatiematrix P niet op voorhand kennen, kunnen we dit idee niet gebruiken om de Gauss eliminatie met pivotering te implementeren.

In Figuur 5.18 zien we de code van de Gauss eliminatie met gedeeltelijke pivotering die duidelijk dezelfde complexiteit heeft als deze zonder pivotering, daar opnieuw de laatste lijn bepalend is voor de complexiteit. We zien dat de

Gauss eliminatie met gedeeltelijke pivoting $\sim 2m^3/3$ flops $U = A, L = I, P = I$ **for** $k = 1$ **to** $m - 1$ Kies $j \geq k$ zodat $|U_{j,k}|$ maximaal isVerwissel rijen $U_{k,k:m}$ en $U_{j,k:m}$ Verwissel rijen $L_{k,1:k-1}$ en $L_{j,1:k-1}$ Verwissel rijen $P_{k,1:m}$ en $P_{j,1:m}$ $L_{k+1:m,k} = U_{k+1:m,k}/U_{k,k}$ $U_{k+1:m,k:m} = U_{k+1:m,k:m} - L_{k+1:m,k}U_{k,k:m}$ Figure 5.18: LU factorisatie van PA .

verschillende verwisselingen die we moeten toepassen op de elementen van L_k , om L'_k te bekomen, één voor één worden uitgevoerd.

EIGENSCHAP 5.3.1. *Laat $A \in \mathbb{C}^{m,m}$ inverteerbaar zijn, dan bestaat er een permutatiematrix P zodat PA een LU factorisatie heeft.*

Bewijs. Stel dat we tijdens stap k vaststellen dat $x_{jk} = 0$, voor alle $j \geq k$, waarbij x_{jk} het (j, k) -de element van $X = L_{k-1}P_{k-1} \dots L_1P_1A$. In dat geval zijn de eerste k elementen van de laatste $m - k + 1$ rijen van X gelijk aan nul. De matrix X heeft dus $m - k + 1$ rijen waarvan enkel de laatste $m - k$ elementen verschillen van nul. Bijgevolg moet één van deze rijen een lineaire combinatie zijn van de overige, wat aangeeft dat $\det(X) = 0$. We zien echter dat

$$\begin{aligned} |\det(X)| &= |\det(L_{k-1}P_{k-1} \dots L_1P_1A)| \\ &= |\det(A)| \prod_{i=1}^{k-1} |\det(L_i)| |\det(P_i)| = |\det(A)|, \end{aligned}$$

aangezien $|\det(P_i)| = 1$ en $\det(L_i) = 1$ (want het is een benedendriehoeksmatrix met alle diagonaalelementen gelijk aan één). We besluiten dat $\det(A) = 0$, wat in tegenspraak is met de inverteerbaarheid van A . $\square$

5.3.3 Gauss eliminatie met complete pivoting

Tijdens de k -de stap van de Gauss eliminatie met gedeeltelijke pivoting hebben we $m - k$ nullen geïntroduceerd in de k -de kolom door x_{jk} als pivot te gebruiken met $j \geq k$. We zouden echter ook in kolom $i > k$ nullen kunnen introduceren door een element uit de i -de kolom te kiezen als pivot:

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & \times & \times \\ 0 & \times & \times & x_{ji} & \times \\ 0 & \times & \times & \times & \times \end{bmatrix} \xrightarrow{x_{ji} \text{ pivot}} \begin{bmatrix} \times & \times & \times & \times & \times \\ 0 & \times & \times & 0 & \times \\ 0 & \times & \times & 0 & \times \\ 0 & \times & \times & x_{ji} & \times \\ 0 & \times & \times & 0 & \times \end{bmatrix}.$$

In dit geval zal zijn er dus $(m - k + 1)^2$ keuzes voor de pivot in plaats van slechts $(m - k + 1)$, zoals bij de gedeeltelijke pivoting. De pivot wordt dan wederom gekozen zodat deze de grootst mogelijke absolute waarde heeft. We spreken in dit geval van Gauss eliminatie met complete pivoting. Stap k bestaat er dan in dat we eerst kolom i en k verwisselen door rechts te vermenigvuldigen met een verwisseling Q_k , vervolgens rij j en k te wisselen door links met de verwisseling P_k te vermenigvuldigen en tenslotte links met L_k te vermenigvuldigen zodat de laatste $m - k$ elementen van kolom k nul worden.

Deze methode zal uiteindelijk resulteren in het bepalen van twee permutatie matrices P en Q , een benedendriehoeksmatrix L en een bovendriehoeksmatrix U zodat

$$PAQ = LU.$$

De Gauss eliminatie met complete pivoting wordt in de praktijk zelden gebruikt. Want indien we tijdens elke stap de pivot moeten bepalen met de maximale absolute waarde, dan zien we dat de selectie van de $m - 1$ pivot elementen in totaal een kost met zich meebrengt van $O(m^3)$, terwijl dit voor de gedeeltelijke pivoting slechts $O(m^2)$ was. De total kost van de Gauss eliminatie met complete pivoting is bijgevolg ook groter dan $2m^3/3$. Verder is ook de implementatie complexer.

5.3.4 Stabiliteit

Voor de Gauss eliminatie zonder pivoting gaf Eigenschap 5.2.1 reeds aan dat $\tilde{L}\tilde{U} = A + \delta A$ met $\|\delta A\| = O(\epsilon_{\text{machine}}) \|L\| \|U\|$, waarbij $\tilde{L}$ en $\tilde{U}$ de berekende

matrices voorstellen. Om te kunnen spreken van achterwaartse stabiliteit moest $\|L\| \|U\| = O(\|A\|)$, wat in het algemeen niet opging. Wanneer we gedeeltelijke pivotering toepassen zijn alle elementen van de matrix $\tilde{L}$ in absolute waarde kleiner of gelijk aan 1, waardoor $\|L\| = O(1)$ voor elke matrix norm (omwille van de equivalentie van de matrixnormen en het feit dat $\|L\|_{F_\infty} = 1$). De vraag of Gauss eliminatie met gedeeltelijke pivotering achterwaarts stabiel is, komt bijgevolg overeen met de vraag of $\|U\| = O(\|A\|)$. Wat neerkomt op de vraag of de elementen u_{ij} van U willekeurig veel groter kunnen worden dan deze van A . Laat ρ de *groeifactor* zijn voor de matrix A , die we definiëren als

$$\rho = \frac{\max_{i,j} |u_{ij}|}{\max_{i,j} |a_{ij}|},$$

dan gaat volgende eigenschap op:

EIGENSCHAP 5.3.2. *Laat $PA = LU$ een LU factorisatie zijn van $A \in \mathbb{C}^{m,m}$ en laat $\tilde{P}$, $\tilde{L}$ en $\tilde{U} \in \mathbb{C}^{m,m}$ de output zijn van Algoritme 5.18 met A als input, dan*

$$\tilde{P}A + \delta A = \tilde{L}\tilde{U},$$

met $\|\delta A\| = O(\rho \epsilon_{\text{machine}}) \|A\|$. Indien $|\ell_{ij}| < 1$ voor alle $i > j$, wat aangeeft dat er nooit een keuze was voor de pivot, dan zal $P = \tilde{P}$ voor $\epsilon_{\text{machine}}$ voldoende klein.

Deze eigenschap geeft aan dat we mogen spreken van achterwaartse stabiliteit indien de groeifactor $\rho = O(1)$ voor alle matrices $A \in \mathbb{C}^{m,m}$, waar de constante in de big-O notatie afhankelijk mag zijn van m . We zullen tonen dat $\rho \leq 2^{m-1}$ (zie oefeningen), wat maakt dat er inderdaad sprake is van achterwaartse stabiliteit. We moeten echter ook opmerken dat voor $m = 100$ en $\epsilon_{\text{machine}} = 2^{-53}$ bovenstaande eigenschap enkel garandeert dat $\|\delta A\| \leq 2^{46} \|A\|$, wat niet onmiddellijk geruststellend is. Kortom, achterwaartse stabiliteit resulteert in de praktijk niet noodzakelijk in nauwkeurige resultaten wanneer de constante in de big-O notatie snel groeit in functie van m , wat hier het geval is daar deze een exponentiele functie is van m .

Om aan te geven dat er ook matrices A bestaan waarvoor $\rho = 2^{m-1}$ gaan we trachten element $u_{mm} = 2^{m-1} \max_{i,j} |a_{ij}|$. Stel dat $\max_{i,j} |a_{ij}| = a_{mm} = 1$, dan gaan we trachten het element op positie (m, m) te verdubbelen tijdens elk van de $m - 1$ stappen van de Gauss eliminatie. Het mag duidelijk zijn

dat we het meeste kans op slagen hebben indien $|a_{ij}| = 1$ of 0 , voor alle i, j . Wanneer we de laatste kolom van A gelijk kiezen aan e , dan volstaat het alle elementen op positie (j, m) , met $j > k$, te verdubbelen tijdens stap k . We kunnen hiervoor bijvoorbeeld volgende matrix hanteren:

$$A = \begin{bmatrix} 1 & & & & 1 \\ -1 & 1 & & & 1 \\ -1 & -1 & 1 & & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix},$$

waarvoor

$$L = \begin{bmatrix} 1 & & & & \\ -1 & 1 & & & \\ -1 & -1 & 1 & & \\ -1 & -1 & -1 & 1 & \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix} \quad \text{en} \quad U = \begin{bmatrix} 1 & & & 1 \\ & 1 & & 2 \\ & & 1 & 4 \\ & & & 1 & 8 \\ & & & & 16 \end{bmatrix}.$$

Er bestaan dus wel degelijk matrices A waarvoor de groeifactor $\rho = 2^{m-1}$ en numerieke experimenten (zie oefeningen) bevestigen dat deze inderdaad kunnen resulteren in fouten die exponentieel groeien in m .

Hoewel er dus matrices A bestaan waarvoor de Gauss eliminatie met gedeeltelijke pivoting numeriek faalt, blijkt dit toch nog steeds de meest gehanteerde methode te zijn voor het oplossen van lineaire stelsels $Ax = b$. Dit heeft te maken met het feit dat Gauss eliminatie een veel ouder idee is dan de QR factorisatie en dat deze ook 2 maal sneller is dan de QR factorisatie. Daarnaast blijkt dat de meeste matrices $A \in \mathbb{C}^{m,m}$ een groeifactor ρ hebben die veel kleiner is dan 2^{m-1} . Vele experimenten lijken dan ook uit te wijzen dat de matrices A waarop de Gauss eliminatie met gedeeltelijke pivoting faalt, eerder pathologische voorbeelden zijn die men in de praktijk zelden of nooit tegenkomt.

OEFENING 5.3.1. LU factorisatie

1. Toon aan dat de Gauss eliminatie zonder pivoting succesvol is voor een matrix $A \in \mathbb{C}^{m,m}$ indien $\det(A_{1:k,1:k}) \neq 0$ voor alle $k = 1, \dots, m-1$. Maak gebruik van het feit dat de determinant van een matrix B niet wijzigt wanneer we een veelvoud van een rij van B optellen bij een andere rij van B (zie cursus Lineaire Algebra). Toon eveneens aan dat als A inverteerbaar is, dan is de LU factorisatie eveneens uniek (wanneer we eisen dat de diagonaalelementen gelijk zijn aan 1).

2. Implementeer een Matlab functie `[L,U]=lu_gauss(A)` die een LU factorisatie van A berekent door gebruik te maken van de Gauss eliminatie zonder pivotering. Implementeer eveneens een Matlab functie `[L,U]=lu_gauss_pivot(A)` voor de Gauss eliminatie met gedeeltelijke pivotering. Zorg ervoor dat het geheugengebruik zo laag mogelijk is (door onder andere A te overschrijven met L en U). Test je code op matrices A waarvoor $a_{ij} = 0$ indien $|i - j| > 10$, wat valt je op?
3. Geef aan hoe we de determinant van $A \in \mathbb{C}^{m,m}$ kunnen bepalen aan de hand van de Gauss eliminatie met gedeeltelijke pivotering.
4. Argumenteer dat de groeifactor ρ van een matrix $A \in \mathbb{C}^{m,m}$ kleiner is of gelijk aan 2^{m-1} .
5. Los in Matlab enkele lineaire stelsels $Ax = b$ op via Gauss eliminatie met gedeeltelijke pivotering, waarbij de matrix $A \in \mathbb{C}^{m,m}$ met $m = 60$ en $\rho = 2^{59}$ (zie Sectie 5.3.4 voor een voorbeeld). Treden er numerieke problemen op in je voorbeelden? Mogelijk is dit niet het geval daar A enkel gehele getallen bevat. Pas de matrix A aan (zonder ρ erg te beïnvloeden) zodat er wel grote fouten ontstaan.